

**mc**at

## Computer-Assisted Translation Tool

Version 1.0

# Codebase Comprehensive Report

Generated: 2026-07-17 07:55

Total: 7600 lines across 33 Python files

Repository: <https://github.com/skarmohdi2006/mcat>

# Table of Contents

1. Overview & Statistics
2. Entry Point
3. Core Library Modules
4. Format Handlers
5. Test Suites

Files (alphabetical within each category):

- mcat.py (21 lines)
- cat\_tool\\_\_init\_\_.py (1 lines)
- cat\_toolui.py (1818 lines)
- cat\_tooltm.py (245 lines)
- cat\_toolglossary.py (360 lines)
- cat\_toolsegmentation.py (380 lines)
- cat\_toolsuggest.py (149 lines)
- cat\_toolspellcheck.py (135 lines)
- cat\_toolqa.py (179 lines)
- cat\_toolsettings.py (46 lines)
- cat\_toolproject.py (217 lines)
- cat\_toolfile\_handler.py (113 lines)
- cat\_toolmcat\_format\_manager.py (151 lines)
- cat\_toolstate\_manager.py (252 lines)
- cat\_toolworkers.py (68 lines)
- cat\_toolformats\\_\_init\_\_.py (30 lines)
- cat\_toolformats\factory.py (55 lines)
- cat\_toolformats\\_text\_base.py (27 lines)
- cat\_toolformats>tag\_utils.py (34 lines)
- cat\_toolformats\text.py (16 lines)
- cat\_toolformats\pdf.py (77 lines)
- cat\_toolformats\docx.py (47 lines)
- cat\_toolformats\pptx.py (43 lines)
- cat\_toolformats\xlsx.py (33 lines)
- cat\_toolformats\xls.py (39 lines)
- cat\_toolformats\odf.py (102 lines)
- cat\_toolformats\mqxliif.py (67 lines)
- cat\_toolformats\sdxliiff.py (109 lines)

- cat\_tool\formats\mcatdb.py (125 lines)
- tests\test\_cat\_logic.py (101 lines)
- tests\test\_comprehensive.py (644 lines)
- tests\test\_remaining.py (890 lines)
- tests\test\_more.py (1026 lines)

# 1. Overview & Statistics

| Metric                  | Value                             |
|-------------------------|-----------------------------------|
| Total Python Files      | 33                                |
| Total Lines of Code     | 7600                              |
| Total Classes Defined   | 58                                |
| Total Functions Defined | 538                               |
| Core Library Modules    | 14                                |
| Format Handlers         | 14                                |
| Test Files              | 4                                 |
| Lines per File (avg)    | 230                               |
| GUI Framework           | PyQt5                             |
| Database Engine         | SQLite3 + sqlite-vec              |
| Fuzzy Matching          | rapidfuzz                         |
| Embedding Models        | fastembed (sentence-transformers) |
| Spell Checking          | spylls (Hunspell)                 |

## Supported File Formats

| Format                        | Handler Class   | Libraries Used               |
|-------------------------------|-----------------|------------------------------|
| Plain Text (.txt)             | TextHandler     | builtins                     |
| PDF (.pdf)                    | PdfHandler      | pdfplumber, pypdf, reportlab |
| Word (.docx)                  | DocxHandler     | python-docx                  |
| PowerPoint (.pptx/.ppsx)      | PptxHandler     | python-pptx                  |
| Excel (.xlsx)                 | XlsxHandler     | openpyxl                     |
| Excel (.xls)                  | XlsHandler      | xlrd, xlutils                |
| OpenDocument (.odt/.ods/.odp) | OdfHandler      | odfpy                        |
| memoQ XLIFF (.mqxliff)        | MqxliffHandler  | xml.etree                    |
| SDL Trados XLIFF (.sdlxliff)  | SdlxliffHandler | xml.etree                    |
| Native MCAT (.mcat.db)        | McatDbHandler   | sqlite3                      |
| PO / XLIFF / TMX / TBX / CSV  | FactoryHandler  | translate-toolkit            |

## 2. Entry Point

### mcat.py — 21 lines

- Function **main** (line 7)

```
import os
import sys
from PyQt5.QtWidgets import QApplication
from cat_tool.ui import MainWindow

def main():
    app = QApplication(sys.argv)
    win = MainWindow()
    win.show()

    if len(sys.argv) > 1:
        file_path = sys.argv[1]
        if os.path.exists(file_path):
            win.open_file_path(file_path)

    sys.exit(app.exec_())

if __name__ == "__main__":
    main()
```

## 3. Core Library

### cat\_tool\\_\_init\_\_.py — 1 lines

```
# Package marker for cat_tool
```

### cat\_toolui.py — 1818 lines

- Class **MainWindow** (line 157)
- Function **\_\_init\_\_** (line 162)
- Function **\_ensure\_seg\_worker** (line 189)
- Function **\_ensure\_emb\_worker** (line 200)
- Function **\_ensure\_fuzzy\_worker** (line 209)
- Function **\_stop\_threads** (line 219)
- Function **cleanup** (line 230)
- Function **closeEvent** (line 233)
- Function **init\_menu** (line 237)
- Function **init\_ui** (line 427)
- Function **\_on\_segmentation\_finished** (line 517)
- Function **\_on\_segmentation\_error** (line 527)
- Function **\_on\_embedding\_finished** (line 531)
- Function **\_on\_embedding\_error** (line 534)
- Function **\_on\_selection\_debounced** (line 537)
- Function **open\_file** (line 546)
- Function **open\_file\_path** (line 566)
- Function **populate\_table** (line 573)
- Function **get\_status\_text** (line 596)
- Function **set\_source\_cell\_style** (line 604)
- Function **set\_target\_cell\_style** (line 615)
- Function **\_on\_target\_cell\_changed** (line 626)
- Function **save\_file** (line 664)
- Function **save\_file\_as** (line 675)
- Function **render\_translated\_file** (line 725)
- Function **segment\_selected** (line 744)
- Function **fuzzy\_state\_changed** (line 758)
- Function **update\_table\_row\_status** (line 767)
- Function **confirm\_active\_segment** (line 778)
- Function **\_edit\_target\_cell** (line 821)
- Function **\_go\_next\_segment** (line 825)
- Function **\_go\_prev\_segment** (line 835)
- Function **\_go\_next\_untranslated** (line 845)
- Function **\_table\_context\_menu** (line 858)
- Function **\_show\_tag\_dialog\_for\_target** (line 880)
- Function **\_spellcheck\_segment** (line 884)
- Function **show\_options** (line 889)
- Function **\_apply\_settings** (line 894)
- Function **update\_fuzzy\_matches** (line 905)
- Function **\_on\_fuzzy\_matches\_ready** (line 913)
- Function **apply\_tm\_suggestion** (line 925)
- Function **update\_glossary\_matches** (line 933)
- Function **insert\_glossary\_term** (line 948)
- Function **add\_glossary\_term** (line 960)
- Function **import\_tmx\_to\_glossary** (line 975)
- Function **export\_glossary\_as\_tmx** (line 1006)
- Function **create\_collection\_dialog** (line 1024)

- Function **manage\_collections** (line 1051)
- Function **import\_collection\_tmx** (line 1228)
- Function **export\_collection\_tmx** (line 1249)
- Function **view\_glossary** (line 1292)
- Function **import\_po\_to\_tm** (line 1346)
- Function **clear\_tm** (line 1363)
- Function **\_toggle\_fuzzy** (line 1375)
- Function **\_go\_first\_segment** (line 1379)
- Function **\_go\_last\_segment** (line 1384)
- Function **show\_shortcuts** (line 1390)
- Function **\_open\_dicts\_page** (line 1394)
- Function **navigate\_to\_segment** (line 1398)
- Function **run\_qa** (line 1403)
- Function **import\_dictionary** (line 1411)
- Function **\_load\_spelling\_dict\_for\_language** (line 1432)
- Function **run\_spellcheck** (line 1442)
- Function **update\_stats** (line 1451)
- Function **close\_project** (line 1474)
- Function **new\_project** (line 1496)
- Function **open\_project** (line 1528)
- Function **\_load\_project** (line 1572)
- Function **\_load\_omegat** (line 1590)
- Function **project\_settings** (line 1608)
- Function **\_open\_project\_source\_file** (line 1623)
- Function **save\_project** (line 1638)
- Function **save\_project\_as** (line 1654)
- Function **open\_segmentation\_rules** (line 1722)
- Function **import\_omegat\_project** (line 1732)
- Function **import\_memoq\_project** (line 1758)
- Function **\_load\_memoq** (line 1771)
- Function **import\_sdlxliff** (line 1795)
- Function **\_update\_recent\_menu** (line 1805)
- Function **refresh\_collection\_list** (line 1117)
- Function **refresh\_terms** (line 1124)
- Function **on\_coll\_selected** (line 1138)
- Function **do\_create** (line 1141)
- Function **do\_delete** (line 1146)
- Function **do\_add\_term** (line 1161)
- Function **do\_remove\_term** (line 1200)
- Function **delete\_selected** (line 1316)

```
import os
import sys
```

```
parent_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
if parent_dir not in sys.path:
    sys.path.insert(0, parent_dir)
```

```
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout, QHBoxLayout,
                             QTableWidgetItem, QSplitter, QTextEditor,
                             QPushButton, QLabel, QListWidget, QListWidgetItem,
                             QFileDialog, QStatusBar, QProgressBar, QMessageBox,
                             QCheckBox, QDialog, QFormLayout, QLineEdit, QHeaderView,
                             QComboBox, QDialogButtonBox, QMenu, QScrollArea, QSpinBox)
```

```
from PyQt5.QtCore import Qt, pyqtSignal, QThread, QTimer
from PyQt5.QtGui import QColor, QFont, QKeySequence
```

```
from cat_tool.file_handler import CATFileHandler, convert_to_po
from cat_tool.formats.tag_utils import PH_L, PH_R, extract_all_tags, apply_tags, restore_tags
from cat_tool import qa
from cat_tool.state_manager import AppState
```

```

from cat_tool.workers import SegmentationWorker, EmbeddingWorker, FuzzyMatchWorker

from cat_tool.dialogs import (
    GlossaryDialog, LangDialog, NewProjectDialog, ProjectSettingsDialog,
    ShortcutsDialog, QADialog, SpellCheckDialog, SettingsDialog, TagViewDialog,
    FILE_FILTER,
)

# Cached color constants for cell styling
_SRC_FUZZY_BG = QColor("#3d2e00")
_SRC_FUZZY_FG = QColor("#ff9800")
_SRC_TRANSLATED_BG = QColor("#002b00")
_SRC_TRANSLATED_FG = QColor("#28a745")
_SRC_DEFAULT_BG = QColor("#1a1a1e")
_SRC_DEFAULT_FG = QColor("#a0a0a8")
_TGT_FUZZY_BG = QColor("#3d2e00")
_TGT_FUZZY_FG = QColor("#ff9800")
_TGT_TRANSLATED_BG = QColor("#003300")
_TGT_TRANSLATED_FG = QColor("#ffffff")
_TGT_DEFAULT_BG = QColor("#1a1a1e")
_TGT_DEFAULT_FG = QColor("#808080")

DARK_STYLESHEET = """
QMainWindow {
    background-color: #121214;
}
QWidget {
    background-color: #1e1e24;
    color: #ffffff;
    font-family: "Segoe UI", Arial, sans-serif;
    font-size: 13px;
}
QTableWidget {
    background-color: #1a1a1e;
    gridline-color: #2c2c35;
    border: 1px solid #2c2c35;
    border-radius: 4px;
}
QTableWidget::item {
    padding: 8px;
    border-bottom: 1px solid #2c2c35;
}
QTableWidget::item:selected {
    background-color: #0078d4;
    color: white;
}
QHeaderView::section {
    background-color: #25252b;
    color: #b0b0b8;
    padding: 6px;
    border: 1px solid #2c2c35;
    font-weight: bold;
}
QTextEdit {
    background-color: #1a1a1e;
    border: 1px solid #2c2c35;
    border-radius: 4px;
    padding: 6px;
    font-size: 14px;
    color: #ffffff;
}
QTextEdit[readOnly="true"] {
    background-color: #151518;
    color: #a0a0a8;
}
QPushButton {
    background-color: #0078d4;
    color: white;
    border: none;
    padding: 8px 16px;
    border-radius: 4px;
    font-weight: bold;
}
QPushButton:hover {
    background-color: #1085e0;
}
QPushButton:pressed {
    background-color: #0066b3;
}

```

```

}
QPushButton:disabled {
    background-color: #3a3a42;
    color: #808088;
}
QListWidget {
    background-color: #1a1a1e;
    border: 1px solid #2c2c35;
    border-radius: 4px;
}
QListWidget::item {
    padding: 8px;
    border-bottom: 1px solid #2c2c35;
}
QListWidget::item:hover {
    background-color: #25252b;
}
QListWidget::item:selected {
    background-color: #0078d4;
    color: white;
}
QCheckBox {
    spacing: 5px;
}
QCheckBox::indicator {
    width: 18px;
    height: 18px;
}
QStatusBar {
    background-color: #121214;
    color: #b0b0b8;
}
QProgressBar {
    border: 1px solid #2c2c35;
    border-radius: 4px;
    text-align: center;
    background-color: #1a1a1e;
    color: white;
}
QProgressBar::chunk {
    background-color: #20c997;
    border-radius: 3px;
}
QLineEdit {
    background-color: #1a1a1e;
    border: 1px solid #2c2c35;
    border-radius: 4px;
    padding: 6px;
    color: white;
}
QLabel {
    font-weight: bold;
    color: #e0e0e8;
}
"""

```

```

class MainWindow(QMainWindow):
    _seg_requested = pyqtSignal(str)
    _emb_requested = pyqtSignal(str, int)
    _fuzzy_requested = pyqtSignal(str, str, str)

    def __init__(self):
        super().__init__()
        self.setWindowTitle("mcat")
        self.resize(1100, 750)
        self.setStyleSheet(DARK_STYLESHEET)

        self.state = AppState()
        self.current_segment_index = -1
        self.modified = False
        self._seg_thread = None
        self._seg_worker = None
        self._emb_thread = None
        self._emb_worker = None
        self._fuzzy_thread = None
        self._fuzzy_worker = None
        self._pending_fuzzy_source = ""
        self._selection_debounce = QTimer()

```

```

self._selection_debounce.setSingleShot(True)
self._selection_debounce.setInterval(80)
self._selection_debounce.timeout.connect(self._on_selection_debounced)

self.init_menu()
self.init_ui()
self._apply_settings()
self.update_stats()

def _ensure_seg_worker(self):
    if self._seg_thread is not None:
        return
    self._seg_thread = QThread()
    self._seg_worker = SegmentationWorker()
    self._seg_worker.moveToThread(self._seg_thread)
    self._seg_requested.connect(self._seg_worker.load_file)
    self._seg_worker.finished.connect(self._on_segmentation_finished)
    self._seg_worker.error.connect(self._on_segmentation_error)
    self._seg_thread.start()

def _ensure_emb_worker(self):
    if self._emb_thread is not None:
        return
    self._emb_thread = QThread()
    self._emb_worker = EmbeddingWorker(self.state.tm.db_path)
    self._emb_worker.moveToThread(self._emb_thread)
    self._emb_requested.connect(self._emb_worker.add_embedding)
    self._emb_thread.start()

def _ensure_fuzzy_worker(self):
    if self._fuzzy_thread is not None:
        return
    self._fuzzy_thread = QThread()
    self._fuzzy_worker = FuzzyMatchWorker(self.state.tm.db_path)
    self._fuzzy_worker.moveToThread(self._fuzzy_thread)
    self._fuzzy_requested.connect(self._fuzzy_worker.find_matches)
    self._fuzzy_worker.finished.connect(self._on_fuzzy_matches_ready)
    self._fuzzy_thread.start()

def _stop_threads(self):
    if self._seg_thread is not None and self._seg_thread.isRunning():
        self._seg_thread.quit()
        self._seg_thread.wait(2000)
    if self._emb_thread is not None and self._emb_thread.isRunning():
        self._emb_thread.quit()
        self._emb_thread.wait(2000)
    if self._fuzzy_thread is not None and self._fuzzy_thread.isRunning():
        self._fuzzy_thread.quit()
        self._fuzzy_thread.wait(2000)

def cleanup(self):
    self._stop_threads()

def closeEvent(self, event):
    self._stop_threads()
    super().closeEvent(event)

def init_menu(self):
    menubar = self.menuBar()

    # File Menu
    file_menu = menubar.addMenu("&File")

    self.new_project_action = file_menu.addAction("&New Project...")
    self.new_project_action.setShortcut(QKeySequence("Ctrl+N"))
    self.new_project_action.setStatusTip("Create a new translation project")
    self.new_project_action.triggered.connect(self.new_project)

    self.open_project_action = file_menu.addAction("&Open Project...")
    self.open_project_action.setShortcut(QKeySequence("Ctrl+Shift+O"))
    self.open_project_action.setStatusTip("Open an existing project")
    self.open_project_action.triggered.connect(self.open_project)

    self.save_project_action = file_menu.addAction("&Save Project")
    self.save_project_action.setShortcut(QKeySequence("Ctrl+Shift+S"))
    self.save_project_action.setStatusTip("Save the current project")
    self.save_project_action.triggered.connect(self.save_project)

    self.save_project_as_action = file_menu.addAction("Save Project &As...")

```

```

self.save_project_as_action.setStatusTip("Save the project with a new name")
self.save_project_as_action.triggered.connect(self.save_project_as)

self.close_project_action = file_menu.addAction("&Close Project")
self.close_project_action.setStatusTip("Close the current project")
self.close_project_action.triggered.connect(self.close_project)

file_menu.addSeparator()

self.project_settings_action = file_menu.addAction("Project &Settings...")
self.project_settings_action.triggered.connect(self.project_settings)

file_menu.addSeparator()

self.recent_menu = file_menu.addMenu("&Recent Projects")

file_menu.addSeparator()

import_menu = file_menu.addMenu("&Import Project")

import_omegat_action = import_menu.addAction("Import Omega&T Project...")
import_omegat_action.triggered.connect(self.import_omegat_project)

import_menu.addSeparator()

import_memoq_action = import_menu.addAction("Import &memoQ Project...")
import_memoq_action.triggered.connect(self.import_memoq_project)

import_menu.addSeparator()

import_sdl_action = import_menu.addAction("Import &SDL Trados SDLXLIFF...")
import_sdl_action.triggered.connect(self.import_sdlxliff)

file_menu.addSeparator()

open_action = file_menu.addAction("&Open File...")
open_action.setShortcut(QKeySequence.Open)
open_action.setStatusTip("Open a translation file (PO, MQXLIFF, SDLXLIFF, etc.)")
open_action.triggered.connect(self.open_file)

save_action = file_menu.addAction("&Save File")
save_action.setShortcut(QKeySequence.Save)
save_action.setStatusTip("Save the current translation file")
save_action.triggered.connect(self.save_file)

save_as_action = file_menu.addAction("Save File &As...")
save_as_action.setStatusTip("Save the translation file in a different format")
save_as_action.triggered.connect(self.save_file_as)

file_menu.addSeparator()

self.render_action = file_menu.addAction("&Render Translated File...")
self.render_action.setShortcut(QKeySequence("Ctrl+R"))
self.render_action.triggered.connect(self.render_translated_file)

file_menu.addSeparator()
exit_action = file_menu.addAction("E&xit")
exit_action.triggered.connect(self.close)

# Go / Navigation Menu
go_menu = menubar.addMenu("&Go")

self.next_seg_action = go_menu.addAction("&Next Segment")
self.next_seg_action.setShortcut(QKeySequence("Ctrl+Down"))
self.next_seg_action.triggered.connect(self._go_next_segment)

self.prev_seg_action = go_menu.addAction("&Previous Segment")
self.prev_seg_action.setShortcut(QKeySequence("Ctrl+Up"))
self.prev_seg_action.triggered.connect(self._go_prev_segment)

go_menu.addSeparator()

self.next_untranslated_action = go_menu.addAction("Next &Untranslated")
self.next_untranslated_action.setShortcut(QKeySequence("Ctrl+U"))
self.next_untranslated_action.triggered.connect(self._go_next_untranslated)

go_menu.addSeparator()

self.toggle_fuzzy_action = go_menu.addAction("Toggle &Fuzzy")

```

```

self.toggle_fuzzy_action.setShortcut(QKeySequence("Ctrl+F"))
self.toggle_fuzzy_action.triggered.connect(self._toggle_fuzzy)

go_menu.addSeparator()

self.first_seg_action = go_menu.addAction("&First Segment")
self.first_seg_action.setShortcut(QKeySequence("Ctrl+Home"))
self.first_seg_action.triggered.connect(self._go_first_segment)

self.last_seg_action = go_menu.addAction("&Last Segment")
self.last_seg_action.setShortcut(QKeySequence("Ctrl+End"))
self.last_seg_action.triggered.connect(self._go_last_segment)

# TM Menu
tm_menu = menubar.addMenu("&Translation Memory")
import_tm_action = tm_menu.addAction("&Import PO into TM...")
import_tm_action.setShortcut(QKeySequence("Ctrl+M"))
import_tm_action.triggered.connect(self.import_po_to_tm)

clear_tm_action = tm_menu.addAction("&Clear TM Database")
clear_tm_action.triggered.connect(self.clear_tm)

# Glossary Menu
glossary_menu = menubar.addMenu("&Glossary")
add_term_action = glossary_menu.addAction("&Add Term...")
add_term_action.setShortcut(QKeySequence("Ctrl+G"))
add_term_action.triggered.connect(self.add_glossary_term)

view_all_action = glossary_menu.addAction("&View/Manage Glossary")
view_all_action.setShortcut(QKeySequence("Ctrl+Shift+G"))
view_all_action.triggered.connect(self.view_glossary)

glossary_menu.addSeparator()
import_tm_action = glossary_menu.addAction("Import &TMX into Glossary...")
import_tm_action.triggered.connect(self.import_tm_to_glossary)
export_tm_action = glossary_menu.addAction("&Export Glossary as TMX...")
export_tm_action.triggered.connect(self.export_glossary_as_tm)

glossary_menu.addSeparator()
collections_menu = glossary_menu.addMenu("&Collections")
create_coll_action = collections_menu.addAction("&Create Collection...")
create_coll_action.triggered.connect(self.create_collection_dialog)
manage_coll_action = collections_menu.addAction("&Manage Collections...")
manage_coll_action.triggered.connect(self.manage_collections)
collections_menu.addSeparator()
import_coll_tm_action = collections_menu.addAction("&Import Collection as TMX...")
import_coll_tm_action.triggered.connect(self.import_collection_tm)
export_coll_tm_action = collections_menu.addAction("&Export Collection as TMX...")
export_coll_tm_action.triggered.connect(self.export_collection_tm)

# Tools Menu
tools_menu = menubar.addMenu("&Tools")
seg_rules_action = tools_menu.addAction("&Segmentation Rules...")
seg_rules_action.setShortcut(QKeySequence("Ctrl+Shift+T"))
seg_rules_action.triggered.connect(self.open_segmentation_rules)

tools_menu.addSeparator()

self.qa_action = tools_menu.addAction("&Quality Assurance...")
self.qa_action.setShortcut(QKeySequence("Ctrl+Shift+Q"))
self.qa_action.setStatusTip("Run quality assurance checks on the translation")
self.qa_action.triggered.connect(self.run_qa)

tools_menu.addSeparator()
self.import_dict_action = tools_menu.addAction("&Import Dictionary...")
self.import_dict_action.triggered.connect(self.import_dictionary)

tools_menu.addSeparator()
self.spellcheck_action = tools_menu.addAction("&Spell Check...")
self.spellcheck_action.setShortcut(QKeySequence("F7"))
self.spellcheck_action.triggered.connect(self.run_spellcheck)

tools_menu.addSeparator()
self.options_action = tools_menu.addAction("&Options...")
self.options_action.setShortcut(QKeySequence("Ctrl+,"))
self.options_action.triggered.connect(self.show_options)

# Help Menu
help_menu = menubar.addMenu("&Help")

```

```

self.shortcuts_action = help_menu.addAction("&Keyboard Shortcuts...")
self.shortcuts_action.setShortcuts(["F1", "Ctrl+Shift+H"])
self.shortcuts_action.triggered.connect(self.show_shortcuts)

help_menu.addSeparator()
get_dicts_action = help_menu.addAction("&Get Dictionaries...")
get_dicts_action.triggered.connect(self._open_dicts_page)

self._update_recent_menu()

def init_ui(self):
    main_splitter = QSplitter(Qt.Horizontal)

    # ----- LEFT PANEL: 2-Column Editable Table -----
    left_widget = QWidget()
    left_layout = QVBoxLayout(left_widget)
    left_layout.setContentsMargins(0, 0, 0, 0)

    self.table = QTableWidget()
    self.table.setColumnCount(2)
    self.table.setHorizontalHeaderLabels(["Source Segment", "Translation"])
    self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    self.table.horizontalHeader().setSectionResizeMode(0, QHeaderView.Stretch)
    self.table.horizontalHeader().setSectionResizeMode(1, QHeaderView.Stretch)
    self.table.setSelectionBehavior(QTableWidget.SelectRows)
    self.table.setSelectionMode(QTableWidget.SingleSelection)
    self.table.itemSelectionChanged.connect(self.segment_selected)
    self.table.cellChanged.connect(self._on_target_cell_changed)
    self.table.setToolTip("Main translation table: source segments (left) and your translations (right). Select a row to edit.")
    self.table.setContextMenuPolicy(Qt.CustomContextMenu)
    self.table.customContextMenuRequested.connect(self._table_context_menu)

    sl = self.state.project.source_lang if self.state.project else None
    tl = self.state.project.target_lang if self.state.project else None
    from cat_tool.suggest import CompletionDelegate
    self.completion_delegate = CompletionDelegate(self.table, self.state.tm, self.state.glossary, self.state.spellchecker, self.state.seg_label)
    self.table.setItemDelegateForColumn(1, self.completion_delegate)

    seg_label = QLabel("Segments")
    left_layout.addWidget(seg_label)
    left_layout.addWidget(self.table)

    # Bottom button bar
    btn_layout = QHBoxLayout()
    self.fuzzy_checkbox = QCheckBox("Fuzzy / Needs Review")
    self.fuzzy_checkbox.setToolTip("Mark this segment as fuzzy or needing review")
    self.fuzzy_checkbox.stateChanged.connect(self.fuzzy_state_changed)

    self.confirm_btn = QPushButton("Confirm & Next (Ctrl+Enter)")
    self.confirm_btn.setToolTip("Confirm the current translation and move to the next segment")
    self.confirm_btn.clicked.connect(self.confirm_active_segment)
    self.confirm_btn.setShortcut(QKeySequence("Ctrl+Return"))

    btn_layout.addWidget(self.fuzzy_checkbox)
    btn_layout.addStretch()
    btn_layout.addWidget(self.confirm_btn)
    left_layout.addLayout(btn_layout)

    # ----- RIGHT PANEL: Sidebars -----
    right_widget = QWidget()
    right_layout = QVBoxLayout(right_widget)
    right_layout.setContentsMargins(0, 0, 0, 0)

    tm_label = QLabel("TM Fuzzy Matches (Double-click to apply)")
    right_layout.addWidget(tm_label)
    self.tm_list = QListWidget()
    self.tm_list.itemDoubleClicked.connect(self.apply_tm_suggestion)
    right_layout.addWidget(self.tm_list)

    glossary_label = QLabel("Detected Terminology / Glossary")
    right_layout.addWidget(glossary_label)
    self.glossary_table = QTableWidget()
    self.glossary_table.setColumnCount(2)
    self.glossary_table.setHorizontalHeaderLabels(["Source Term", "Translation"])
    self.glossary_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    self.glossary_table.setSelectionBehavior(QTableWidget.SelectRows)
    self.glossary_table.setSelectionMode(QTableWidget.SingleSelection)
    self.glossary_table.itemDoubleClicked.connect(self.insert_glossary_term)
    right_layout.addWidget(self.glossary_table)

```

```

main_splitter.addWidget(left_widget)
main_splitter.addWidget(right_widget)
main_splitter.setStretchFactor(0, 2)
main_splitter.setStretchFactor(1, 1)

self.setCentralWidget(main_splitter)

# ----- STATUS BAR -----
self.status_bar = QStatusBar()
self.setStatusBar(self.status_bar)

self.progress_bar = QProgressBar()
self.progress_bar.setMaximumWidth(200)
self.progress_bar.setValue(0)
self.status_bar.addPermanentWidget(self.progress_bar)

self.status_label = QLabel("Ready")
self.status_bar.addWidget(self.status_label)

# ----- BACKGROUND WORKER CALLBACKS -----
def _on_segmentation_finished(self, segments, label):
    self.state.file_handler.segments = segments
    self.state.file_handler.current_file_path = label if os.path.isfile(label) else None
    self.populate_table(segments)
    self.modified = False
    self.update_stats()
    self.status_label.setText(f"Loaded: {os.path.basename(label)} if os.path.isfile(label) else label)")
    if segments:
        self.table.selectRow(0)

def _on_segmentation_error(self, error_msg):
    QMessageBox.critical(self, "Error Loading File", error_msg)
    self.status_label.setText("Load failed")

def _on_embedding_finished(self, row_id):
    pass

def _on_embedding_error(self, error_msg, row_id):
    print(f"Embedding error for row {row_id}: {error_msg}")

def _on_selection_debounced(self):
    if self.current_segment_index < 0:
        return
    seg = self.state.file_handler.segments[self.current_segment_index]
    self._pending_fuzzy_source = seg["source"]
    self.update_fuzzy_matches(seg["source"])
    self.update_glossary_matches(seg["source"])

# ----- FILE OPERATIONS -----
def open_file(self):
    if self.modified:
        reply = QMessageBox.question(
            self, "Unsaved Changes",
            "You have unsaved changes. Open a new file anyway?",
            QMessageBox.Yes | QMessageBox.No
        )
        if reply == QMessageBox.No:
            return

    file_path, _ = QFileDialog.getOpenFileName(
        self, "Open Translation File", "", FILE_FILTER
    )
    if not file_path:
        return

    self.status_label.setText("Loading file...")
    self._ensure_seg_worker()
    self._seg_requested.emit(file_path)

def open_file_path(self, file_path):
    if not os.path.exists(file_path):
        return
    self.status_label.setText("Loading file...")
    self._ensure_seg_worker()
    self._seg_requested.emit(file_path)

def populate_table(self, segments):
    self.table.blockSignals(True)
    self.table.setUpdatesEnabled(False)

```

```

self.table.setRowCount(len(segments))
for idx, seg in enumerate(segments):
    src_text = seg.get("source_clean", seg["source"])
    src_item = QTableWidgetItem(src_text)
    src_item.setFlags(src_item.flags() & ~Qt.ItemIsEditable)
    self.set_source_cell_style(src_item, seg)
    self.table.setItem(idx, 0, src_item)

    tgt_text = seg.get("target_clean", seg["target"])
    tgt_item = QTableWidgetItem(tgt_text)
    tgt_item.setFlags(tgt_item.flags() | Qt.ItemIsEditable)
    self.set_target_cell_style(tgt_item, seg)
    self.table.setItem(idx, 1, tgt_item)

self.table.setUpdatesEnabled(True)
self.table.blockSignals(False)

source_texts = [seg.get("source_clean", seg["source"]) for seg in segments]
self.completion_delegate.set_source_texts(source_texts)

def get_status_text(self, seg):
    if seg["fuzzy"]:
        return "Fuzzy"
    elif seg["target"] and seg["target"].strip():
        return "Translated"
    else:
        return "Untranslated"

def set_source_cell_style(self, item, seg):
    if seg["fuzzy"]:
        item.setBackground(_SRC_FUZZY_BG)
        item.setForeground(_SRC_FUZZY_FG)
    elif seg.get("translated", False) or (seg["target"] and seg["target"].strip()):
        item.setBackground(_SRC_TRANSLATED_BG)
        item.setForeground(_SRC_TRANSLATED_FG)
    else:
        item.setBackground(_SRC_DEFAULT_BG)
        item.setForeground(_SRC_DEFAULT_FG)

def set_target_cell_style(self, item, seg):
    if seg["fuzzy"]:
        item.setBackground(_TGT_FUZZY_BG)
        item.setForeground(_TGT_FUZZY_FG)
    elif seg.get("translated", False) or (seg["target"] and seg["target"].strip()):
        item.setBackground(_TGT_TRANSLATED_BG)
        item.setForeground(_TGT_TRANSLATED_FG)
    else:
        item.setBackground(_TGT_DEFAULT_BG)
        item.setForeground(_TGT_DEFAULT_FG)

def _on_target_cell_changed(self, row, col):
    if col != 1:
        return
    seg = self.state.file_handler.segments[row]
    item = self.table.item(row, 1)
    if item is None:
        return

    new_text = item.text().strip()
    tags = seg.get("all_tags", [])
    restored = restore_tags(new_text, tags)

    if seg["target"] == restored:
        return

    seg["target"] = restored
    seg["translated"] = bool(restored)
    new_tgt_tags = extract_all_tags(restored)
    all_tags = list(tags)
    for t in new_tgt_tags:
        if t not in all_tags:
            all_tags.append(t)
    seg["all_tags"] = all_tags
    seg["target_clean"] = apply_tags(restored, all_tags)

    if restored:
        row_id = self.state.tm.add_translation(seg["source"], restored, self.state.tm_sl, self.state.tm_tl)
        if row_id is not None:
            self._ensure_emb_worker()

```

```

        self._emb_requested.emit(seg["source"], row_id)

    item.setText(seg["target_clean"])
    self.set_target_cell_style(item, seg)
    self.set_source_cell_style(self.table.item(row, 0), seg)

    self.modified = True
    self.update_stats()

def save_file(self):
    if not self.state.file_handler.current_file_path:
        self.save_file_as()
        return
    try:
        self.state.file_handler.save_file()
        self.modified = False
        self.status_label.setText(f"Saved: {os.path.basename(self.state.file_handler.current_file_path)}")
    except Exception as e:
        QMessageBox.critical(self, "Error Saving File", str(e))

def save_file_as(self):
    file_path, selected_filter = QFileDialog.getSaveFileName(
        self, "Save Translation File As", "",
        "MCAT Project (*.mcat.db);;PO Files (*.po);;memoQ MQXLIFF (*.mqxliff);;SDL Trados SDLXLIFF (*.sdlxliiff);;OmegaT Project folder;*"
    )
    if not file_path:
        return
    try:
        src_lang = self.state.project.source_lang if self.state.project else "en"
        tgt_lang = self.state.project.target_lang if self.state.project else "es"

        if "OmegaT" in selected_filter:
            dir_path = QFileDialog.getExistingDirectory(
                self, "Select folder to create OmegaT project in",
                os.path.dirname(file_path) if os.path.dirname(file_path) else ""
            )
            if not dir_path:
                return
            target_dir = os.path.join(dir_path, os.path.splitext(os.path.basename(file_path))[0])
            p = self.state.project if self.state.project else Project()
            if not self.state.project:
                p.new(os.path.basename(target_dir), src_lang, tgt_lang, [])
            p.save_as_omegat(target_dir)
            self.status_label.setText(f"OmegaT project saved: {target_dir}")
            QMessageBox.information(self, "Success",
                f"Exported to OmegaT format:\n{target_dir}")
            return
        elif "MCAT" in selected_filter:
            if not file_path.lower().endswith(".mcat.db"):
                file_path += ".mcat.db"
            self.state.file_handler.save_file(file_path)
        elif "MQXLIFF" in selected_filter:
            if not file_path.endswith(".mqxliff"):
                file_path += ".mqxliff"
            self.state.file_handler.save_as_mqxliiff(file_path, src_lang, tgt_lang)
        elif "SDLXLIFF" in selected_filter:
            if not file_path.endswith(".sdlxliiff"):
                file_path += ".sdlxliiff"
            from cat_tool.formats.sdlxliiff import SdlxliiffHandler
            SdlxliiffHandler().save(self.state.file_handler.segments, file_path, src_lang, tgt_lang)
            self.state.file_handler.current_file_path = file_path
        else:
            self.state.file_handler.save_file(file_path)
            self.modified = False
            self.status_label.setText(f"Saved: {os.path.basename(file_path)}")
            QMessageBox.information(self, "Success", "File saved successfully.")
    except Exception as e:
        QMessageBox.critical(self, "Error Saving File", str(e))

# ----- RENDER TRANSLATED -----
def render_translated_file(self):
    if not self.state.file_handler.segments:
        QMessageBox.warning(self, "No Data", "No file loaded. Open a file first.")
        return
    ext = os.path.splitext(self.state.file_handler.current_file_path or "")[1].lower()
    ext_filter = f"Translated {ext.upper()} (*{ext})" if ext else "All Files (*)"
    file_path, _ = QFileDialog.getSaveFileName(
        self, "Save Translated File As", f"translated{ext}", ext_filter
    )

```

```

if not file_path:
    return
try:
    self.state.file_handler.render_translated(file_path)
    self.status_label.setText(f"Rendered: {os.path.basename(file_path)}")
    QMessageBox.information(self, "Success", f"Translated file saved:\n{file_path}")
except Exception as e:
    QMessageBox.critical(self, "Render Error", str(e))

# ----- SELECTION & CONFIRMATION -----
def segment_selected(self):
    row = self.table.currentRow()
    if row < 0:
        return

    self.current_segment_index = row
    seg = self.state.file_handler.segments[row]

    self.fuzzy_checkbox.blockSignals(True)
    self.fuzzy_checkbox.setChecked(seg["fuzzy"])
    self.fuzzy_checkbox.blockSignals(False)

    self._selection_debounce.start()

def fuzzy_state_changed(self, state):
    if self.current_segment_index < 0:
        return
    seg = self.state.file_handler.segments[self.current_segment_index]
    seg["fuzzy"] = (state == Qt.Checked)
    self.modified = True
    self.update_table_row_status(self.current_segment_index, seg)
    self.update_stats()

def update_table_row_status(self, row, seg):
    self.table.blockSignals(True)
    item = self.table.item(row, 1)
    if item:
        item.setText(seg.get("target_clean", seg["target"]))
        self.set_target_cell_style(item, seg)
    src_item = self.table.item(row, 0)
    if src_item:
        self.set_source_cell_style(src_item, seg)
    self.table.blockSignals(False)

def confirm_active_segment(self):
    if self.current_segment_index < 0:
        return

    row = self.current_segment_index
    item = self.table.item(row, 1)
    cell_text = item.text().strip() if item else ""

    seg = self.state.file_handler.segments[row]
    tags = seg.get("all_tags", [])
    restored = restore_tags(cell_text, tags)
    if seg["target"] == restored and seg["translated"] == bool(restored):
        pass
    else:
        seg["target"] = restored
        seg["translated"] = bool(restored)
        new_tgt_tags = extract_all_tags(restored)
        all_tags = list(tags)
        for t in new_tgt_tags:
            if t not in all_tags:
                all_tags.append(t)
        seg["all_tags"] = all_tags
        seg["target_clean"] = apply_tags(restored, all_tags)
        if restored:
            row_id = self.state.tm.add_translation(seg["source"], restored, self.state.tm_sl, self.state.tm_tl)
            if row_id is not None:
                self._ensure_emb_worker()
                self._emb_requested.emit(seg["source"], row_id)

    self.modified = True
    if item:
        item.setText(seg["target_clean"])
    self.update_table_row_status(row, seg)

self.update_stats()

```

```

        if row < len(self.state.file_handler.segments) - 1:
            self.table.selectRow(row + 1)
            self._edit_target_cell(row + 1)
        else:
            self.status_label.setText("Reached the end of the file!")

# ----- SEGMENT NAVIGATION -----
def _edit_target_cell(self, row):
    idx = self.table.model().index(row, 1)
    self.table.edit(idx)

def _go_next_segment(self):
    if not self.state.file_handler.segments:
        return
    next_idx = self.current_segment_index + 1
    if next_idx < len(self.state.file_handler.segments):
        self.table.selectRow(next_idx)
        self._edit_target_cell(next_idx)
    else:
        self.status_label.setText("Already at the last segment.")

def _go_prev_segment(self):
    if not self.state.file_handler.segments:
        return
    prev_idx = self.current_segment_index - 1
    if prev_idx >= 0:
        self.table.selectRow(prev_idx)
        self._edit_target_cell(prev_idx)
    else:
        self.status_label.setText("Already at the first segment.")

def _go_next_untranslated(self):
    if not self.state.file_handler.segments:
        return
    start = self.current_segment_index + 1
    for i in range(start, len(self.state.file_handler.segments)):
        seg = self.state.file_handler.segments[i]
        if not seg.get("translated", False) or not seg.get("target", "").strip():
            self.table.selectRow(i)
            self._edit_target_cell(i)
            return
    self.status_label.setText("No more untranslated segments.")

# ----- CONTEXT MENU -----
def _table_context_menu(self, pos):
    index = self.table.indexAt(pos)
    if not index.isValid():
        return
    row = index.row()
    col = index.column()
    if row < 0 or row >= len(self.state.file_handler.segments):
        return
    seg = self.state.file_handler.segments[row]
    self.table.selectRow(row)

    menu = QMenu(self)
    view_action = menu.addAction("View Tags")
    if col == 0:
        view_action.triggered.connect(lambda: TagViewDialog(seg, 0, self).exec_())
    else:
        view_action.triggered.connect(lambda: self._show_tag_dialog_for_target(seg))
    menu.addSeparator()
    spell_action = menu.addAction("Spell Check Segment")
    spell_action.triggered.connect(lambda: self._spellcheck_segment(seg))
    menu.exec_(self.table.viewport().mapToGlobal(pos))

def _show_tag_dialog_for_target(self, seg):
    dlg = TagViewDialog(seg, 1, self)
    dlg.exec_()

def _spellcheck_segment(self, seg):
    dlg = SpellCheckDialog([seg], self.state.spellchecker, self)
    dlg.exec_()
    self.populate_table(self.state.file_handler.segments)

def show_options(self):
    dlg = SettingsDialog(self.state.settings, self)
    if dlg.exec_() == QDialog.Accepted:
        self._apply_settings()

```

```

def _apply_settings(self):
    font = self.table.font()
    font.setPointSize(self.state.settings["source_font_size"])
    self.table.horizontalHeaderItem(0).setFont(font) if self.table.horizontalHeaderItem(0) else None
    if self.state.settings["source_font_size"] != self.state.settings["target_font_size"]:
        tgt_font = QFont(font)
        tgt_font.setPointSize(self.state.settings["target_font_size"])
        self.table.horizontalHeaderItem(1).setFont(tgt_font) if self.table.horizontalHeaderItem(1) else None
    self.table.setFont(font)

# ----- SIDEBAR SUGGESTIONS & INTERACTIONS -----
def update_fuzzy_matches(self, source_text):
    self._ensure_fuzzy_worker()
    self._fuzzy_requested.emit(
        source_text,
        self.state.tm_sl or "",
        self.state.tm_tl or ""
    )

def _on_fuzzy_matches_ready(self, matches):
    if self.current_segment_index < 0:
        return
    current_source = self.state.file_handler.segments[self.current_segment_index]["source"]
    if current_source != self._pending_fuzzy_source:
        return
    self.tm_list.clear()
    for m in matches:
        item = QListWidgetItem(f"[{m['score']}]% Match] {m['target']}\n(From: \"{m['source']}\")")
        item.setData(Qt.UserRole, m["target"])
        self.tm_list.addItem(item)

def apply_tm_suggestion(self, item):
    target_suggestion = item.data(Qt.UserRole)
    if target_suggestion and self.current_segment_index >= 0:
        tgt_item = self.table.item(self.current_segment_index, 1)
        if tgt_item:
            tgt_item.setText(target_suggestion)
            self._edit_target_cell(self.current_segment_index)

def update_glossary_matches(self, source_text):
    self.glossary_table.setRowCount(0)
    matches = self.state.glossary.check_segment(source_text)
    self.glossary_table.setRowCount(len(matches))
    for idx, m in enumerate(matches):
        src_item = QTableWidgetItem(m["source"])
        tgt_item = QTableWidgetItem(m["target"])
        src_item.setFlags(src_item.flags() & ~Qt.ItemIsEditable)
        tgt_item.setFlags(tgt_item.flags() & ~Qt.ItemIsEditable)
        if m["description"]:
            src_item.setToolTip(m["description"])
            tgt_item.setToolTip(m["description"])
        self.glossary_table.setItem(idx, 0, src_item)
        self.glossary_table.setItem(idx, 1, tgt_item)

def insert_glossary_term(self, item):
    glossary_row = item.row()
    term_item = self.glossary_table.item(glossary_row, 1)
    if not term_item or self.current_segment_index < 0:
        return
    tgt_item = self.table.item(self.current_segment_index, 1)
    if tgt_item:
        current = tgt_item.text()
        tgt_item.setText(current + term_item.text())
        self._edit_target_cell(self.current_segment_index)

# ----- GLOSSARY MANAGEMENT -----
def add_glossary_term(self):
    src_lang = self.state.project.source_lang if self.state.project else "en"
    tgt_lang = self.state.project.target_lang if self.state.project else ""
    dial = GlossaryDialog(self, source_lang=src_lang, target_lang=tgt_lang)
    if dial.exec_() == QDialog.Accepted:
        src, tgt, desc, sl, tl = dial.get_data()
        if src and tgt:
            self.state.glossary.add_term(src, tgt, desc, source_lang=sl, target_lang=tl)
            self.status_label.setText(f"Added glossary term: '{src}'")
            if self.current_segment_index >= 0:
                seg = self.state.file_handler.segments[self.current_segment_index]
                self.update_glossary_matches(seg["source"])
        else:

```

```

        QMessageBox.warning(self, "Invalid Input", "Source and Target cannot be empty.")

def import_tm_x_to_glossary(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Import TMX into Glossary", "",
        "TMX Files (*.tmx);;All Files (*)")
    if not path:
        return
    try:
        info = self.state.glossary.peek_tm_x(path)
        if info["count"] == 0 and not info.get("source_lang"):
            QMessageBox.information(self, "No Terms",
                "No translatable term pairs found in this TMX file.")
            return

        sl = info["source_lang"] or (self.state.project.source_lang if self.state.project else "en")
        tl = info["target_lang"] or ""

        dlg = LangDialog(sl, tl, info["count"], self)
        if dlg.exec_() != QDialog.Accepted:
            return

        source_lang = dlg.src_combo.currentData() or sl
        target_lang = dlg.tgt_combo.currentData() or tl

        count = self.state.glossary.import_tm_x(path, source_lang=source_lang)
        QMessageBox.information(
            self, "Import Complete",
            f"Imported {count} terms from TMX into glossary.\n"
            f"Source: {source_lang} | Target: {target_lang}")
    except Exception as e:
        QMessageBox.critical(self, "Import Error", f"Failed to import TMX:\n{e}")

def export_glossary_as_tm_x(self):
    path, _ = QFileDialog.getSaveFileName(
        self, "Export Glossary as TMX", "",
        "TMX Files (*.tmx);;All Files (*)")
    if not path:
        return
    if not path.lower().endswith(".tmx"):
        path += ".tmx"
    try:
        count = self.state.glossary.export_tm_x(path)
        if count:
            QMessageBox.information(self, "Export Complete", f"Exported {count} terms to TMX.")
        else:
            QMessageBox.information(self, "Export Complete", "Glossary is empty – no terms exported.")
    except Exception as e:
        QMessageBox.critical(self, "Export Error", f"Failed to export TMX:\n{e}")

# ----- COLLECTIONS MANAGEMENT -----
def create_collection_dialog(self):
    dialog = QDialog(self)
    dialog.setWindowTitle("Create Collection")
    dialog.setMinimumWidth(400)
    layout = QFormLayout(dialog)
    name_edit = QLineEdit()
    name_edit.setPlaceholderText("My Collection")
    desc_edit = QLineEdit()
    desc_edit.setPlaceholderText("Optional description")
    layout.addRow("Collection Name:", name_edit)
    layout.addRow("Description:", desc_edit)
    btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
    btn_box.accepted.connect(dialog.accept)
    btn_box.rejected.connect(dialog.reject)
    layout.addRow(btn_box)
    if dialog.exec_() != QDialog.Accepted:
        return
    name = name_edit.text().strip()
    if not name:
        QMessageBox.warning(self, "Invalid", "Collection name cannot be empty.")
        return
    coll_id = self.state.glossary.create_collection(name, desc_edit.text().strip())
    if coll_id is None:
        QMessageBox.warning(self, "Duplicate", f"Collection '{name}' already exists.")
        return
    self.status_label.setText(f"Created collection: '{name}'")

def manage_collections(self):

```

```

collections = self.state.glossary.get_all_collections()
if not collections:
    if QMessageBox.question(self, "No Collections",
        "No collections yet. Create one now?",
        QMessageBox.Yes | QMessageBox.No) == QMessageBox.Yes:
        self.create_collection_dialog()
        collections = self.state.glossary.get_all_collections()
    if not collections:
        return

dialog = QDialog(self)
dialog.setWindowTitle("Manage Collections")
dialog.setMinimumSize(700, 450)
layout = QVBoxLayout(dialog)
hsplit = QSplitter(Qt.Horizontal)

left_widget = QWidget()
left_layout = QVBoxLayout(left_widget)
left_layout.setContentsMargins(0, 0, 0, 0)
left_layout.addWidget(QLabel("Collections:"))
coll_list = QListWidget()
left_layout.addWidget(coll_list)
coll_btn_layout = QHBoxLayout()
create_btn = QPushButton("Create")
create_btn.setToolTip("Create a new collection")
delete_btn = QPushButton("Delete")
delete_btn.setToolTip("Delete the selected collection")
coll_btn_layout.addWidget(create_btn)
coll_btn_layout.addWidget(delete_btn)
left_layout.addLayout(coll_btn_layout)

right_widget = QWidget()
right_layout = QVBoxLayout(right_widget)
right_layout.setContentsMargins(0, 0, 0, 0)
right_layout.addWidget(QLabel("Terms in collection:"))
terms_tbl = QTableWidgetItem()
terms_tbl.setColumnCount(4)
terms_tbl.setHorizontalHeaderLabels(["Source", "Translation", "SL", "TL"])
terms_tbl.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
right_layout.addWidget(terms_tbl)
term_btn_layout = QHBoxLayout()
add_term_btn = QPushButton("Add Term...")
add_term_btn.setToolTip("Select a glossary term to add to this collection")
rem_term_btn = QPushButton("Remove Selected")
rem_term_btn.setToolTip("Remove the selected term from this collection")
term_btn_layout.addWidget(add_term_btn)
term_btn_layout.addWidget(rem_term_btn)
term_btn_layout.addStretch()
right_layout.addLayout(term_btn_layout)

hsplit.addWidget(left_widget)
hsplit.addWidget(right_widget)
hsplit.setStretchFactor(0, 1)
hsplit.setStretchFactor(1, 2)
layout.addWidget(hsplit)

btn_layout = QHBoxLayout()
close_btn = QPushButton("Close")
close_btn.setToolTip("Close collections manager")
close_btn.clicked.connect(dialog.accept)
btn_layout.addStretch()
btn_layout.addWidget(close_btn)
layout.addLayout(btn_layout)

coll_map = {}
def refresh_collection_list():
    coll_list.clear()
    coll_map.clear()
    for c in self.state.glossary.get_all_collections():
        coll_list.addItem(f"{c['name']} ({c['description']})" if c['description'] else c['name'])
        coll_map[coll_list.count() - 1] = c["id"]

def refresh_terms():
    terms_tbl.setRowCount(0)
    sel = coll_list.currentRow()
    if sel < 0 or sel not in coll_map:
        return
    cid = coll_map[sel]
    terms = self.state.glossary.get_collection_terms(cid)

```

```

terms_tbl.setRowCount(len(terms))
for idx, t in enumerate(terms):
    terms_tbl.setItem(idx, 0, QTableWidgetItem(t["source"]))
    terms_tbl.setItem(idx, 1, QTableWidgetItem(t["target"]))
    terms_tbl.setItem(idx, 2, QTableWidgetItem(t.get("source_lang", "")))
    terms_tbl.setItem(idx, 3, QTableWidgetItem(t.get("target_lang", "")))

def on_coll_selected():
    refresh_terms()

def do_create():
    dialog.close()
    self.create_collection_dialog()
    refresh_collection_list()

def do_delete():
    sel = coll_list.currentRow()
    if sel < 0 or sel not in coll_map:
        return
    cid = coll_map[sel]
    name = coll_list.currentItem().text().split(" ")[0] if coll_list.currentItem() else ""
    if QMessageBox.question(dialog, "Confirm Delete",
        f"Delete collection '{name}' and all its term associations?",
        QMessageBox.Yes | QMessageBox.No) != QMessageBox.Yes:
        return
    self.state.glossary.delete_collection(cid)
    self.status_label.setText(f"Deleted collection: '{name}'")
    refresh_collection_list()
    terms_tbl.setRowCount(0)

def do_add_term():
    sel = coll_list.currentRow()
    if sel < 0 or sel not in coll_map:
        QMessageBox.information(dialog, "No Collection", "Select a collection first.")
        return
    cid = coll_map[sel]
    all_terms = self.state.glossary.get_all_terms()
    if not all_terms:
        QMessageBox.information(dialog, "No Terms", "The glossary is empty.")
        return
    existing = {t["source"] for t in self.state.glossary.get_collection_terms(cid)}
    available = [t for t in all_terms if t["source"] not in existing]
    if not available:
        QMessageBox.information(dialog, "All Added", "All glossary terms are already in this collection.")
        return
    picker = QDialog(dialog)
    picker.setWindowTitle("Add Term to Collection")
    picker.setMinimumWidth(450)
    ply = QVBoxLayout(picker)
    ply.addWidget(QLabel("Select a term to add:"))
    pick_list = QListWidget()
    for t in available:
        pick_list.addItem(f"{t['source']} → {t['target']}")
    ply.addWidget(pick_list)
    pb = QHBoxLayout()
    ok_btn = QPushButton("Add")
    ok_btn.clicked.connect(picker.accept)
    cancel_btn = QPushButton("Cancel")
    cancel_btn.clicked.connect(picker.reject)
    pb.addWidget(ok_btn)
    pb.addWidget(cancel_btn)
    ply.addLayout(pb)
    if picker.exec_() != QDialog.Accepted or pick_list.currentRow() < 0:
        return
    selected_term = available[pick_list.currentRow()]["source"]
    self.state.glossary.add_term_to_collection(cid, selected_term)
    self.status_label.setText(f"Added '{selected_term}' to collection")
    refresh_terms()

def do_remove_term():
    sel = coll_list.currentRow()
    if sel < 0 or sel not in coll_map:
        return
    cid = coll_map[sel]
    row = terms_tbl.currentRow()
    if row < 0:
        return
    src = terms_tbl.item(row, 0).text()
    if QMessageBox.question(dialog, "Confirm Remove",

```

```

        f"Remove '{src}' from this collection? (Term stays in glossary)",
        QMessageBox.Yes | QMessageBox.No) != QMessageBox.Yes:
            return
        self.state.glossary.remove_term_from_collection(cid, src)
        self.status_label.setText(f"Removed '{src}' from collection")
        refresh_terms()

    coll_list.currentRowChanged.connect(on_coll_selected)
    create_btn.clicked.connect(do_create)
    delete_btn.clicked.connect(do_delete)
    add_term_btn.clicked.connect(do_add_term)
    rem_term_btn.clicked.connect(do_remove_term)

    refresh_collection_list()
    if coll_list.count() > 0:
        coll_list.setCurrentRow(0)
    dialog.exec_()

def import_collection_tmx(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Import Collection as TMX", "",
        "TMX Files (*.tmx);;All Files (*)")
    if not path:
        return
    try:
        count, coll_id = self.state.glossary.import_collection_tmx(path)
        if coll_id is None:
            QMessageBox.warning(self, "Import Failed",
                                "Could not create collection. Name may already exist.")
            return
        if count:
            QMessageBox.information(self, "Import Complete",
                                    f"Imported {count} terms into a new collection.")
        else:
            QMessageBox.information(self, "No Terms",
                                    "No translatable term pairs found in that TMX.")
    except Exception as e:
        QMessageBox.critical(self, "Import Error", str(e))

def export_collection_tmx(self):
    collections = self.state.glossary.get_all_collections()
    if not collections:
        QMessageBox.information(self, "No Collections", "No collections to export.")
        return
    coll_names = [c["name"] for c in collections]
    picker = QDialog(self)
    picker.setWindowTitle("Export Collection as TMX")
    picker.setMinimumWidth(400)
    ply = QVBoxLayout(picker)
    ply.addWidget(QLabel("Select a collection to export:"))
    pick_list = QListWidget()
    for n in coll_names:
        pick_list.addItem(n)
    ply.addWidget(pick_list)
    pb = QHBoxLayout()
    ok_btn = QPushButton("Export")
    ok_btn.clicked.connect(picker.accept)
    cancel_btn = QPushButton("Cancel")
    cancel_btn.clicked.connect(picker.reject)
    pb.addWidget(ok_btn)
    pb.addWidget(cancel_btn)
    ply.addLayout(pb)
    if picker.exec_() != QDialog.Accepted or pick_list.currentRow() < 0:
        return
    coll = collections[pick_list.currentRow()]
    path, _ = QFileDialog.getSaveFileName(
        self, "Save Collection TMX", f"{coll['name']}.tmx",
        "TMX Files (*.tmx);;All Files (*)")
    if not path:
        return
    if not path.lower().endswith(".tmx"):
        path += ".tmx"
    try:
        count = self.state.glossary.export_collection_tmx(coll["id"], path)
        if count:
            QMessageBox.information(self, "Export Complete",
                                    f"Exported {count} terms from '{coll['name']}' to TMX.")
        else:
            QMessageBox.information(self, "Empty Collection", "That collection has no terms.")

```

```

except Exception as e:
    QMessageBox.critical(self, "Export Error", str(e))

def view_glossary(self):
    terms = self.state.glossary.get_all_terms()
    dialog = QDialog(self)
    dialog.setWindowTitle("Glossary Terms List")
    dialog.setMinimumSize(650, 400)
    layout = QVBoxLayout(dialog)
    tbl = QTableWidgetItem()
    tbl.setColumnCount(5)
    tbl.setHorizontalHeaderLabels(["Source Term", "SL", "Target Translation", "TL", "Description"])
    tbl.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    tbl.setRowCount(len(terms))

    for idx, t in enumerate(terms):
        tbl.setItem(idx, 0, QTableWidgetItem(t["source"]))
        tbl.setItem(idx, 1, QTableWidgetItem(t.get("source_lang", "")))
        tbl.setItem(idx, 2, QTableWidgetItem(t["target"]))
        tbl.setItem(idx, 3, QTableWidgetItem(t.get("target_lang", "")))
        tbl.setItem(idx, 4, QTableWidgetItem(t["description"]))

    layout.addWidget(tbl)
    btn_layout = QHBoxLayout()
    del_btn = QPushButton("Delete Selected", dialog)
    del_btn.setToolTip("Remove the selected glossary term")

    def delete_selected():
        selected_items = tbl.selectedItems()
        if not selected_items:
            return
        row = selected_items[0].row()
        src = tbl.item(row, 0).text()
        confirm = QMessageBox.question(
            dialog, "Confirm Delete",
            f"Delete glossary term '{src}'?",
            QMessageBox.Yes | QMessageBox.No)
        if confirm != QMessageBox.Yes:
            return
        self.state.glossary.remove_term(src)
        tbl.removeRow(row)
        self.status_label.setText(f"Deleted term: '{src}'")
        if self.current_segment_index >= 0:
            seg = self.state.file_handler.segments[self.current_segment_index]
            self.update_glossary_matches(seg["source"])

    del_btn.clicked.connect(delete_selected)
    btn_layout.addWidget(del_btn)
    close_btn = QPushButton("Close", dialog)
    close_btn.setToolTip("Close the glossary list")
    close_btn.clicked.connect(dialog.accept)
    btn_layout.addWidget(close_btn)
    btn_layout.addStretch()
    layout.addLayout(btn_layout)
    dialog.exec_()

# ----- TM MANAGEMENT -----
def import_po_to_tm(self):
    file_path, _ = QFileDialog.getOpenFileName(
        self, "Import PO for Translation Memory", "", "PO Files (*.po);;All Files (*)"
    )
    if not file_path:
        return
    try:
        imported_count, segments = self.state.import_po_to_tm(file_path)
        QMessageBox.information(
            self, "Success", f"Imported {imported_count} translations into translation memory."
        )
        if self.current_segment_index >= 0:
            seg = self.state.file_handler.segments[self.current_segment_index]
            self.update_fuzzy_matches(seg["source"])
    except Exception as e:
        QMessageBox.critical(self, "Import Error", str(e))

def clear_tm(self):
    reply = QMessageBox.question(
        self, "Confirm Delete", "Are you sure you want to clear the entire Translation Memory?",
        QMessageBox.Yes | QMessageBox.No
    )

```

```

        if reply == QMessageBox.Yes:
            self.state.clear_tm()
            self.status_label.setText("Translation Memory cleared.")
            if self.current_segment_index >= 0:
                seg = self.state.file_handler.segments[self.current_segment_index]
                self.update_fuzzy_matches(seg["source"])

def _toggle_fuzzy(self):
    if self.current_segment_index >= 0:
        self.fuzzy_checkbox.toggle()

def _go_first_segment(self):
    if self.state.file_handler.segments:
        self.table.selectRow(0)
        self._edit_target_cell(0)

def _go_last_segment(self):
    if self.state.file_handler.segments:
        last = len(self.state.file_handler.segments) - 1
        self.table.selectRow(last)
        self._edit_target_cell(last)

def show_shortcuts(self):
    dlg = ShortcutsDialog(self)
    dlg.exec_()

def _open_dicts_page(self):
    import webbrowser
    webbrowser.open("https://extensions.libreoffice.org/en?Tags%5B%5D=50")

def navigate_to_segment(self, segment_index):
    if 0 <= segment_index < len(self.state.file_handler.segments):
        self.table.selectRow(segment_index)
        self._edit_target_cell(segment_index)

def run_qa(self):
    if not self.state.file_handler.segments:
        QMessageBox.warning(self, "No Data", "Open a file first before running QA.")
        return
    issues = qa.run_all_checks(self.state.file_handler.segments, glossary=self.state.glossary)
    dlg = QADialog(issues, self)
    dlg.exec_()

def import_dictionary(self):
    path, _ = QFileDialog.getOpenFileName(
        self, "Import Dictionary", "",
        "Dictionary (*.oxt *.zip *.dic);;All Files (*)"
    )
    if not path:
        return
    if path.endswith(".dic"):
        ok = self.state.spellchecker.load_dictionary(path)
    else:
        ok = self.state.spellchecker.load_from_archive(path)
    if ok:
        names = self.state.spellchecker.loaded_names
        QMessageBox.information(
            self, "Success",
            f"Dictionary imported.\nLoaded: {' '.join(names)}"
        )
    else:
        QMessageBox.warning(self, "Error", "Failed to load dictionary.\n\n"
            "Make sure the file contains valid .dic/.aff files.")

def _load_spelling_dict_for_language(self, lang_code):
    found = self.state.load_spelling_dict(lang_code)
    if found:
        self.status_label.setText(f"Loaded spelling dictionary: {found}")
    elif lang_code:
        self.status_label.setText(
            f"No spelling dictionary found for '{lang_code}'. "
            f"Use Tools > Import Dictionary to add one."
        )

def run_spellcheck(self):
    if not self.state.file_handler.segments:
        QMessageBox.warning(self, "No Data", "Open a file first before spell check.")
        return
    dlg = SpellCheckDialog(self.state.file_handler.segments, self.state.spellchecker, self)

```

```

        dlg.exec_()
        self.populate_table(self.state.file_handler.segments)

# ----- STATS UPDATE -----
def update_stats(self):
    stats = self.state.compute_stats()
    if stats["total"] == 0:
        self.progress_bar.setValue(0)
        self.progress_bar.setFormat("No file loaded")
        return

    self.progress_bar.setValue(stats["percent"])
    self.progress_bar.setFormat(f"%p% ({stats['translated']}/{stats['total']})")

    proj_name = self.state.project.name if self.state.project else "No Project"
    self.status_label.setText(
        f"[{proj_name}] "
        f"Active File: {os.path.basename(self.state.file_handler.current_file_path or 'None')} | "
        f"Translated: {stats['translated']} | Fuzzy: {stats['fuzzy']} | Total: {stats['total']} | "
        f"Words: {stats['total_words']} | Chars: {stats['total_chars']}"
    )
    title = f"mcat - {proj_name}"
    if self.state.file_handler.current_file_path:
        title += f" [{os.path.basename(self.state.file_handler.current_file_path)}]"
    self.setWindowTitle(title)

# ----- PROJECT OPERATIONS -----
def close_project(self):
    if not self.state.project:
        return
    if self.modified:
        reply = QMessageBox.question(
            self, "Unsaved Changes",
            "You have unsaved changes. Save before closing?",
            QMessageBox.Yes | QMessageBox.No | QMessageBox.Cancel
        )
        if reply == QMessageBox.Cancel:
            return
        if reply == QMessageBox.Yes:
            self.save_project()

    proj_name = self.state.close_project()
    self.current_segment_index = -1
    self.modified = False
    self.table.setRowCount(0)
    self.tm_list.clear()
    self.glossary_table.setRowCount(0)
    self.status_label.setText(f"Project closed: {proj_name}")

def new_project(self):
    if self.modified:
        reply = QMessageBox.question(
            self, "Unsaved Changes",
            "You have unsaved changes. Start a new project anyway?",
            QMessageBox.Yes | QMessageBox.No
        )
        if reply == QMessageBox.No:
            return

    dial = NewProjectDialog(self)
    if dial.exec_() != QDialog.Accepted:
        return

    name, src_lang, tgt_lang, src_files = dial.get_data()
    if not name:
        QMessageBox.warning(self, "Invalid", "Project name cannot be empty.")
        return

    self.state.new_project(name, src_lang, tgt_lang, src_files)
    self.current_segment_index = -1
    self.modified = False

    if src_files:
        self._open_project_source_file(0)
    else:
        self.table.setRowCount(0)

    self.update_stats()
    self._load_spelling_dict_for_language(tgt_lang)

```

```

self.status_label.setText(f"Created project: {name}")

def open_project(self):
    if self.modified:
        reply = QMessageBox.question(
            self, "Unsaved Changes",
            "You have unsaved changes. Open a different project anyway?",
            QMessageBox.Yes | QMessageBox.No
        )
        if reply == QMessageBox.No:
            return

    filter_str = "CAT Project (*.mcatproj);;OmegaT Project (folder with omegat.project);;memoQ Project (folder with manifest.xml)"
    file_path, selected_filter = QFileDialog.getOpenFileName(
        self, "Open Project", "", filter_str
    )
    if not file_path:
        return

    if "OmegaT" in selected_filter:
        dir_path = QFileDialog.getExistingDirectory(
            self, "Select OmegaT Project Folder (contains omegat.project)"
        )
        if not dir_path:
            return
        proj_file = os.path.join(dir_path, "omegat.project")
        if not os.path.exists(proj_file):
            QMessageBox.warning(self, "Invalid Project",
                               "Selected folder does not contain omegat.project")
            return
        self._load_omegat(dir_path)
    elif "memoQ" in selected_filter:
        dir_path = QFileDialog.getExistingDirectory(
            self, "Select memoQ Project Folder (contains manifest.xml)"
        )
        if not dir_path:
            return
        manifest_file = os.path.join(dir_path, "manifest.xml")
        if not os.path.exists(manifest_file):
            QMessageBox.warning(self, "Invalid Project",
                               "Selected folder does not contain manifest.xml")
            return
        self._load_memoq(dir_path)
    else:
        self._load_project(file_path)

def _load_project(self, file_path):
    try:
        self.state.load_project(file_path)
        self.current_segment_index = -1
        self.modified = False

        if self.state.project.source_files:
            self._open_project_source_file(0)
        else:
            self.table.setRowCount(0)

        self._update_recent_menu()
        self.update_stats()
        self._load_spelling_dict_for_language(self.state.project.target_lang)
        self.status_label.setText(f"Opened project: {self.state.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Opening Project", str(e))

def _load_omegat(self, dir_path):
    try:
        self.state.load_omegat(dir_path)
        self.current_segment_index = -1
        self.modified = False

        if self.state.project.source_files:
            self._open_project_source_file(0)
        else:
            self.table.setRowCount(0)

        self._update_recent_menu()
        self.update_stats()
        self._load_spelling_dict_for_language(self.state.project.target_lang)
        self.status_label.setText(f"Opened OmegaT project: {self.state.project.name}")

```

```

except Exception as e:
    QMessageBox.critical(self, "Error Opening Project", str(e))

def project_settings(self):
    if not self.state.project:
        QMessageBox.information(self, "No Project", "No project is currently open.")
        return

    dial = ProjectSettingsDialog(self.state.project, self)
    if dial.exec_() != QDialog.Accepted:
        return

    dial.apply()
    self.modified = True
    self.update_stats()
    self._load_spelling_dict_for_language(self.state.project.target_lang)
    self.status_label.setText("Project settings updated.")

def _open_project_source_file(self, index):
    segments, label, ok, error = self.state.open_project_source_file(index)
    if not ok:
        if error and "not found" in error.lower():
            QMessageBox.warning(self, "File Not Found", error)
        else:
            QMessageBox.critical(self, "Error Loading Source File", error or "Unknown error")
        return

    self.populate_table(segments)
    if segments:
        self.table.selectRow(0)
    self.update_stats()
    self.status_label.setText(f"Loaded: {label}")

def save_project(self):
    if not self.state.project:
        QMessageBox.information(self, "No Project", "No project is currently open.")
        return

    if not self.state.project.file_path:
        self.save_project_as()
        return

    try:
        self.state.save_project()
        self.modified = False
        self._update_recent_menu()
        self.update_stats()
        self.status_label.setText(f"Project saved: {self.state.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Saving Project", str(e))

def save_project_as(self):
    if not self.state.project:
        QMessageBox.information(self, "No Project", "No project is currently open.")
        return

    filter_str = "CAT Project (*.mcatproj);;OmegaT Project folder;;memoQ Project folder"
    file_path, selected_filter = QFileDialog.getSaveFileName(
        self, "Save Project As", f"{self.state.project.name}.mcatproj", filter_str
    )
    if not file_path:
        return

    if "OmegaT" in selected_filter:
        dir_path = QFileDialog.getExistingDirectory(
            self, "Select folder to create OmegaT project in",
            os.path.dirname(file_path) if os.path.dirname(file_path) else ""
        )
        if not dir_path:
            return
        target_dir = os.path.join(dir_path, self.state.project.name)
        try:
            self.state.save_project_as_omegat(target_dir)
            self.modified = False
            self._update_recent_menu()
            self.update_stats()
            self.status_label.setText(f"OmegaT project saved: {target_dir}")
            QMessageBox.information(
                self, "Success",
                f"Project exported to OmegaT format:\n{target_dir}\n\n"
                f"Source files copied to '{target_dir}\\source\\'.\n\n"
                f"You can open this folder directly in OmegaT."
            )
        except Exception as e:
            QMessageBox.critical(self, "Error Saving OmegaT Project", str(e))
    else:
        self.save_project()
        self.modified = False
        self._update_recent_menu()
        self.update_stats()
        self.status_label.setText(f"Project saved: {self.state.project.name}")


```

```

    )
    except Exception as e:
        QMessageBox.critical(self, "Export Error", str(e))
elif "memoQ" in selected_filter:
    dir_path = QFileDialog.getExistingDirectory(
        self, "Select folder to create memoQ project in",
        os.path.dirname(file_path) if os.path.dirname(file_path) else ""
    )
    if not dir_path:
        return
    target_dir = os.path.join(dir_path, self.state.project.name)
    try:
        self.state.save_project_as_memoq(target_dir, self.state.file_handler)
        self.modified = False
        self._update_recent_menu()
        self.update_stats()
        self.status_label.setText(f"memoQ project saved: {target_dir}")
        QMessageBox.information(
            self, "Success",
            f"Project exported to memoQ format:\n{target_dir}\n\n"
            f"Source files copied to '{target_dir}\\source\\'.\n"
            f"Translations saved as MQXLIFF in '{target_dir}\\target\\'.\n"
            f"You can open the .mqxliff file directly in memoQ."
        )
    except Exception as e:
        QMessageBox.critical(self, "Export Error", str(e))
else:
    if not file_path.endswith(".mcatproj"):
        file_path += ".mcatproj"
    try:
        self.state.save_project_as_mcatproj(file_path)
        self.modified = False
        self._update_recent_menu()
        self.update_stats()
        self.status_label.setText(f"Project saved: {self.state.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Saving Project", str(e))

def open_segmentation_rules(self):
    code = self.state.current_lang_code
    from cat_tool.segmentation import AdvancedRulesDialog
    dlg = AdvancedRulesDialog(self, self.state.segmentation_rules, lang_code=code)
    if dlg.exec_() == QDialog.Accepted:
        self.state.segmentation_rules = dlg.get_rules()
        self.status_label.setText(
            f"Segmentation rules saved to {code}.json ({len(self.state.segmentation_rules)} rules)"
        )

def import_omegat_project(self):
    dir_path = QFileDialog.getExistingDirectory(
        self, "Select OmegaT Project Folder (contains omegat.project)"
    )
    if not dir_path:
        return
    proj_file = os.path.join(dir_path, "omegat.project")
    if not os.path.exists(proj_file):
        QMessageBox.warning(self, "Invalid Project",
            "Selected folder does not contain omegat.project")
        return
    try:
        self.state.load_omegat(dir_path)
        self.current_segment_index = -1
        self.modified = False
        if self.state.project.source_files:
            self._open_project_source_file(0)
        else:
            self.table.setRowCount(0)
            self._update_recent_menu()
            self.update_stats()
            self.status_label.setText(f"Imported OmegaT project: {self.state.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Import Error", str(e))

# ----- MEMOQ PROJECTS -----
def import_memoq_project(self):
    dir_path = QFileDialog.getExistingDirectory(
        self, "Select memoQ Project Folder (contains manifest.xml)"
    )
    if not dir_path:

```

```

        return
    manifest_file = os.path.join(dir_path, "manifest.xml")
    if not os.path.exists(manifest_file):
        QMessageBox.warning(self, "Invalid Project",
                            "Selected folder does not contain manifest.xml")
        return
    self._load_memoq(dir_path)

def _load_memoq(self, dir_path):
    try:
        self.state.load_memoq(dir_path)
        self.current_segment_index = -1
        self.modified = False

        if self.state.project.mqxliff_files:
            self.state.file_handler.load_file(self.state.project.mqxliff_files[0])
            self.populate_table(self.state.file_handler.segments)
            if self.state.file_handler.segments:
                self.table.selectRow(0)
        elif self.state.project.source_files:
            self._open_project_source_file(0)
        else:
            self.table.setRowCount(0)

        self._update_recent_menu()
        self.update_stats()
        self._load_spelling_dict_for_language(self.state.project.target_lang)
        self.status_label.setText(f"Opened memoQ project: {self.state.project.name}")
    except Exception as e:
        QMessageBox.critical(self, "Error Opening memoQ Project", str(e))

# ----- SDL TRADOS -----
def import_sdlxliff(self):
    file_path, _ = QFileDialog.getOpenFileName(
        self, "Open SDL Trados SDLXLIFF", "", "SDLXLIFF (*.sdlxliff);;All Files (*)"
    )
    if not file_path:
        return
    self.status_label.setText("Loading SDLXLIFF...")
    self._ensure_seg_worker()
    self._seg_requested.emit(file_path)

def _update_recent_menu(self):
    self.recent_menu.clear()
    from cat_tool.project import Project
    recents = Project.get_recent()
    if not recents:
        action = self.recent_menu.addAction("No recent projects")
        action.setEnabled(False)
        return
    for r in recents:
        path = r.get("path", "")
        name = r.get("name", os.path.basename(path))
        action = self.recent_menu.addAction(f"{name} [{os.path.basename(os.path.dirname(path))}]"
        action.setData(path)
        action.triggered.connect(lambda checked, p=path: self._load_project(p))

```

## cat\_tooltm.py — 245 lines

- Class **TranslationMemory** (line 21)
- Function **\_load\_vec\_extension** (line 9)
- Function **\_\_init\_\_** (line 22)
- Function **\_init\_db** (line 29)
- Function **\_get\_embedding\_model** (line 56)
- Function **\_float32\_to\_int8** (line 66)
- Function **\_compute\_embedding** (line 70)
- Function **add\_translation** (line 82)
- Function **\_add\_embedding\_for\_row** (line 109)
- Function **clear** (line 128)
- Function **get\_fuzzy\_matches** (line 141)
- Function **get\_semantic\_matches** (line 182)

```

import sqlite3
import os
import sys
from datetime import datetime
from rapidfuzz import fuzz
import numpy as np

def _load_vec_extension(conn):
    conn.enable_load_extension(True)
    if getattr(sys, 'frozen', False):
        our_dir = os.path.dirname(os.path.abspath(__file__))
        dll_path = os.path.normpath(os.path.join(our_dir, '..', 'sqlite_vec', 'vec0.dll'))
        conn.load_extension(dll_path)
    else:
        import sqlite_vec
        sqlite_vec.load(conn)
    conn.enable_load_extension(False)

class TranslationMemory:
    def __init__(self, db_path="translation_memory.db"):
        self.db_path = db_path
        self._embedding_model = None
        self._match_cache = {}
        self._cache_max = 256
        self._init_db()

    def _init_db(self):
        conn = sqlite3.connect(self.db_path)
        _load_vec_extension(conn)
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS translation_memory (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                source TEXT UNIQUE,
                target TEXT,
                source_lang TEXT DEFAULT '',
                target_lang TEXT DEFAULT '',
                created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
        for col in ("source_lang", "target_lang"):
            try:
                cursor.execute(f"ALTER TABLE translation_memory ADD COLUMN {col} TEXT DEFAULT ''")
            except sqlite3.OperationalError:
                pass
        cursor.execute("""
            CREATE VIRTUAL TABLE IF NOT EXISTS vec_embeddings USING vec0(
                embedding int8[384] distance_metric=cosine
            )
        """)
        conn.commit()
        conn.close()

    def _get_embedding_model(self):
        if self._embedding_model is None:
            try:
                from fastembed import TextEmbedding
                self._embedding_model = TextEmbedding("sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")
            except Exception as e:
                print(f"Failed to load embedding model: {e}")
                self._embedding_model = False
        return self._embedding_model if self._embedding_model is not False else None

    def _float32_to_int8(self, vec):
        scaled = np.clip(vec * 127.0, -128.0, 127.0)
        return np.round(scaled).astype(np.int8)

    def _compute_embedding(self, text):
        model = self._get_embedding_model()
        if model is None or not text or not text.strip():
            return None
        try:
            embeddings = list(model.embed([text.strip()]))
            if embeddings:
                return self._float32_to_int8(np.array(embeddings[0], dtype=np.float32))
        except Exception as e:
            print(f"Embedding failed: {e}")

```

```

return None

def add_translation(self, source, target, source_lang="", target_lang=""):
    if not source or not source.strip():
        return None
    if not target or not target.strip():
        return None
    source = source.strip()
    target = target.strip()
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    try:
        cursor.execute("""
            INSERT INTO translation_memory (source, target, source_lang, target_lang)
            VALUES (?, ?, ?, ?)
            ON CONFLICT(source) DO UPDATE SET
                target = excluded.target,
                source_lang = excluded.source_lang,
                target_lang = excluded.target_lang
            """, (source, target, source_lang.strip(), target_lang.strip()))
        conn.commit()
        self._match_cache.clear()
        return cursor.lastrowid
    except Exception as e:
        print(f"Error adding to TM: {e}")
        return None
    finally:
        conn.close()

def _add_embedding_for_row(self, source, row_id):
    emb = self._compute_embedding(source)
    if emb is None:
        return
    conn = sqlite3.connect(self.db_path)
    _load_vec_extension(conn)
    cursor = conn.cursor()
    try:
        cursor.execute("DELETE FROM vec_embeddings WHERE rowid = ?", [row_id])
        cursor.execute(
            "INSERT INTO vec_embeddings(rowid, embedding) VALUES (?, vec_int8(?))",
            [row_id, emb.tobytes()]
        )
        conn.commit()
    except Exception as e:
        print(f"Error storing embedding: {e}")
    finally:
        conn.close()

def clear(self):
    conn = sqlite3.connect(self.db_path)
    _load_vec_extension(conn)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM translation_memory")
    conn.commit()
    try:
        cursor.execute("DELETE FROM vec_embeddings")
        conn.commit()
    except Exception:
        pass
    conn.close()

def get_fuzzy_matches(self, query_source, min_score=50.0, limit=5, source_lang=None, target_lang=None):
    if not query_source or not query_source.strip():
        return []

    query_source = query_source.strip()
    key = (query_source, min_score, limit, source_lang, target_lang)
    cached = self._match_cache.get(key)
    if cached is not None:
        return cached

    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("SELECT source, target, source_lang, target_lang FROM translation_memory")
    rows = cursor.fetchall()
    conn.close()

    q = query_source.lower()
    matches = []

```

```

for source, target, sl, tl in rows:
    if source_lang and sl and sl != source_lang:
        continue
    if target_lang and tl and tl != target_lang:
        continue
    s = source.lower()
    ratio = fuzz.ratio(q, s)
    sort_ratio = fuzz.token_sort_ratio(q, s)
    score = max(ratio, sort_ratio)
    if score >= min_score:
        matches.append({
            "source": source,
            "target": target,
            "score": round(score, 1)
        })

matches.sort(key=lambda x: x["score"], reverse=True)
result = matches[:limit]
self._match_cache[key] = result
if len(self._match_cache) > self._cache_max:
    self._match_cache.pop(next(iter(self._match_cache)))
return result

def get_semantic_matches(self, query_source, limit=5, source_lang=None, target_lang=None):
    if not query_source or not query_source.strip():
        return []

    model = self._get_embedding_model()
    if model is None:
        return []

    query_source = query_source.strip()
    try:
        query_emb = list(model.embed([query_source]))[0]
        query_int8 = self._float32_to_int8(np.array(query_emb, dtype=np.float32))
    except Exception as e:
        print(f"Query embedding failed: {e}")
        return []

    conn = sqlite3.connect(self.db_path)
    _load_vec_extension(conn)
    cursor = conn.cursor()

    try:
        vec_rows = cursor.execute(
            """
            SELECT rowid, distance
            FROM vec_embeddings
            WHERE embedding MATCH vec_int8(?)
            AND k = ?
            ORDER BY distance
            """,
            [query_int8.tobytes(), limit * 3]
        ).fetchall()
    except Exception as e:
        print(f"Sematic search failed: {e}")
        conn.close()
        return []

    q = query_source.lower()
    matches = []
    for row_id, distance in vec_rows:
        cursor.execute(
            "SELECT source, target, source_lang, target_lang FROM translation_memory WHERE id = ?",
            [row_id]
        )
        row = cursor.fetchone()
        if row is None:
            continue
        source, target, sl, tl = row
        if source_lang and sl and sl != source_lang:
            continue
        if target_lang and tl and tl != target_lang:
            continue
        if source.lower() == q:
            continue
        sim = max(0, (1.0 - distance) * 100)
        if sim > 0:
            matches.append({

```

```

        "source": source,
        "target": target,
        "score": round(sim, 1),
    })

conn.close()
matches.sort(key=lambda x: x["score"], reverse=True)
return matches[:limit]

```

## cat\_toolglossary.py — 360 lines

- Class **Glossary** (line 6)
- Function **\_\_init\_\_** (line 7)
- Function **\_init\_db** (line 11)
- Function **add\_term** (line 48)
- Function **remove\_term** (line 69)
- Function **get\_all\_terms** (line 77)
- Function **\_tokenize** (line 85)
- Function **peek\_tmx** (line 88)
- Function **import\_tmx** (line 119)
- Function **export\_tmx** (line 160)
- Function **create\_collection** (line 198)
- Function **delete\_collection** (line 211)
- Function **get\_all\_collections** (line 219)
- Function **add\_term\_to\_collection** (line 227)
- Function **remove\_term\_from\_collection** (line 238)
- Function **get\_collection\_terms** (line 246)
- Function **export\_collection\_tmx** (line 260)
- Function **import\_collection\_tmx** (line 289)
- Function **check\_segment** (line 329)

```

import sqlite3
import os
import re
import xml.etree.ElementTree as ET

class Glossary:
    def __init__(self, db_path="glossary.db"):
        self.db_path = db_path
        self._init_db()

    def _init_db(self):
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS glossary (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                source_term TEXT UNIQUE,
                target_translation TEXT,
                description TEXT,
                source_lang TEXT DEFAULT '',
                target_lang TEXT DEFAULT ''
            )
        """)
        for col in ("source_lang", "target_lang"):
            try:
                cursor.execute(f"ALTER TABLE glossary ADD COLUMN {col} TEXT DEFAULT ''")
            except sqlite3.OperationalError:
                pass
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS collections (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                name TEXT UNIQUE,
                description TEXT DEFAULT '',
                created_at TEXT DEFAULT CURRENT_TIMESTAMP
            )
        """)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS collection_terms (

```

```

        collection_id INTEGER,
        term_source TEXT,
        PRIMARY KEY (collection_id, term_source),
        FOREIGN KEY (collection_id) REFERENCES collections(id) ON DELETE CASCADE
    )
    """
    conn.commit()
    conn.close()

def add_term(self, source, target, description="", source_lang="", target_lang=""):
    if not source or not target:
        return
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    try:
        cursor.execute("""
            INSERT INTO glossary (source_term, target_translation, description, source_lang, target_lang)
            VALUES (?, ?, ?, ?, ?)
            ON CONFLICT(source_term) DO UPDATE SET
                target_translation = excluded.target_translation,
                description = excluded.description,
                source_lang = excluded.source_lang,
                target_lang = excluded.target_lang
            """, (source.strip(), target.strip(), description.strip(), source_lang.strip(), target_lang.strip()))
        conn.commit()
    except Exception as e:
        print(f"Error adding to Glossary: {e}")
    finally:
        conn.close()

def remove_term(self, source):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM glossary WHERE source_term = ?", (source.strip(),))
    cursor.execute("DELETE FROM collection_terms WHERE term_source = ?", (source.strip(),))
    conn.commit()
    conn.close()

def get_all_terms(self):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("SELECT source_term, target_translation, description, source_lang, target_lang FROM glossary")
    rows = cursor.fetchall()
    conn.close()
    return [{"source": r[0], "target": r[1], "description": r[2], "source_lang": r[3] or "", "target_lang": r[4] or ""} for r in rows]

def _tokenize(self, text):
    return re.findall(r"\w+(?:'\w+)?", text.lower())

def peek_tmx(self, file_path):
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"TMX file not found: {file_path}")
    tree = ET.parse(file_path)
    root = tree.getroot()
    XML_LANG = "http://www.w3.org/XML/1998/namespace:lang"

    header = root.find("header")
    srclang = header.get("srclang", "") if header is not None else ""

    body = root.find("body")
    if body is None:
        return {"source_lang": srclang, "target_lang": "", "count": 0}

    all_langs = set()
    seen_langs = set()
    tuv_count = 0
    for tu in body.findall("tu"):
        for tuv in tu.findall("tuv"):
            lang = tuv.get(XML_LANG) or tuv.get("lang") or ""
            seg = tuv.find("seg")
            seg_text = seg.text.strip() if seg is not None and seg.text else ""
            if seg_text:
                seen_langs.add(lang)
                tuv_count += 1

    total_pairs = tuv_count // 2
    target_langs = seen_langs - {srclang} if srclang else seen_langs
    target_lang = next(iter(target_langs)) if len(target_langs) == 1 else ", ".join(sorted(target_langs)) if target_langs else ""
    return {"source_lang": srclang, "target_lang": target_lang, "count": total_pairs}

```

```

def import_tmx(self, file_path, source_lang=None):
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"TMX file not found: {file_path}")
    tree = ET.parse(file_path)
    root = tree.getroot()
    XML_LANG = "{http://www.w3.org/XML/1998/namespace}lang"

    header = root.find("header")
    if header is not None and source_lang is None:
        source_lang = header.get("srclang", "en")
    if source_lang is None:
        source_lang = "en"

    body = root.find("body")
    if body is None:
        return 0

    count = 0
    for tu in body.findall("tu"):
        tuvs = tu.findall("tuv")
        source_text = None
        target_text = None
        target_lang = ""
        for tuv in tuvs:
            lang = tuv.get(XML_LANG) or tuv.get("lang") or ""
            seg = tuv.find("seg")
            seg_text = seg.text.strip() if seg is not None and seg.text else ""
            if not seg_text:
                continue
            if lang == source_lang:
                source_text = seg_text
            elif target_text is None:
                target_text = seg_text
                target_lang = lang
            if source_text and target_text:
                self.add_term(source_text, target_text,
                             f"Imported from TMX: {os.path.basename(file_path)}",
                             source_lang=source_lang, target_lang=target_lang)
                count += 1
    return count

def export_tmx(self, file_path, source_lang=None, target_lang=None):
    terms = self.get_all_terms()
    if not terms:
        return 0

    if source_lang is None:
        source_lang = terms[0].get("source_lang") or "en"
    if target_lang is None:
        target_lang = terms[0].get("target_lang") or ""

    XML_LANG = "{http://www.w3.org/XML/1998/namespace}lang"

    root = ET.Element("tmx", version="1.4")
    header = ET.SubElement(root, "header")
    header.set("srclang", source_lang)
    header.set("datatype", "plaintext")
    body = ET.SubElement(root, "body")

    for t in terms:
        tu = ET.SubElement(body, "tu")
        note = ET.SubElement(tu, "note")
        note.text = t.get("description", "")

        tuv_src = ET.SubElement(tu, "tuv")
        tuv_src.set(XML_LANG, t.get("source_lang") or source_lang)
        seg_src = ET.SubElement(tuv_src, "seg")
        seg_src.text = t["source"]

        t1 = t.get("target_lang") or target_lang
        tuv_tgt = ET.SubElement(tu, "tuv")
        tuv_tgt.set(XML_LANG, t1 if t1 else source_lang)
        seg_tgt = ET.SubElement(tuv_tgt, "seg")
        seg_tgt.text = t["target"]

    tree = ET.ElementTree(root)
    tree.write(file_path, encoding="utf-8", xml_declaration=True)
    return len(terms)

def create_collection(self, name, description=""):

```

```

conn = sqlite3.connect(self.db_path)
cursor = conn.cursor()
try:
    cursor.execute("INSERT INTO collections (name, description) VALUES (?, ?)",
                    (name.strip(), description.strip()))
    conn.commit()
    return cursor.lastrowid
except sqlite3.IntegrityError:
    return None
finally:
    conn.close()

def delete_collection(self, collection_id):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM collection_terms WHERE collection_id = ?", (collection_id,))
    cursor.execute("DELETE FROM collections WHERE id = ?", (collection_id,))
    conn.commit()
    conn.close()

def get_all_collections(self):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("SELECT id, name, description, created_at FROM collections ORDER BY name")
    rows = cursor.fetchall()
    conn.close()
    return [{"id": r[0], "name": r[1], "description": r[2], "created_at": r[3]} for r in rows]

def add_term_to_collection(self, collection_id, term_source):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    try:
        cursor.execute("INSERT OR IGNORE INTO collection_terms (collection_id, term_source) VALUES (?, ?)",
                        (collection_id, term_source.strip()))
        conn.commit()
        return cursor.rowcount > 0
    finally:
        conn.close()

def remove_term_from_collection(self, collection_id, term_source):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("DELETE FROM collection_terms WHERE collection_id = ? AND term_source = ?",
                    (collection_id, term_source.strip()))
    conn.commit()
    conn.close()

def get_collection_terms(self, collection_id):
    conn = sqlite3.connect(self.db_path)
    cursor = conn.cursor()
    cursor.execute("""
        SELECT g.source_term, g.target_translation, g.description, g.source_lang, g.target_lang
        FROM glossary g
        JOIN collection_terms ct ON g.source_term = ct.term_source
        WHERE ct.collection_id = ?
        ORDER BY g.source_term
    """, (collection_id,))
    rows = cursor.fetchall()
    conn.close()
    return [{"source": r[0], "target": r[1], "description": r[2], "source_lang": r[3] or "", "target_lang": r[4] or ""} for r in rows]

def export_collection_tmx(self, collection_id, file_path):
    terms = self.get_collection_terms(collection_id)
    if not terms:
        return 0
    source_lang = terms[0].get("source_lang") or "en"
    target_lang = terms[0].get("target_lang") or ""
    XML_LANG = "{http://www.w3.org/XML/1998/namespace}lang"
    root = ET.Element("tmx", version="1.4")
    header = ET.SubElement(root, "header")
    header.set("srclang", source_lang)
    header.set("datatype", "plaintext")
    body = ET.SubElement(root, "body")
    for t in terms:
        tu = ET.SubElement(body, "tu")
        note = ET.SubElement(tu, "note")
        note.text = t.get("description", "")
        tuv_src = ET.SubElement(tu, "tuv")
        tuv_src.set(XML_LANG, t.get("source_lang") or source_lang)

```

```

        seg_src = ET.SubElement(tuv_src, "seg")
        seg_src.text = t["source"]
        t1 = t.get("target_lang") or target_lang
        tuv_tgt = ET.SubElement(tu, "tuv")
        tuv_tgt.set(XML_LANG, t1 if t1 else source_lang)
        seg_tgt = ET.SubElement(tuv_tgt, "seg")
        seg_tgt.text = t["target"]
    tree = ET.ElementTree(root)
    tree.write(file_path, encoding="utf-8", xml_declaration=True)
    return len(terms)

def import_collection_tmx(self, file_path, name=None):
    if not os.path.exists(file_path):
        raise FileNotFoundError(f"TMX file not found: {file_path}")
    tree = ET.parse(file_path)
    root = tree.getroot()
    XML_LANG = "(http://www.w3.org/XML/1998/namespace)lang"
    header = root.find("header")
    srclang = header.get("srclang", "en") if header is not None else "en"
    body = root.find("body")
    if body is None:
        return 0, None
    coll_name = name or os.path.splitext(os.path.basename(file_path))[0]
    coll_id = self.create_collection(coll_name, f"Imported from {os.path.basename(file_path)}")
    if coll_id is None:
        return 0, None
    count = 0
    for tu in body.findall("tu"):
        tuvs = tu.findall("tuv")
        source_text = None
        target_text = None
        target_lang = ""
        for tuv in tuvs:
            lang = tuv.get(XML_LANG) or tuv.get("lang") or ""
            seg = tuv.find("seg")
            seg_text = seg.text.strip() if seg is not None and seg.text else ""
            if not seg_text:
                continue
            if lang == srclang:
                source_text = seg_text
            elif target_text is None:
                target_text = seg_text
                target_lang = lang
        if source_text and target_text:
            self.add_term(source_text, target_text,
                          f"Imported from TMX: {os.path.basename(file_path)}",
                          source_lang=srclang, target_lang=target_lang)
            self.add_term_to_collection(coll_id, source_text)
            count += 1
    return count, coll_id

def check_segment(self, text, source_lang=None, target_lang=None):
    if not text or not text.strip():
        return []

    tokens = self._tokenize(text)
    terms = self.get_all_terms()
    matched_terms = []

    for term in terms:
        if source_lang and term.get("source_lang") and term["source_lang"] != source_lang:
            continue
        if target_lang and term.get("target_lang") and term["target_lang"] != target_lang:
            continue

        src_lower = term["source"].lower()
        term_tokens = self._tokenize(src_lower)

        if not term_tokens:
            continue

        n_tokens = len(tokens)
        n_term = len(term_tokens)
        is_match = False
        for i in range(n_tokens - n_term + 1):
            if tokens[i:i+n_term] == term_tokens:
                is_match = True
                break
        if is_match:

```

```

        matched_terms.append(term)

    return matched_terms

```

## cat\_toolsegmentation.py — 380 lines

- Class **AdvancedRulesDialog** (line 92)
- Function **\_segrules\_dir** (line 31)
- Function **load\_rules** (line 37)
- Function **save\_rules** (line 48)
- Function **\_rule\_summary** (line 70)
- Function **advanced\_segmenter** (line 284)
- Function **\_is\_exception** (line 366)
- Function **\_\_init\_\_** (line 93)
- Function **\_build\_ui** (line 105)
- Function **\_rule\_summary** (line 185)
- Function **\_refresh\_list** (line 188)
- Function **\_on\_selection\_changed** (line 197)
- Function **\_clear\_form** (line 218)
- Function **\_save\_rule** (line 229)
- Function **\_start\_add** (line 249)
- Function **\_remove\_selected** (line 253)
- Function **\_apply** (line 268)
- Function **get\_rules** (line 280)

```

import re
import json
import os
from PyQt5.QtWidgets import (
    QDialog, QVBoxLayout, QHBoxLayout, QListWidget, QListWidgetItem,
    QLineEdit, QPushButton, QWidget, QCheckBox, QComboBox, QLabel,
    QGroupBox, QSplitter, QMessageBox,
)
from PyQt5.QtCore import Qt

LANGUAGES = [
    "All", "English", "Spanish", "French", "German", "Italian",
    "Portuguese", "Dutch", "Russian", "Arabic", "Chinese", "Japanese",
    "Korean", "Turkish", "Polish", "Swedish", "Danish", "Norwegian",
    "Finnish", "Greek", "Hebrew", "Hindi", "Thai", "Vietnamese",
    "Czech", "Slovak", "Hungarian", "Romanian", "Bulgarian", "Ukrainian",
]

LANG_NAME_TO_CODE = {
    "All": "all", "English": "en", "Spanish": "es", "French": "fr",
    "German": "de", "Italian": "it", "Portuguese": "pt", "Dutch": "nl",
    "Russian": "ru", "Arabic": "ar", "Chinese": "zh", "Japanese": "ja",
    "Korean": "ko", "Turkish": "tr", "Polish": "pl", "Swedish": "sv",
    "Danish": "da", "Norwegian": "no", "Finnish": "fi", "Greek": "el",
    "Hebrew": "he", "Hindi": "hi", "Thai": "th", "Vietnamese": "vi",
    "Czech": "cs", "Slovak": "sk", "Hungarian": "hu", "Romanian": "ro",
    "Bulgarian": "bg", "Ukrainian": "uk",
}

def _segrules_dir():
    d = os.path.join(os.environ.get("APPDATA", os.path.expanduser("~")), "mcat", "segrules")
    os.makedirs(d, exist_ok=True)
    return d

def load_rules(lang_code):
    path = os.path.join(_segrules_dir(), f"{lang_code}.json")
    if os.path.exists(path):
        try:
            with open(path, "r", encoding="utf-8") as f:
                return json.load(f)
            except (json.JSONDecodeError, OSError):
                pass
    return None

```

```

def save_rules(rules, lang_code):
    path = os.path.join(_segrules_dir(), f"{lang_code}.json")
    with open(path, "w", encoding="utf-8") as f:
        json.dump(rules, f, indent=2, ensure_ascii=False)

DEFAULT_RULES = [
    {"pattern": ".", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": "!", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": "?", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": ";", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": ":", "break_after": True, "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": "Mr", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Mrs", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Dr", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Ms", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Prof", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Sr", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
    {"pattern": "Jr", "break_after": False, "case_sensitive": True, "whole_word": True, "language": "English"},
]

def _rule_summary(rule):
    parts = [f"\{rule['pattern']}\"]
    ba = rule.get("break_after", False)
    bb = rule.get("break_before", False)
    if ba:
        parts.append("break after")
    elif bb:
        parts.append("break before")
    else:
        parts.append("no break")
    flags = []
    if rule.get("case_sensitive"):
        flags.append("case")
    if rule.get("whole_word"):
        flags.append("whole word")
    if flags:
        parts.append(", ".join(flags))
    lang = rule.get("language", "All")
    parts.append(f"[{lang}]")
    return " \u2014 ".join(parts)

class AdvancedRulesDialog(QDialog):
    def __init__(self, parent=None, rules=None, lang_code=None):
        super().__init__(parent)
        self.lang_code = lang_code or "all"
        title = f"Segmentation Rules - {self.lang_code}.json"
        self.setWindowTitle(title)
        self.setMinimumWidth(720)
        self.setMinimumHeight(500)
        self.rules = rules if rules is not None else [dict(r) for r in DEFAULT_RULES]
        self._modified_index = None
        self._build_ui()
        self._refresh_list()

    def _build_ui(self):
        outer = QVBoxLayout(self)

        splitter = QSplitter(Qt.Horizontal)

        left_panel = QWidget()
        left_layout = QVBoxLayout(left_panel)
        left_layout.setContentsMargins(0, 0, 0, 0)

        left_layout.addWidget(QLabel("Rules:"))
        self.rule_list = QListWidget()
        self.rule_list.currentRowChanged.connect(self._on_selection_changed)
        left_layout.addWidget(self.rule_list)

        list_btn_row = QHBoxLayout()
        self.add_btn = QPushButton("Add")
        self.add_btn.clicked.connect(self._start_add)
        self.remove_btn = QPushButton("Remove")
        self.remove_btn.clicked.connect(self._remove_selected)
        self.remove_btn.setEnabled(False)
        list_btn_row.addWidget(self.add_btn)
        list_btn_row.addWidget(self.remove_btn)

```

```

list_btn_row.addStretch()
left_layout.addLayout(list_btn_row)

splitter.addWidget(left_panel)

right_panel = QWidget()
right_layout = QVBoxLayout(right_panel)
right_layout.setContentsMargins(0, 0, 0, 0)

detail_group = QGroupBox("Rule Details")
detail_layout = QVBoxLayout(detail_group)

detail_layout.addWidget(QLabel("Pattern (word, punctuation, or phrase):"))
self.pattern_input = QLineEdit()
self.pattern_input.setPlaceholderText("e.g. Mr, ., !, ...")
detail_layout.addWidget(self.pattern_input)

detail_layout.addWidget(QLabel("Language:"))
self.lang_combo = QComboBox()
self.lang_combo.addItems(LANGUAGES)
detail_layout.addWidget(self.lang_combo)

detail_layout.addWidget(QLabel("Options:"))
self.break_after_cb = QCheckBox("Break after this pattern")
self.break_after_cb.setChecked(True)
self.break_before_cb = QCheckBox("Break before this pattern")
self.case_cb = QCheckBox("Case sensitive")
self.wv_cb = QCheckBox("Whole word only")
detail_layout.addWidget(self.break_after_cb)
detail_layout.addWidget(self.break_before_cb)
detail_layout.addWidget(self.case_cb)
detail_layout.addWidget(self.wv_cb)

detail_btn_row = QHBoxLayout()
self.save_btn = QPushButton("Add Rule")
self.save_btn.clicked.connect(self._save_rule)
self.save_btn.setStyleSheet("background-color: #0078d4; color: white;")
self.clear_btn = QPushButton("Clear")
self.clear_btn.clicked.connect(self._clear_form)
detail_btn_row.addWidget(self.save_btn)
detail_btn_row.addWidget(self.clear_btn)
detail_btn_row.addStretch()
detail_layout.addLayout(detail_btn_row)

detail_layout.addStretch()
right_layout.addWidget(detail_group)

apply_row = QHBoxLayout()
apply_row.addStretch()
self.apply_btn = QPushButton("Apply Rules")
self.apply_btn.clicked.connect(self._apply)
apply_row.addWidget(self.apply_btn)
right_layout.addLayout(apply_row)

splitter.addWidget(right_panel)
splitter.setSizes([300, 420])
outer.addWidget(splitter)

def _rule_summary(self, rule):
    return _rule_summary(rule)

def _refresh_list(self):
    self.rule_list.blockSignals(True)
    self.rule_list.clear()
    for rule in self.rules:
        item = QListWidgetItem(self._rule_summary(rule))
        self.rule_list.addItem(item)
    self.rule_list.blockSignals(False)
    self.remove_btn.setEnabled(self.rule_list.count() > 0)

def _on_selection_changed(self, row):
    if row < 0 or row >= len(self.rules):
        self._clear_form()
        self.remove_btn.setEnabled(False)
        return
    self.remove_btn.setEnabled(True)
    rule = self.rules[row]
    self._modified_index = row
    self.pattern_input.setText(rule["pattern"])

```

```

lang = rule.get("language", "All")
idx = self.lang_combo.findText(lang)
if idx >= 0:
    self.lang_combo.setCurrentIndex(idx)
else:
    self.lang_combo.setCurrentIndex(0)
self.break_after_cb.setChecked(rule.get("break_after", False))
self.break_before_cb.setChecked(rule.get("break_before", False))
self.case_cb.setChecked(rule.get("case_sensitive", False))
self.wv_cb.setChecked(rule.get("whole_word", False))
self.save_btn.setText("Update Rule")

def _clear_form(self):
    self._modified_index = None
    self.pattern_input.clear()
    self.lang_combo.setCurrentIndex(0)
    self.break_after_cb.setChecked(True)
    self.break_before_cb.setChecked(False)
    self.case_cb.setChecked(False)
    self.wv_cb.setChecked(False)
    self.save_btn.setText("Add Rule")
    self.rule_list.clearSelection()

def _save_rule(self):
    pattern = self.pattern_input.text().strip()
    if not pattern:
        QMessageBox.warning(self, "Missing Pattern", "Please enter a pattern.")
        return
    rule = {
        "pattern": pattern,
        "break_after": self.break_after_cb.isChecked(),
        "break_before": self.break_before_cb.isChecked(),
        "case_sensitive": self.case_cb.isChecked(),
        "whole_word": self.wv_cb.isChecked(),
        "language": self.lang_combo.currentText(),
    }
    if self._modified_index is not None and 0 <= self._modified_index < len(self.rules):
        self.rules[self._modified_index] = rule
    else:
        self.rules.append(rule)
    self._refresh_list()
    self._clear_form()

def _start_add(self):
    self._clear_form()
    self.pattern_input.setFocus()

def _remove_selected(self):
    row = self.rule_list.currentRow()
    if row < 0 or row >= len(self.rules):
        return
    reply = QMessageBox.question(
        self, "Remove Rule",
        f"Remove rule: {self._rule_summary(self.rules[row])}?",
        QMessageBox.Yes | QMessageBox.No, QMessageBox.No,
    )
    if reply != QMessageBox.Yes:
        return
    self.rules.pop(row)
    self._refresh_list()
    self._clear_form()

def _apply(self):
    if self.pattern_input.text().strip():
        self._save_rule()
    by_lang = {}
    for rule in self.rules:
        lang_name = rule.get("language", "All")
        code = LANG_NAME_TO_CODE.get(lang_name, "all")
        by_lang.setdefault(code, []).append(rule)
    for code, lang_rules in by_lang.items():
        save_rules(lang_rules, code)
    self.accept()

def get_rules(self):
    return self.rules

def advanced_segmenter(text, custom_rules=None, language=None, lang_code=None):

```

```

if not text:
    return []

rules = custom_rules
if rules is None and lang_code:
    lang_rules = []
    loaded = load_rules(lang_code)
    if loaded:
        lang_rules.extend(loaded)
    loaded_all = load_rules("all")
    if loaded_all:
        lang_rules.extend(loaded_all)
    if lang_rules:
        rules = lang_rules
if rules is None:
    rules = DEFAULT_RULES

if language and language != "All":
    rules = [r for r in rules if r.get("language", "All") in ("All", language)]
    if not rules:
        return text.splitlines()

after_rules = [r for r in rules if r.get("break_after", False)]
before_rules = [r for r in rules if r.get("break_before", False)]
exceptions = [r for r in rules if not r.get("break_after", False) and not r.get("break_before", False)]

try:
    positions = []

    # --- break-after: single-char splitters ---
    after_chars = set()
    for r in after_rules:
        if len(r["pattern"]) == 1 and not r.get("whole_word", False):
            after_chars.add(r["pattern"])

    if after_chars:
        cc = "".join(re.escape(c) for c in sorted(after_chars))
        for m in re.finditer(rf"({cc})\s+", text):
            if not _is_exception(text[:m.start()].rstrip(), exceptions):
                positions.append((m.start(), m.end()))

    # --- break-before: single-char splitters ---
    before_chars = set()
    for r in before_rules:
        if len(r["pattern"]) == 1 and not r.get("whole_word", False):
            before_chars.add(r["pattern"])

    if before_chars:
        cc = "".join(re.escape(c) for c in sorted(before_chars))
        for m in re.finditer(rf"(?<={cc})\s+", text):
            if not _is_exception(text[:m.start()].rstrip(), exceptions):
                positions.append((m.start(), m.start()))

    positions.sort(key=lambda x: (x[0], x[1]))

    # Merge overlapping / adjacent positions
    merged = []
    for p in positions:
        if merged and p[0] < merged[-1][1]:
            merged[-1] = (merged[-1][0], max(merged[-1][1], p[1]))
        else:
            merged.append(p)
    positions = merged

    if not positions:
        return [text]

    segments = []
    prev = 0
    for start, end in positions:
        if start > prev:
            segments.append(text[prev:start].strip())
            prev = end
    segments.append(text[prev:].strip())

    return [s for s in segments if s]

except re.error:
    return text.splitlines()

```

```

def _is_exception(prefix, exceptions):
    for r in exceptions:
        pat = r["pattern"]
        flags = 0 if r.get("case_sensitive", False) else re.IGNORECASE
        ww = r.get("whole_word", False)
        try:
            if ww:
                if re.search(rf"\b{re.escape(pat)}\b$", prefix, flags):
                    return True
            else:
                if re.search(rf"{re.escape(pat)}$", prefix, flags):
                    return True
        except re.error:
            continue
    return False

```

## cat\_toolssuggest.py — 149 lines

- Class **CompletionDelegate** (line 31)
- Function **\_\_init\_\_** (line 32)
- Function **set\_source\_texts** (line 42)
- Function **createEditor** (line 45)
- Function **setModelData** (line 147)
- Function **update\_model** (line 73)
- Function **on\_text\_changed** (line 130)
- Function **on\_typing** (line 137)

```

from PyQt5.QtWidgets import QStyledItemDelegate, QLineEdit, QCompleter, QListView
from PyQt5.QtCore import QStringListModel
from PyQt5.QtCore import Qt, QTimer

```

```

VSCODE_POPUP = """
QListView {
    background-color: #252526;
    border: 1px solid #3c3c3c;
    outline: none;
    padding: 2px;
    font-family: "Segoe UI", Consolas, "Courier New", monospace;
    font-size: 12px;
    color: #d4d4d4;
}
QListView::item {
    padding: 4px 8px;
    min-height: 20px;
    border: none;
}
QListView::item:selected {
    background-color: #094771;
    color: white;
}
QListView::item:hover {
    background-color: #2a2d2e;
}
"""

```

```

class CompletionDelegate(QStyledItemDelegate):
    def __init__(self, parent, tm, glossary, spellcheck=None, settings=None, source_lang=None, target_lang=None):
        super().__init__(parent)
        self.tm = tm
        self.glossary = glossary
        self.spellcheck = spellcheck
        self.settings = settings
        self.source_lang = source_lang
        self.target_lang = target_lang
        self._source_texts = []

    def set_source_texts(self, texts):
        self._source_texts = texts

    def createEditor(self, parent, option, index):
        editor = QLineEdit(parent)
        editor.setStyleSheet("font-size: 14px; padding: 2px;")

```

```

completer = QCompleter()
completer.setCaseSensitivity(Qt.CaseInsensitive)
completer.setFilterMode(Qt.MatchContains)

popup = QListView()
popup.setStyleSheet(VSCODE_POPUP)
popup.setWindowFlags(Qt.ToolTip)
popup.setFocusPolicy(Qt.StrongFocus)
completer.setPopup(popup)
completer.setMaxVisibleItems(15)
editor.setCompleter(completer)

source_text = ""
row = index.row()
if row < len(self._source_texts):
    source_text = self._source_texts[row]

min_score = 40.0
if self.settings:
    min_score = self.settings.get("fuzzy_match_min_score", 40.0)

debounce = QTimer(editor)
debounce.setSingleShot(True)
debounce.setInterval(180)

def update_model(current_text=None):
    all_items = []
    seen_targets = set()

    if source_text:
        tm_matches = self.tm.get_fuzzy_matches(source_text, min_score=min_score, limit=15, source_lang=self.source_lang, target_lang=self.target_lang)
        for m in tm_matches:
            target = m.get("target", "")
            score = m.get("score", 0)
            src = m.get("source", "")
            if not current_text or current_text.lower() in target.lower():
                if target.lower() not in seen_targets:
                    seen_targets.add(target.lower())
                    label = f"{int(score):>2}% {target}"
                    all_items.append((score, 0, label, target))

    if source_text:
        sem_matches = self.tm.get_semantic_matches(source_text, limit=5, source_lang=self.source_lang, target_lang=self.target_lang)
        for m in sem_matches:
            target = m.get("target", "")
            score = m.get("score", 0)
            if not current_text or current_text.lower() in target.lower():
                if target.lower() not in seen_targets:
                    seen_targets.add(target.lower())
                    label = f"SEM {int(score):>2}% {target}"
                    all_items.append((score, 1, label, target))

    if self.glossary and source_text:
        gl_terms = self.glossary.check_segment(source_text, source_lang=self.source_lang, target_lang=self.target_lang) if hasattr(self.glossary, 'check_segment') else []
        for term in gl_terms:
            target = term.get("target", "")
            src = term.get("source", "")
            if not current_text or current_text.lower() in target.lower():
                if target.lower() not in seen_targets:
                    seen_targets.add(target.lower())
                    label = f"GLOS {target} ({src})"
                    all_items.append((100, 2, label, target))

    if current_text and self.spellcheck and hasattr(self.spellcheck, 'suggest'):
        last_word = current_text.strip().split()[-1] if current_text.strip() else ""
        if last_word and len(last_word) > 2:
            sp_suggestions = self.spellcheck.suggest(last_word)
            for s in sp_suggestions[:5]:
                replacement = current_text.rstrip(last_word) + s
                if replacement.lower() not in seen_targets:
                    seen_targets.add(replacement.lower())
                    label = f"SPELL {replacement}"
                    all_items.append((90, 3, label, replacement))

    all_items.sort(key=lambda x: (-x[0], x[1]))
    suggestions = [item[2] for item in all_items[:30]]

    model = QStringListModel(suggestions)
    completer.setModel(model)

```

```

        if suggestions:
            completer.complete()

    def on_text_changed(text):
        debounce.stop()
        if text:
            debounce.start()
        else:
            update_model(text)

    def on_typing():
        update_model(editor.text())

    debounce.timeout.connect(on_typing)
    editor.textChanged.connect(on_text_changed)

    QTimer.singleShot(50, lambda: update_model())

    return editor

def setModelData(self, editor, model, index):
    text = editor.text()
    model.setData(index, text, Qt.EditRole)

```

## cat\_toolspellcheck.py — 135 lines

- Class **SpellChecker** (line 10)
- Function **\_\_init\_\_** (line 11)
- Function **auto\_load** (line 16)
- Function **\_lang\_id\_from\_archive** (line 34)
- Function **load\_from\_archive** (line 47)
- Function **load\_dictionary** (line 70)
- Function **\_load\_from\_dir** (line 85)
- Function **has\_dict** (line 101)
- Function **loaded\_names** (line 105)
- Function **check** (line 108)
- Function **suggest** (line 116)
- Function **check\_text** (line 123)

```

import os
import re
import shutil
import zipfile
from spylls.hunspell import Dictionary

DICTS_DIR = os.path.join(os.environ.get("APPDATA", os.path.expanduser("~")), "mcat", "dicts")

class SpellChecker:
    def __init__(self):
        self.dictionaries = []
        self._loaded_names = []
        os.makedirs(DICTS_DIR, exist_ok=True)

    def auto_load(self):
        for name in os.listdir(DICTS_DIR):
            dir_path = os.path.join(DICTS_DIR, name)
            if not os.path.isdir(dir_path):
                continue
            for f in os.listdir(dir_path):
                if f.endswith(".dic"):
                    base = os.path.join(dir_path, os.path.splitext(f)[0])
                    aff = base + ".aff"
                    if os.path.exists(aff):
                        try:
                            d = Dictionary.from_files(base)
                            self.dictionaries.append(d)
                            self._loaded_names.append(name)
                        except Exception:
                            pass
            break

    def _lang_id_from_archive(self, archive_path):

```

```

        candidates = []
        with zipfile.ZipFile(archive_path) as z:
            for name in z.namelist():
                if name.endswith(".dic"):
                    base = os.path.splitext(os.path.basename(name))[0]
                    if "_" in base or "-" in base:
                        candidates.append(base)
        if not candidates:
            base = os.path.splitext(os.path.basename(archive_path))[0]
            return base.replace(" ", "_").lower()
        return candidates[0]

def load_from_archive(self, archive_path):
    try:
        lang_id = self._lang_id_from_archive(archive_path)
    except Exception:
        return False
    target_dir = os.path.join(DICTS_DIR, lang_id)
    os.makedirs(target_dir, exist_ok=True)

    extracted = False
    with zipfile.ZipFile(archive_path) as z:
        for name in z.namelist():
            if name.endswith(".dic") or name.endswith(".aff"):
                dest = os.path.join(target_dir, os.path.basename(name))
                if not os.path.exists(dest):
                    with z.open(name) as src, open(dest, "wb") as dst:
                        shutil.copyfileobj(src, dst)
                    extracted = True

    if not extracted:
        return False

    return self._load_from_dir(target_dir, lang_id)

def load_dictionary(self, dic_path, aff_path=None):
    if aff_path is None:
        aff_path = os.path.splitext(dic_path)[0] + ".aff"
    if not os.path.exists(aff_path):
        return False

    base_name = os.path.splitext(os.path.basename(dic_path))[0]
    target_dir = os.path.join(DICTS_DIR, base_name)
    os.makedirs(target_dir, exist_ok=True)

    shutil.copy2(dic_path, os.path.join(target_dir, os.path.basename(dic_path)))
    shutil.copy2(aff_path, os.path.join(target_dir, os.path.basename(aff_path)))

    return self._load_from_dir(target_dir, base_name)

def _load_from_dir(self, dir_path, name):
    if name in self._loaded_names:
        return True
    for f in os.listdir(dir_path):
        if f.endswith(".dic"):
            base = os.path.join(dir_path, os.path.splitext(f)[0])
            try:
                d = Dictionary.from_files(base)
                self.dictionaries.append(d)
                self._loaded_names.append(name)
                return True
            except Exception:
                return False
    return False

@property
def has_dict(self):
    return len(self.dictionaries) > 0

@property
def loaded_names(self):
    return list(self._loaded_names)

def check(self, word):
    if not self.dictionaries:
        return True
    for d in self.dictionaries:
        if d.lookup(word):
            return True

```

```

        return False

    def suggest(self, word):
        suggestions = set()
        for d in self.dictionaries:
            for s in d.suggest(word):
                suggestions.add(s)
        return sorted(suggestions)

    def check_text(self, text):
        if not text:
            return []
        words = re.findall(r"\b[^\W\d_]+\b", text, re.UNICODE)
        errors = []
        seen = set()
        for word in words:
            if word.lower() in seen:
                continue
            if not self.check(word):
                seen.add(word.lower())
                errors.append((word, self.suggest(word)))
        return errors

```

## cat\_toolqa.py — 179 lines

- Class **QAIssue** (line 5)
- Function **\_extract\_numbers** (line 17)
- Function **\_count\_tags** (line 21)
- Function **check\_missing\_translations** (line 28)
- Function **check\_unedited\_fuzzy** (line 41)
- Function **check\_inconsistent\_translations** (line 53)
- Function **check\_tag\_mismatch** (line 74)
- Function **check\_number\_mismatch** (line 90)
- Function **check\_whitespace** (line 113)
- Function **check\_terminology** (line 147)
- Function **run\_all\_checks** (line 169)
- Function **\_\_init\_\_** (line 8)

```

import re
from collections import defaultdict

class QAIssue:
    __slots__ = ("severity", "check_name", "segment_index", "source", "target", "description")

    def __init__(self, severity, check_name, segment_index, source, target, description):
        self.severity = severity
        self.check_name = check_name
        self.segment_index = segment_index
        self.source = source
        self.target = target
        self.description = description

    def _extract_numbers(text):
        return set(re.findall(r"\d+(?:\.\d+)?", text))

    def _count_tags(text):
        if not text:
            return 0
        from cat_tool.formats.tag_utils import PH_L, PH_R
        return text.count(PH_L)

    def check_missing_translations(segments):
        issues = []
        for idx, seg in enumerate(segments):
            target = seg.get("target", "") or ""
            if not target.strip():
                issues.append(QAIssue(
                    "error", "Missing translation", idx,

```

```

        seg.get("source", ""), "",
        "Segment has no translation."
    ))
    return issues

def check_unedited_fuzzy(segments):
    issues = []
    for idx, seg in enumerate(segments):
        if seg.get("fuzzy", False):
            issues.append(QAIssue(
                "warning", "Unedited fuzzy match", idx,
                seg.get("source", ""), seg.get("target", ""),
                "Segment is still marked as fuzzy/needs review."
            ))
    return issues

def check_inconsistent_translations(segments):
    source_map = defaultdict(set)
    for seg in segments:
        src = seg.get("source", "").strip()
        tgt = seg.get("target", "").strip()
        if src and tgt:
            source_map[src].add(tgt)

    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "").strip()
        tgt = seg.get("target", "").strip()
        if src and tgt and len(source_map.get(src, set())) > 1:
            issues.append(QAIssue(
                "warning", "Inconsistent translation", idx,
                src, tgt,
                f"Source appears {len(source_map[src])} times with different translations: {' '.join(repr(t) for t in source_map[src])}"
            ))
    return issues

def check_tag_mismatch(segments):
    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        src_count = _count_tags(src)
        tgt_count = _count_tags(tgt)
        if src_count != tgt_count:
            issues.append(QAIssue(
                "error", "Tag mismatch", idx,
                src, tgt,
                f"Source has {src_count} tag(s), target has {tgt_count} tag(s)."
            ))
    return issues

def check_number_mismatch(segments):
    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        src_nums = _extract_numbers(src)
        tgt_nums = _extract_numbers(tgt)
        if src_nums and src_nums != tgt_nums:
            missing = src_nums - tgt_nums
            extra = tgt_nums - src_nums
            parts = []
            if missing:
                parts.append(f"missing: {' '.join(sorted(missing))}")
            if extra:
                parts.append(f"extra: {' '.join(sorted(extra))}")
            issues.append(QAIssue(
                "error", "Number mismatch", idx,
                src, tgt,
                "; ".join(parts)
            ))
    return issues

def check_whitespace(segments):

```

```

issues = []
for idx, seg in enumerate(segments):
    src = seg.get("source", "") or ""
    tgt = seg.get("target", "") or ""
    if not tgt.strip():
        continue
    if src.startswith(" ") and not tgt.startswith(" "):
        issues.append(QAIssue(
            "warning", "Leading whitespace", idx,
            src, tgt,
            "Source starts with whitespace but target does not."
        ))
    elif not src.startswith(" ") and tgt.startswith(" "):
        issues.append(QAIssue(
            "warning", "Leading whitespace", idx,
            src, tgt,
            "Target starts with whitespace but source does not."
        ))
    if src.endswith(" ") and not tgt.endswith(" "):
        issues.append(QAIssue(
            "warning", "Trailing whitespace", idx,
            src, tgt,
            "Source ends with whitespace but target does not."
        ))
    elif not src.endswith(" ") and tgt.endswith(" "):
        issues.append(QAIssue(
            "warning", "Trailing whitespace", idx,
            src, tgt,
            "Target ends with whitespace but source does not."
        ))
return issues

def check_terminology(segments, glossary):
    issues = []
    for idx, seg in enumerate(segments):
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        if not src.strip() or not tgt.strip():
            continue
        terms = glossary.check_segment(src)
        for term in terms:
            tgt_lower = tgt.lower()
            term_target = term.get("target", "").lower()
            if term_target and term_target not in tgt_lower:
                desc = term.get("description") or ""
                extra = f" ({desc})" if desc else ""
                issues.append(QAIssue(
                    "warning", "Terminology", idx,
                    src, tgt,
                    f"Glossary term '{term['source']}' should be '{term['target']}'{extra}."
                ))
    return issues

def run_all_checks(segments, glossary=None):
    issues = []
    issues.extend(check_missing_translations(segments))
    issues.extend(check_unedited_fuzzy(segments))
    issues.extend(check_inconsistent_translations(segments))
    issues.extend(check_tag_mismatch(segments))
    issues.extend(check_number_mismatch(segments))
    issues.extend(check_whitespace(segments))
    if glossary is not None:
        issues.extend(check_terminology(segments, glossary))
    return issues

```

## cat\_toolsettings.py — 46 lines

- Class **AppSettings** (line 15)
- Function **\_\_init\_\_** (line 16)
- Function **load** (line 19)
- Function **save** (line 28)
- Function **\_\_getitem\_\_** (line 33)
- Function **\_\_setitem\_\_** (line 36)

- Function **\_\_contains\_\_** (line 39)
- Function **get** (line 42)
- Function **as\_dict** (line 45)

```
import json
import os

SETTINGS_FILE = os.path.join(os.environ.get("APPDATA", os.path.expanduser("~")), "mcat", "settings.json")

DEFAULTS = {
    "source_font_size": 13,
    "target_font_size": 13,
    "auto_save_on_segment_change": False,
    "fuzzy_match_min_score": 40.0,
    "show_tag_placeholders": True,
}

class AppSettings:
    def __init__(self):
        self._data = dict(DEFAULTS)

    def load(self):
        try:
            with open(SETTINGS_FILE, encoding="utf-8") as f:
                stored = json.load(f)
                self._data = {**DEFAULTS, **stored}
        except (FileNotFoundError, json.JSONDecodeError):
            self._data = dict(DEFAULTS)
        return self

    def save(self):
        os.makedirs(os.path.dirname(SETTINGS_FILE), exist_ok=True)
        with open(SETTINGS_FILE, "w", encoding="utf-8") as f:
            json.dump(self._data, f, indent=2)

    def __getitem__(self, key):
        return self._data[key]

    def __setitem__(self, key, value):
        self._data[key] = value

    def __contains__(self, key):
        return key in self._data

    def get(self, key, default=None):
        return self._data.get(key, default)

    def as_dict(self):
        return dict(self._data)
```

## cat\_tool\project.py — 217 lines

- Class **Project** (line 11)
- Function **\_\_init\_\_** (line 12)
- Function **default\_dir** (line 22)
- Function **new** (line 27)
- Function **to\_dict** (line 37)
- Function **from\_dict** (line 49)
- Function **save** (line 60)
- Function **load** (line 71)
- Function **get\_recent** (line 77)
- Function **add\_recent** (line 88)
- Function **save\_as\_omegat** (line 100)
- Function **load\_omegat** (line 126)
- Function **save\_as\_memoq** (line 157)
- Function **load\_memoq** (line 184)
- Function **\_\_str\_\_** (line 216)

```

import json
import os
import shutil
import xml.etree.ElementTree as ET
from datetime import datetime

RECENT_FILE = "recent_projects.json"

class Project:
    def __init__(self):
        self.name = ""
        self.source_lang = ""
        self.target_lang = ""
        self.source_files = []
        self.file_path = None
        self.created_at = ""
        self.modified_at = ""

    @staticmethod
    def default_dir():
        d = os.path.join(os.environ.get("APPDATA", os.path.expanduser("~")), "mcat")
        os.makedirs(d, exist_ok=True)
        return d

    def new(self, name, source_lang, target_lang, source_files):
        now = datetime.now().isoformat()
        self.name = name
        self.source_lang = source_lang
        self.target_lang = target_lang
        self.source_files = list(source_files)
        self.file_path = None
        self.created_at = now
        self.modified_at = now

    def to_dict(self):
        return {
            "version": 1,
            "name": self.name,
            "source_lang": self.source_lang,
            "target_lang": self.target_lang,
            "source_files": self.source_files,
            "created_at": self.created_at,
            "modified_at": self.modified_at,
        }

    @staticmethod
    def from_dict(data, file_path=None):
        p = Project()
        p.name = data.get("name", "")
        p.source_lang = data.get("source_lang", "")
        p.target_lang = data.get("target_lang", "")
        p.source_files = data.get("source_files", [])
        p.file_path = file_path
        p.created_at = data.get("created_at", "")
        p.modified_at = data.get("modified_at", "")
        return p

    def save(self, file_path=None):
        if file_path:
            self.file_path = file_path
        if not self.file_path:
            raise ValueError("No file path set for project save")
        self.modified_at = datetime.now().isoformat()
        with open(self.file_path, "w", encoding="utf-8") as f:
            json.dump(self.to_dict(), f, indent=2, ensure_ascii=False)
        return self.file_path

    @staticmethod
    def load(file_path):
        with open(file_path, "r", encoding="utf-8") as f:
            data = json.load(f)
        return Project.from_dict(data, file_path)

    @staticmethod
    def get_recent():
        path = os.path.join(Project.default_dir(), RECENT_FILE)
        if not os.path.exists(path):

```

```

        return []
    try:
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    except Exception:
        return []

    @staticmethod
    def add_recent(file_path, name):
        recents = Project.get_recent()
        recents = [r for r in recents if r.get("path") != file_path]
        recents.insert(0, {"path": file_path, "name": name, "opened": datetime.now().isoformat()})
        recents = recents[:10]
        path = os.path.join(Project.default_dir(), RECENT_FILE)
        try:
            with open(path, "w", encoding="utf-8") as f:
                json.dump(recents, f, indent=2)
        except Exception:
            pass

    def save_as_omegat(self, dir_path):
        self.modified_at = datetime.now().isoformat()
        os.makedirs(dir_path, exist_ok=True)
        src_dir = os.path.join(dir_path, "source")
        tgt_dir = os.path.join(dir_path, "target")
        tm_dir = os.path.join(dir_path, "tm")
        glos_dir = os.path.join(dir_path, "glossary")
        for d in (src_dir, tgt_dir, tm_dir, glos_dir):
            os.makedirs(d, exist_ok=True)

        for f in self.source_files:
            if os.path.exists(f):
                shutil.copy2(f, os.path.join(src_dir, os.path.basename(f)))

        root = ET.Element("omegat")
        proj_el = ET.SubElement(root, "project")
        ET.SubElement(proj_el, "source_lang").text = self.source_lang.upper()
        ET.SubElement(proj_el, "target_lang").text = self.target_lang.upper()
        ET.SubElement(proj_el, "source_dir").text = "source"
        ET.SubElement(proj_el, "target_dir").text = "target"
        ET.SubElement(proj_el, "tm_dir").text = "tm"
        ET.SubElement(proj_el, "glossary_dir").text = "glossary"
        tree = ET.ElementTree(root)
        tree.write(os.path.join(dir_path, "omegat.project"), encoding="utf-8", xml_declaration=True)

    @staticmethod
    def load_omegat(dir_path):
        proj_file = os.path.join(dir_path, "omegat.project")
        if not os.path.exists(proj_file):
            raise FileNotFoundError(f"Not an OmegaT project: {dir_path}")
        tree = ET.parse(proj_file)
        root = tree.getroot()
        pe = root.find("project")
        if pe is None:
            raise ValueError("Invalid omegat.project: missing <project> element")

        p = Project()
        p.name = os.path.basename(dir_path)
        p.file_path = dir_path
        p.created_at = datetime.now().isoformat()
        p.modified_at = datetime.now().isoformat()

        raw_sl = (pe.findtext("source_lang") or "").strip()
        raw_tl = (pe.findtext("target_lang") or "").strip()
        p.source_lang = raw_sl.lower()
        p.target_lang = raw_tl.lower()

        source_dir_name = (pe.findtext("source_dir") or "source").strip()
        source_dir = os.path.join(dir_path, source_dir_name)
        p.source_files = []
        if os.path.isdir(source_dir):
            for entry in sorted(os.listdir(source_dir)):
                full = os.path.join(source_dir, entry)
                if os.path.isfile(full):
                    p.source_files.append(full)
        return p

    def save_as_memoq(self, dir_path, handler):
        self.modified_at = datetime.now().isoformat()

```

```

os.makedirs(dir_path, exist_ok=True)
src_dir = os.path.join(dir_path, "source")
tgt_dir = os.path.join(dir_path, "target")
os.makedirs(src_dir, exist_ok=True)
os.makedirs(tgt_dir, exist_ok=True)

for f in self.source_files:
    if os.path.exists(f):
        shutil.copy2(f, os.path.join(src_dir, os.path.basename(f)))

manifest = ET.Element("memoQProject")
ET.SubElement(manifest, "name").text = self.name
ET.SubElement(manifest, "source_lang").text = self.source_lang
ET.SubElement(manifest, "target_lang").text = self.target_lang
ET.SubElement(manifest, "created_at").text = self.created_at
ET.SubElement(manifest, "modified_at").text = self.modified_at
ET.SubElement(manifest, "tool").text = "mcat"
ET.indent(manifest)
tree = ET.ElementTree(manifest)
tree.write(os.path.join(dir_path, "manifest.xml"), encoding="utf-8", xml_declaration=True)

mqxliff_path = os.path.join(tgt_dir, f"{self.name}.mqxliff")
handler.save_as_mqxliff(mqxliff_path, self.source_lang, self.target_lang)

@staticmethod
def load_memoq(dir_path):
    manifest_file = os.path.join(dir_path, "manifest.xml")
    if not os.path.exists(manifest_file):
        raise FileNotFoundError(f"Not a memoQ project: {dir_path}")

    tree = ET.parse(manifest_file)
    root = tree.getroot()
    p = Project()
    p.name = (root.findtext("name") or "").strip()
    p.file_path = dir_path
    p.source_lang = (root.findtext("source_lang") or "en").strip()
    p.target_lang = (root.findtext("target_lang") or "es").strip()
    p.created_at = root.findtext("created_at") or ""
    p.modified_at = root.findtext("modified_at") or ""

    src_dir = os.path.join(dir_path, "source")
    p.source_files = []
    if os.path.isdir(src_dir):
        for entry in sorted(os.listdir(src_dir)):
            full = os.path.join(src_dir, entry)
            if os.path.isfile(full):
                p.source_files.append(full)

    tgt_dir = os.path.join(dir_path, "target")
    p.mqxliff_files = []
    if os.path.isdir(tgt_dir):
        for entry in sorted(os.listdir(tgt_dir)):
            if entry.endswith(".mqxliff"):
                p.mqxliff_files.append(os.path.join(tgt_dir, entry))

    return p

def __str__(self):
    return f"Project({self.name}, {self.source_lang}->{self.target_lang}, files={len(self.source_files)})"

```

## cat\_toolfile\_handler.py — 113 lines

- Class **CATFileHandler** (line 13)
- Function **convert\_to\_po** (line 102)
- Function **\_\_init\_\_** (line 14)
- Function **load\_file** (line 19)
- Function **\_atomic\_save** (line 46)
- Function **save\_file** (line 57)
- Function **save\_as\_mqxliff** (line 85)
- Function **render\_translated** (line 90)
- Function **do\_save** (line 65)

```

import os
from cat_tool.formats import FORMAT_HANDLERS, FACTORY_EXTS

```

```

from cat_tool.formats.tag_utils import PH_L, PH_R, TAG_RE, extract_all_tags, apply_tags, restore_tags
from cat_tool.formats.factory import FactoryHandler

```

```

SUPPORTED_MSG = (
    "Supported: MCAT.DB, PO, POT, XLIFF, TS, TMX, TBX, CSV, TXT, "
    "PDF, DOCX, PPTX, XLSX, XLS, ODT, ODS, ODP, MQXLIFF"
)

```

```

class CATFileHandler:
    def __init__(self):
        self.current_file_path = None
        self.segments = []
        self._handler = None

    def load_file(self, file_path):
        if not os.path.exists(file_path):
            raise FileNotFoundError(f"File not found: {file_path}")

        ext = os.path.splitext(file_path)[1].lower()
        if file_path.lower().endswith(".mcats.db"):
            ext = ".mcats.db"
        self.current_file_path = file_path

        handler_cls = FORMAT_HANDLERS.get(ext)
        if handler_cls is not None:
            self._handler = handler_cls()
            self.segments = self._handler.load(file_path)
            return self.segments

        if ext in FACTORY_EXTS:
            self._handler = FactoryHandler()
            self.segments = self._handler.load(file_path)
            return self.segments

        try:
            self._handler = FactoryHandler()
            self.segments = self._handler.load(file_path)
            return self.segments
        except Exception:
            raise ValueError(f"Unsupported file format: {ext}\n{SUPPORTED_MSG}")

    def _atomic_save(self, save_fn, target_path):
        dir_name = os.path.dirname(target_path) or "."
        tmp_path = os.path.join(dir_name, f".mcats_tmp_{os.getpid()}_{id(self)}")
        try:
            save_fn(tmp_path)
            os.replace(tmp_path, target_path)
        except BaseException:
            if os.path.exists(tmp_path):
                os.remove(tmp_path)
            raise

    def save_file(self, target_path=None):
        if not self.segments:
            raise ValueError("No file loaded to save.")
        save_path = target_path or self.current_file_path
        ext = os.path.splitext(save_path)[1].lower()
        if save_path.lower().endswith(".mcats.db"):
            ext = ".mcats.db"

        def do_save(dest):
            if ext == ".mqxliff":
                from cat_tool.formats.mqxliff import MqxliffHandler
                MqxliffHandler().save(self.segments, dest)
            elif ext == ".sdlxliiff":
                from cat_tool.formats.sdlxliiff import SdlxliiffHandler
                SdlxliiffHandler().save(self.segments, dest)
            elif ext == ".mcats.db":
                from cat_tool.formats.mcatsdb import McatsDbHandler
                McatsDbHandler().save(self.segments, dest)
            elif ext in FACTORY_EXTS:
                FactoryHandler().save(self.segments, dest)
            elif self._handler is not None and hasattr(self._handler, "save"):
                self._handler.save(self.segments, dest)
            else:
                raise ValueError(f"Cannot save to format: {ext}")

        self._atomic_save(do_save, save_path)

```

```

        self.current_file_path = save_path

    def save_as_mqxliif(self, output_path, src_lang="en", tgt_lang="es"):
        from cat_tool.formats.mqxliif import MqxliiffHandler
        MqxliiffHandler().save(self.segments, output_path, src_lang, tgt_lang)
        self.current_file_path = output_path

    def render_translated(self, output_path):
        if not self.segments:
            raise ValueError("No segments to render.")
        if not self.current_file_path:
            raise ValueError("No original file path; cannot render.")
        if self._handler is None:
            raise ValueError("No handler available for rendering.")
        if not hasattr(self._handler, "render"):
            raise ValueError(f"{type(self._handler).__name__} does not support rendering.")
        self._handler.render(self.current_file_path, self.segments, output_path)

def convert_to_po(source_path, output_path):
    from translate.storage.po import pofile
    handler = CATFileHandler()
    segments = handler.load_file(source_path)
    store = pofile()
    for seg in segments:
        unit = store.addsourceunit(seg["source"])
        unit.target = seg.get("target", "")
        if seg.get("fuzzy"):
            unit.markfuzzy()
    store.savefile(output_path)
    return output_path

```

## cat\_toolmcat\_format\_manager.py — 151 lines

- Class **MCatProjectManager** (line 43)
- Function **\_connect** (line 46)
- Function **create\_project\_file** (line 55)
- Function **import\_extracted\_segments** (line 81)
- Function **get\_editor\_stream** (line 117)
- Function **update\_segment\_translation** (line 137)

```

import sqlite3
import os
from datetime import datetime, timezone

SCHEMA_SQL = """
CREATE TABLE IF NOT EXISTS project_meta (
    key TEXT PRIMARY KEY,
    value TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS translation_units (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    file_id TEXT NOT NULL,
    source_text TEXT NOT NULL,
    target_text TEXT DEFAULT '',
    status TEXT DEFAULT 'Untranslated',
    match_score INTEGER DEFAULT 0
);

CREATE TABLE IF NOT EXISTS format_tags (
    unit_id INTEGER NOT NULL,
    tag_index TEXT NOT NULL,
    raw_xml_tag TEXT NOT NULL,
    PRIMARY KEY (unit_id, tag_index),
    FOREIGN KEY(unit_id) REFERENCES translation_units(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS project_glossary (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    source_term TEXT NOT NULL,
    target_term TEXT NOT NULL,
    definition TEXT DEFAULT ''
);

PRAGMA journal_mode = WAL;

```

```

PRAGMA synchronous = NORMAL;
PRAGMA foreign_keys = ON;
"""

ENGINE_VERSION = "1.0.0"

class MCatProjectManager:

    @staticmethod
    def _connect(file_path):
        conn = sqlite3.connect(file_path)
        conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA journal_mode = WAL")
        conn.execute("PRAGMA synchronous = NORMAL")
        conn.execute("PRAGMA foreign_keys = ON")
        return conn

    @staticmethod
    def create_project_file(file_path, src_lang, tgt_lang):
        if not file_path.endswith(".mcat.db"):
            file_path += ".mcat.db"
        os.makedirs(os.path.dirname(os.path.abspath(file_path)), exist_ok=True)

        conn = MCatProjectManager._connect(file_path)
        try:
            conn.executescript(SCHEMA_SQL)
            now = datetime.now(timezone.utc).isoformat()
            meta = [
                ("source_language", src_lang),
                ("target_language", tgt_lang),
                ("created_at", now),
                ("engine_version", ENGINE_VERSION),
            ]
            conn.executemany(
                "INSERT OR REPLACE INTO project_meta (key, value) VALUES (?, ?)", meta
            )
            conn.commit()
        except BaseException:
            conn.rollback()
            raise
        finally:
            conn.close()

    @staticmethod
    def import_extracted_segments(file_path, segments_list):
        conn = MCatProjectManager._connect(file_path)
        try:
            with conn:
                for seg in segments_list:
                    cursor = conn.execute(
                        """INSERT INTO translation_units
                        (file_id, source_text, target_text, status, match_score)
                        VALUES (?, ?, ?, ?, ?)""",
                        (
                            seg.get("file_id", ""),
                            seg.get("source", ""),
                            seg.get("target", ""),
                            seg.get("status", "Untranslated"),
                            seg.get("match_score", 0),
                        ),
                    )
                    unit_id = cursor.lastrowid
                    tags = seg.get("all_tags", [])
                    if tags:
                        tag_rows = [
                            (unit_id, tag.get("index", f"{{{i}}}" ), tag.get("raw", ""))
                            for i, tag in enumerate(tags)
                        ]
                        conn.executemany(
                            """INSERT OR IGNORE INTO format_tags
                            (unit_id, tag_index, raw_xml_tag) VALUES (?, ?, ?)""",
                            tag_rows,
                        )
        except BaseException:
            conn.rollback()
            raise
        finally:
            conn.close()

```

```

@staticmethod
def get_editor_stream(file_path, filter_status=None):
    conn = MCatProjectManager._connect(file_path)
    try:
        if filter_status:
            rows = conn.execute(
                "SELECT * FROM translation_units WHERE status = ? ORDER BY id",
                (filter_status,),
            ).fetchall()
        else:
            rows = conn.execute(
                "SELECT * FROM translation_units ORDER BY id"
            ).fetchall()
        return [dict(r) for r in rows]
    except BaseException:
        conn.rollback()
        raise
    finally:
        conn.close()

@staticmethod
def update_segment_translation(file_path, unit_id, target_text, status="Confirmed"):
    conn = MCatProjectManager._connect(file_path)
    try:
        conn.execute(
            """UPDATE translation_units
               SET target_text = ?, status = ?
               WHERE id = ?""",
            (target_text, status, unit_id),
        )
        conn.commit()
    except BaseException:
        conn.rollback()
        raise
    finally:
        conn.close()

```

## cat\_toolstate\_manager.py — 252 lines

- Class **AppState** (line 19)
- Function **\_get\_data\_dir** (line 13)
- Function **\_\_init\_\_** (line 27)
- Function **tm\_sl** (line 44)
- Function **tm\_tl** (line 48)
- Function **current\_lang\_code** (line 52)
- Function **has\_segments** (line 58)
- Function **segment\_count** (line 62)
- Function **current\_segment** (line 66)
- Function **\_load\_segmentation\_rules** (line 75)
- Function **new\_project** (line 89)
- Function **load\_project** (line 96)
- Function **load\_omegat** (line 103)
- Function **load\_memoq** (line 110)
- Function **close\_project** (line 117)
- Function **save\_project** (line 125)
- Function **save\_project\_as\_mcatproj** (line 131)
- Function **save\_project\_as\_omegat** (line 138)
- Function **save\_project\_as\_memoq** (line 144)
- Function **open\_project\_source\_file** (line 152)
- Function **import\_po\_to\_tm** (line 177)
- Function **clear\_tm** (line 187)
- Function **get\_fuzzy\_matches** (line 190)
- Function **load\_spelling\_dict** (line 198)
- Function **compute\_stats** (line 226)

```

"""Application state manager - owns backends, project state, DB operations."""
import os

```

```

from cat_tool.file_handler import CATFileHandler, convert_to_po
from cat_tool.tm import TranslationMemory
from cat_tool.glossary import Glossary
from cat_tool.project import Project
from cat_tool.segmentation import DEFAULT_RULES, load_rules
from cat_tool.settings import AppSettings
from cat_tool.spellcheck import SpellChecker, DICTS_DIR

def _get_data_dir():
    d = os.path.join(os.environ["APPDATA"], "mcat")
    os.makedirs(d, exist_ok=True)
    return d

class AppState:
    """Central state manager for the mcat application.

    Owns all backend services (file_handler, TM, glossary, etc.),
    project state, and provides business-logic methods that the
    MainWindow UI delegates to.
    """

    def __init__(self):
        data_dir = _get_data_dir()
        self.file_handler = CATFileHandler()
        self.tm = TranslationMemory(os.path.join(data_dir, "translation_memory.db"))
        self.glossary = Glossary(os.path.join(data_dir, "glossary.db"))
        self.project = None
        self.settings = AppSettings().load()
        self.spellchecker = SpellChecker()
        self.spellchecker.auto_load()
        self.segmentation_rules = [dict(r) for r in DEFAULT_RULES]
        self.current_segment_index = -1
        self.modified = False
        self._load_segmentation_rules()

    # ---- property helpers ----

    @property
    def tm_sl(self):
        return self.project.source_lang if self.project else ""

    @property
    def tm_tl(self):
        return self.project.target_lang if self.project else ""

    @property
    def current_lang_code(self):
        if self.project and self.project.source_lang:
            return self.project.source_lang
        return "all"

    @property
    def has_segments(self):
        return bool(self.file_handler.segments)

    @property
    def segment_count(self):
        return len(self.file_handler.segments) if self.file_handler.segments else 0

    @property
    def current_segment(self):
        if not self.file_handler.segments:
            return None
        if 0 <= self.current_segment_index < len(self.file_handler.segments):
            return self.file_handler.segments[self.current_segment_index]
        return None

    # ---- segmentation rules ----

    def _load_segmentation_rules(self):
        from cat_tool.segmentation import LANG_NAME_TO_CODE
        all_rules = []
        for code in set(LANG_NAME_TO_CODE.values()):
            loaded = load_rules(code)
            if loaded:
                all_rules.extend(loaded)
        if all_rules:

```

```

        self.segmentation_rules = all_rules
    else:
        self.segmentation_rules = [dict(r) for r in DEFAULT_RULES]

# ---- project operations ----

def new_project(self, name, src_lang, tgt_lang, src_files):
    self.project = Project()
    self.project.new(name, src_lang, tgt_lang, src_files)
    self.file_handler = CATFileHandler()
    self.current_segment_index = -1
    self.modified = False

def load_project(self, file_path):
    self.project = Project.load(file_path)
    self.file_handler = CATFileHandler()
    self.current_segment_index = -1
    self.modified = False
    Project.add_recent(file_path, self.project.name)

def load_omegat(self, dir_path):
    self.project = Project.load_omegat(dir_path)
    self.file_handler = CATFileHandler()
    self.current_segment_index = -1
    self.modified = False
    Project.add_recent(dir_path, self.project.name)

def load_memoq(self, dir_path):
    self.project = Project.load_memoq(dir_path)
    self.file_handler = CATFileHandler()
    self.current_segment_index = -1
    self.modified = False
    Project.add_recent(dir_path, self.project.name)

def close_project(self):
    proj_name = self.project.name if self.project else ""
    self.project = None
    self.file_handler = CATFileHandler()
    self.current_segment_index = -1
    self.modified = False
    return proj_name

def save_project(self):
    if self.project and self.project.file_path:
        self.project.save()
        Project.add_recent(self.project.file_path, self.project.name)
        self.modified = False

def save_project_as_mcatproj(self, file_path):
    if not file_path.endswith(".mcatproj"):
        file_path += ".mcatproj"
    self.project.save(file_path)
    self.modified = False
    Project.add_recent(file_path, self.project.name)

def save_project_as_omegat(self, target_dir):
    self.project.save_as_omegat(target_dir)
    self.modified = False
    self.project.file_path = target_dir
    Project.add_recent(target_dir, self.project.name)

def save_project_as_memoq(self, target_dir, file_handler):
    self.project.save_as_memoq(target_dir, file_handler)
    self.modified = False
    self.project.file_path = target_dir
    Project.add_recent(target_dir, self.project.name)

# ---- source file loading ----

def open_project_source_file(self, index):
    """Load a project source file. Returns (segments, display_label, is_error, error_msg)."""
    if not self.project or index >= len(self.project.source_files):
        return None, "", False, "No source file at that index"
    file_path = self.project.source_files[index]
    if not os.path.exists(file_path):
        return None, "", False, f"Source file not found:\n{file_path}"
    ext = os.path.splitext(file_path)[1].lower()
    is_po = ext in (".po", ".pot")
    try:

```

```

        if is_po:
            self.file_handler.load_file(file_path)
        else:
            po_path = file_path + ".po"
            convert_to_po(file_path, po_path)
            self.file_handler.load_file(po_path)
            label = os.path.basename(file_path)
            if not is_po:
                label += " (auto-converted to PO)"
            return self.file_handler.segments, label, True, None
    except Exception as e:
        return None, "", False, str(e)

# ---- TM operations ----

def import_po_to_tm(self, po_path):
    handler = CATFileHandler()
    segments = handler.load_file(po_path)
    imported_count = 0
    for seg in segments:
        if seg["target"] and seg["target"].strip() and not seg["fuzzy"]:
            self.tm.add_translation(seg["source"], seg["target"], self.tm_sl, self.tm_tl)
            imported_count += 1
    return imported_count, segments

def clear_tm(self):
    self.tm.clear()

def get_fuzzy_matches(self, source_text):
    return self.tm.get_fuzzy_matches(
        source_text, min_score=40.0,
        source_lang=self.tm_sl or None, target_lang=self.tm_tl or None
    )

# ---- spelling dictionary ----

def load_spelling_dict(self, lang_code):
    if not lang_code:
        return None
    code = lang_code.lower()
    found = None
    if os.path.isdir(DICTS_DIR):
        for name in os.listdir(DICTS_DIR):
            if name.lower().startswith(code):
                found = name
                break
    if found and found not in self.spellchecker.loaded_names:
        path = os.path.join(DICTS_DIR, found)
        for f in os.listdir(path):
            if f.endswith(".dic"):
                base = os.path.join(path, os.path.splitext(f)[0])
                try:
                    from spyls.hunspell import Dictionary
                    d = Dictionary.from_files(base)
                    self.spellchecker.dictionaries.append(d)
                    self.spellchecker._loaded_names.append(found)
                except Exception:
                    pass
            break
        return found
    return None

# ---- statistics ----

def compute_stats(self):
    """Return a dict with translation statistics."""
    if not self.file_handler.segments:
        return {"total": 0, "translated": 0, "fuzzy": 0,
                "total_words": 0, "total_chars": 0, "translated_words": 0, "percent": 0}
    total = len(self.file_handler.segments)
    translated = 0
    fuzzy = 0
    total_words = 0
    total_chars = 0
    translated_words = 0
    for seg in self.file_handler.segments:
        src = seg.get("source", "") or ""
        tgt = seg.get("target", "") or ""
        total_words += len(src.split())

```

```

        total_chars += len(src)
    if seg["fuzzy"]:
        fuzzy += 1
    elif tgt.strip():
        translated += 1
        translated_words += len(tgt.split())
    percent = int((translated / total) * 100) if total > 0 else 0
    return {
        "total": total, "translated": translated, "fuzzy": fuzzy,
        "total_words": total_words, "total_chars": total_chars,
        "translated_words": translated_words, "percent": percent,
    }

```

## cat\_toolworkers.py — 68 lines

- Class **SegmentationWorker** (line 4)
- Class **EmbeddingWorker** (line 33)
- Class **FuzzyMatchWorker** (line 51)
- Function **\_\_init\_\_** (line 8)
- Function **load\_file** (line 12)
- Function **open\_project\_source** (line 22)
- Function **\_\_init\_\_** (line 37)
- Function **add\_embedding** (line 41)
- Function **\_\_init\_\_** (line 54)
- Function **find\_matches** (line 58)

```

from PyQt5.QtCore import QObject, pyqtSignal

class SegmentationWorker(QObject):
    finished = pyqtSignal(list, str)
    error = pyqtSignal(str)

    def __init__(self):
        super().__init__()
        self._file_path = None

    def load_file(self, file_path):
        self._file_path = file_path
        try:
            from cat_tool.file_handler import CATFileHandler
            handler = CATFileHandler()
            segments = handler.load_file(file_path)
            self.finished.emit(segments, file_path)
        except Exception as e:
            self.error.emit(str(e))

    def open_project_source(self, index, project):
        try:
            segments, label, ok, error = project.open_project_source_file(index)
            if not ok:
                self.error.emit(error or "Unknown error opening source file")
                return
            self.finished.emit(segments, label)
        except Exception as e:
            self.error.emit(str(e))

class EmbeddingWorker(QObject):
    finished = pyqtSignal(int)
    error = pyqtSignal(str, int)

    def __init__(self, db_path):
        super().__init__()
        self.db_path = db_path

    def add_embedding(self, source, row_id):
        try:
            from cat_tool.tm import TranslationMemory
            tm = TranslationMemory(self.db_path)
            tm.add_embedding_for_row(source, row_id)
            self.finished.emit(row_id)
        except Exception as e:

```

```
        self.error.emit(str(e), row_id)

class FuzzyMatchWorker(QObject):
    finished = pyqtSignal(list)

    def __init__(self, db_path):
        super().__init__()
        self.db_path = db_path

    def find_matches(self, source_text, source_lang, target_lang, min_score=40.0, limit=5):
        try:
            from cat_tool.tm import TranslationMemory
            tm = TranslationMemory(self.db_path)
            matches = tm.get_fuzzy_matches(
                source_text, min_score=min_score, limit=limit,
                source_lang=source_lang, target_lang=target_lang
            )
            self.finished.emit(matches)
        except Exception:
            self.finished.emit([])
```

## 4. Format Handlers

### cat\_toolformats\\_\_init\_\_.py — 30 lines

```
from cat_tool.formats.tag_utils import PH_L, PH_R, TAG_RE, extract_all_tags, apply_tags, restore_tags
from cat_tool.formats.factory import FactoryHandler
from cat_tool.formats.mqxliif import MqxliifHandler
from cat_tool.formats.sdlxliff import SdlxliffHandler
from cat_tool.formats.text import TextHandler
from cat_tool.formats.pdf import PdfHandler
from cat_tool.formats.docx import DocxHandler
from cat_tool.formats.pptx import PptxHandler
from cat_tool.formats.xlsx import XlsxHandler
from cat_tool.formats.xls import XlsHandler
from cat_tool.formats.odf import OdfHandler
from cat_tool.formats.mcatdb import McatDbHandler

FORMAT_HANDLERS = {
    ".txt": TextHandler,
    ".pdf": PdfHandler,
    ".docx": DocxHandler,
    ".pptx": PptxHandler,
    ".ppsx": PptxHandler,
    ".xlsx": XlsxHandler,
    ".xls": XlsHandler,
    ".odt": OdfHandler,
    ".ods": OdfHandler,
    ".odp": OdfHandler,
    ".mqxliif": MqxliifHandler,
    ".sdlxliff": SdlxliffHandler,
    ".mcat.db": McatDbHandler,
}

FACTORY_EXTS = frozenset({"po", ".pot", ".mo", ".xlf", ".xliff", ".ts", ".tmx", ".tbx", ".csv"})
```

### cat\_toolformats\factory.py — 55 lines

- Class **FactoryHandler** (line 4)
- Function **load** (line 5)
- Function **save** (line 41)
- Function **render** (line 54)

```
from cat_tool.formats.tag_utils import extract_all_tags, apply_tags

class FactoryHandler:
    def load(self, file_path):
        from translate.storage.factory import getclass
        store_class = getclass(file_path)
        if not store_class:
            raise ValueError(f"Unsupported translation format: {file_path}")
        store = store_class.parsefile(file_path)
        segments = []
        for idx, unit in enumerate(store.units):
            if unit.isheader():
                continue
            locations = unit.getlocations()
            loc_str = ", ".join(locations) if locations else ""
            source_text = str(unit.source) if unit.source else ""
            target_text = str(unit.target) if unit.target else ""

            src_tags = extract_all_tags(source_text)
            tgt_tags = extract_all_tags(target_text)
            all_tags = list(src_tags)
            for t in tgt_tags:
                if t not in all_tags:
                    all_tags.append(t)
            source_clean = apply_tags(source_text, all_tags)
            target_clean = apply_tags(target_text, all_tags)

            segments.append({
                "index": idx, "source": source_text, "target": target_text,
                "source_clean": source_clean, "target_clean": target_clean,
```

```

        "all_tags": all_tags,
        "notes": unit.getnotes() or "",
        "fuzzy": unit.isfuzzy(), "translated": unit.istranslated(),
        "locations": loc_str,
        "source_location": {"unit_index": idx},
        "store": store, "unit": unit,
    })
    return segments

def save(self, segments, target_path):
    if not segments:
        raise ValueError("No segments to save.")
    store = segments[0].get("store")
    if store is None:
        raise ValueError("No store object available; cannot save factory format.")
    for seg in segments:
        unit = seg["unit"]
        unit.target = seg["target"]
        if hasattr(unit, "markfuzzy"):
            unit.markfuzzy(seg["fuzzy"])
    store.savefile(target_path)

def render(self, original_path, segments, output_path):
    self.save(segments, output_path)

```

## cat\_toolformats\text\_base.py — 27 lines

- Function **make\_segments\_from\_texts** (line 5)

```

from translate.storage.po import pofile
from cat_tool.formats.tag_utils import extract_all_tags, apply_tags

def make_segments_from_texts(texts, notes_prefix="", source_locations=None):
    store = pofile()
    segments = []
    for idx, text in enumerate(texts):
        if not text or not text.strip():
            continue
        all_tags = extract_all_tags(text)
        source_clean = apply_tags(text, all_tags)
        unit = store.addsourceunit(text)
        unit.source = text
        unit.target = ""
        loc = source_locations[idx] if source_locations and idx < len(source_locations) else {}
        segments.append({
            "index": idx, "source": text, "target": "",
            "source_clean": source_clean, "target_clean": "",
            "all_tags": all_tags,
            "notes": f"{notes_prefix}{idx + 1}" if notes_prefix else "",
            "fuzzy": False, "translated": False,
            "locations": f"{notes_prefix.strip().lower()}:{idx + 1}" if notes_prefix else "",
            "source_location": loc,
            "store": store, "unit": unit,
        })
    return segments

```

## cat\_toolformats>tag\_utils.py — 34 lines

- Function **extract\_all\_tags** (line 8)
- Function **apply\_tags** (line 17)
- Function **restore\_tags** (line 27)
- Function **\_repl** (line 21)
- Function **\_repl** (line 31)

```

import re

TAG_RE = re.compile(r'<[^\>]+\>')
PH_L = '\u27e6'
PH_R = '\u27e7'

def extract_all_tags(text):
    seen = []

```

```

for m in TAG_RE.finditer(text or ""):
    t = m.group(0)
    if t not in seen:
        seen.append(t)
return seen

def apply_tags(text, tag_list):
    if not text or not tag_list:
        return text
    tag_to_idx = {t: i for i, t in enumerate(tag_list)}
    def _repl(m):
        idx = tag_to_idx.get(m.group(0))
        return f'{PH_L}{idx}{PH_R}' if idx is not None else m.group(0)
    return TAG_RE.sub(_repl, text)

def restore_tags(text, tag_list):
    if not text or not tag_list:
        return text
    pat = re.escape(PH_L) + r'(\d+)' + re.escape(PH_R)
    def _repl(m):
        idx = int(m.group(1))
        return tag_list[idx] if idx < len(tag_list) else m.group(0)
    return re.sub(pat, _repl, text)

```

## cat\_tool\formats\text.py — 16 lines

- Class **TextHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 11)

```

from cat_tool.formats._text_base import make_segments_from_texts

class TextHandler:
    def load(self, file_path):
        with open(file_path, "r", encoding="utf-8", errors="replace") as f:
            lines = [l.strip() for l in f.readlines()]
            locations = [{"line": i} for i in range(len(lines))]
            return make_segments_from_texts(lines, "Line ", source_locations=locations)

    def render(self, original_path, segments, output_path):
        translated = []
        for seg in segments:
            translated.append(seg.get("target", "").strip() or seg["source"])
        with open(output_path, "w", encoding="utf-8") as f:
            f.write("\n".join(translated))

```

## cat\_tool\formats\pdf.py — 77 lines

- Class **PdfHandler** (line 8)
- Function **load** (line 9)
- Function **render** (line 37)

```

from cat_tool.formats._text_base import make_segments_from_texts
from reportlab.lib.pagesizes import letter
from reportlab.pdfgen import canvas
from pypdf import PdfReader, PdfWriter
import io

class PdfHandler:
    def load(self, file_path):
        import pdfplumber
        texts = []
        locations = []
        with pdfplumber.open(file_path) as pdf:
            for page_num, page in enumerate(pdf.pages):
                chars = page.chars
                if not chars:
                    continue
                chars.sort(key=lambda c: (c["top"], c["x0"]))
                lines = []

```

```

        current_line = []
        current_top = chars[0]["top"]
        for c in chars:
            if abs(c["top"] - current_top) > max(c.get("height", 8) * 0.5, 3):
                if current_line:
                    lines.append("".join(current_line))
                    current_line = []
                    current_top = c["top"]
                current_line.append(c.get("text", ""))
            if current_line:
                lines.append("".join(current_line))
        block_text = " ".join(line.strip() for line in lines if line.strip())
        if block_text.strip():
            texts.append(block_text.strip())
            locations.append({"page": page_num, "block": len(locations)})
        return make_segments_from_texts(texts, "Page ", source_locations=locations)

def render(self, original_path, segments, output_path):
    reader = PdfReader(original_path)
    writer = PdfWriter()

    page_data = {}
    for seg in segments:
        loc = seg.get("source_location", {})
        if not loc:
            continue
        page_num = loc.get("page", 0)
        target = seg.get("target", "").strip()
        source = seg.get("source", "").strip()
        page_data.setdefault(page_num, []).append({
            "source": source,
            "target": target or source,
            "index": seg.get("index", 0),
        })

    num_pages = len(reader.pages)
    for page_num in range(num_pages):
        packet = io.BytesIO()
        w, h = letter
        c = canvas.Canvas(packet, pagesize=letter)
        c.setFont("Helvetica", 10)
        y = h - 50
        segs = page_data.get(page_num, [])
        if segs:
            for s in segs:
                c.drawString(50, y, f"{s['target']}")
                y -= 16
        else:
            c.drawString(50, y, "(no translated content)")
        c.save()
        packet.seek(0)
        overlay = PdfReader(packet)
        page = reader.pages[page_num]
        page.merge_page(overlay.pages[0])
        writer.add_page(page)

    with open(output_path, "wb") as f:
        writer.write(f)

```

## cat\_tool\formats\docx.py — 47 lines

- Class **DocxHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 24)

```

from cat_tool.formats._text_base import make_segments_from_texts

class DocxHandler:
    def load(self, file_path):
        from docx import Document
        doc = Document(file_path)
        texts = []
        locations = []
        for i, p in enumerate(doc.paragraphs):
            t = p.text.strip()

```

```

        if t:
            texts.append(t)
            locations.append({"type": "paragraph", "index": i})
    for ti, table in enumerate(doc.tables):
        for ri, row in enumerate(table.rows):
            for ci, cell in enumerate(row.cells):
                t = cell.text.strip()
                if t:
                    texts.append(t)
                    locations.append({"type": "table_cell", "table_index": ti, "row_index": ri, "col_index": ci})
    return make_segments_from_texts(texts, "Para ", source_locations=locations)

def render(self, original_path, segments, output_path):
    from docx import Document
    doc = Document(original_path)
    seg_iter = iter(segments)
    for p in doc.paragraphs:
        t = p.text.strip()
        if t:
            seg = next(seg_iter, None)
            if seg and seg.get("source_location", {}).get("type") == "paragraph":
                target = seg.get("target", "").strip()
                if target:
                    p.clear()
                    p.add_run(target)
    for ti, table in enumerate(doc.tables):
        for ri, row in enumerate(table.rows):
            for ci, cell in enumerate(row.cells):
                t = cell.text.strip()
                if t:
                    seg = next(seg_iter, None)
                    if seg and seg.get("source_location", {}).get("type") == "table_cell":
                        target = seg.get("target", "").strip()
                        if target:
                            cell.text = target
    doc.save(output_path)

```

## cat\_tool\formats\pptx.py — 43 lines

- Class **PptxHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 27)

```

from cat_tool.formats._text_base import make_segments_from_texts

class PptxHandler:
    def load(self, file_path):
        from pptx import Presentation
        prs = Presentation(file_path)
        texts = []
        locations = []
        for slide_num, slide in enumerate(prs.slides):
            for shape_idx, shape in enumerate(slide.shapes):
                if shape.has_text_frame:
                    for para_idx, para in enumerate(shape.text_frame.paragraphs):
                        t = para.text.strip()
                        if t:
                            texts.append(t)
                            locations.append({"slide": slide_num, "shape": shape_idx, "paragraph": para_idx})
                if shape.has_table:
                    for ri, row in enumerate(shape.table.rows):
                        for ci, cell in enumerate(row.cells):
                            t = cell.text.strip()
                            if t:
                                texts.append(t)
                                locations.append({"slide": slide_num, "shape": shape_idx, "table_row": ri, "table_col": ci})
        return make_segments_from_texts(texts, "Slide ", source_locations=locations)

    def render(self, original_path, segments, output_path):
        from pptx import Presentation
        prs = Presentation(original_path)
        for seg in segments:
            loc = seg.get("source_location", {})
            if not loc:
                continue

```

```

        target = seg.get("target", "").strip()
        if not target:
            continue
        slide = prs.slides[loc["slide"]]
        shape = slide.shapes[loc["shape"]]
        if "paragraph" in loc:
            shape.text_frame.paragraphs[loc["paragraph"]].text = target
        elif "table_row" in loc:
            shape.table.cell(loc["table_row"], loc["table_col"]).text = target
    prs.save(output_path)

```

## cat\_toolformats\xlsx.py — 33 lines

- Class **XlsxHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 21)

```

from cat_tool.formats._text_base import make_segments_from_texts

class XlsxHandler:
    def load(self, file_path):
        from openpyxl import load_workbook
        wb = load_workbook(file_path, read_only=True, data_only=True)
        texts = []
        locations = []
        for si, ws in enumerate(wb.worksheets):
            for ri, row in enumerate(ws.iter_rows(values_only=True)):
                for ci, cell in enumerate(row):
                    if cell is not None:
                        t = str(cell).strip()
                        if t:
                            texts.append(t)
                            locations.append({"sheet": si, "row": ri, "col": ci})
        wb.close()
        return make_segments_from_texts(texts, "Cell ", source_locations=locations)

    def render(self, original_path, segments, output_path):
        from openpyxl import load_workbook
        wb = load_workbook(original_path)
        for seg in segments:
            loc = seg.get("source_location", {})
            if not loc:
                continue
            target = seg.get("target", "").strip()
            if not target:
                continue
            ws = wb.worksheets[loc["sheet"]]
            ws.cell(row=loc["row"] + 1, column=loc["col"] + 1, value=target)
        wb.save(output_path)

```

## cat\_toolformats\xls.py — 39 lines

- Class **XlsHandler** (line 4)
- Function **load** (line 5)
- Function **render** (line 22)

```

from cat_tool.formats._text_base import make_segments_from_texts

class XlsHandler:
    def load(self, file_path):
        import xlrd
        wb = xlrd.open_workbook(file_path)
        texts = []
        locations = []
        for si in range(wb.nsheets):
            ws = wb.sheet_by_index(si)
            for ri in range(ws.nrows):
                for ci in range(ws.ncols):
                    cell = ws.cell(ri, ci)
                    if cell.ctype != xlrd.XL_CELL_EMPTY:
                        t = str(cell.value).strip()
                        if t:

```

```

        texts.append(t)
        locations.append({"sheet": si, "row": ri, "col": ci})
    return make_segments_from_texts(texts, "Cell ", source_locations=locations)

def render(self, original_path, segments, output_path):
    try:
        import xlrd
        from xlutils.copy import copy
    except ImportError:
        raise ImportError("xlutils is required to render .xls files. Install it with: pip install xlutils")
    rb = xlrd.open_workbook(original_path, formatting_info=True)
    wb = copy(rb)
    for seg in segments:
        loc = seg.get("source_location", {})
        if not loc:
            continue
        target = seg.get("target", "").strip()
        if not target:
            continue
        ws = wb.get_sheet(loc["sheet"])
        ws.write(loc["row"], loc["col"], target)
    wb.save(output_path)

```

## cat\_tool\formats\odf.py — 102 lines

- Class **OdFHandler** (line 4)
- Function **\_is\_in\_table\_cell** (line 6)
- Function **load** (line 15)
- Function **render** (line 52)

```

from cat_tool.formats._text_base import make_segments_from_texts

class OdFHandler:
    @staticmethod
    def _is_in_table_cell(element):
        from odf.table import TableCell
        parent = getattr(element, 'parent', None)
        while parent is not None:
            if isinstance(parent, TableCell):
                return True
            parent = getattr(parent, 'parent', None)
        return False

    def load(self, file_path):
        from odf.opendocument import load
        from odf.text import P, H
        from odf.table import Table, TableRow, TableCell
        from odf.element import Text as OdFText
        doc = load(file_path)
        texts = []
        locations = []
        para_counter = 0
        heading_counter = 0
        table_counter = 0

        for p in doc.getElementsByType(P):
            if self._is_in_table_cell(p):
                continue
            t = "".join(n.data for n in p.childNodes if n.nodeType == OdFText.TEXT_NODE).strip()
            if t:
                texts.append(t)
                locations.append({"type": "paragraph", "index": para_counter})
                para_counter += 1
        for h in doc.getElementsByType(H):
            t = "".join(n.data for n in h.childNodes if n.nodeType == OdFText.TEXT_NODE).strip()
            if t:
                texts.append(t)
                locations.append({"type": "heading", "index": heading_counter})
                heading_counter += 1
        for table in doc.getElementsByType(Table):
            for ri, row in enumerate(table.getElementsByType(TableRow)):
                for ci, cell in enumerate(row.getElementsByType(TableCell)):
                    t = "".join(n.data for n in cell.childNodes if n.nodeType == OdFText.TEXT_NODE).strip()
                    if t:
                        texts.append(t)

```

```

        locations.append({"type": "table_cell", "table_index": table_counter, "row_index": ri, "col_index": ci})
        table_counter += 1

    return make_segments_from_texts(texts, "ODF ", source_locations=locations)

def render(self, original_path, segments, output_path):
    from odf.opendocument import load
    from odf.text import P, H
    from odf.table import Table, TableRow, TableCell
    from odf.element import Text as OdfText
    from copy import deepcopy

    doc = load(original_path)
    seg_iter = iter(segments)

    for p in doc.getElementsByType(P):
        if self._is_in_table_cell(p):
            continue
        t = "".join(n.data for n in p.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
        if t:
            seg = next(seg_iter, None)
            if seg and seg.get("source_location", {}).get("type") == "paragraph":
                target = seg.get("target", "").strip()
                if target:
                    for child in list(p.childNodes):
                        if child.nodeType == OdfText.TEXT_NODE:
                            child.data = target
                            break

    for h in doc.getElementsByType(H):
        t = "".join(n.data for n in h.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
        if t:
            seg = next(seg_iter, None)
            if seg and seg.get("source_location", {}).get("type") == "heading":
                target = seg.get("target", "").strip()
                if target:
                    for child in list(h.childNodes):
                        if child.nodeType == OdfText.TEXT_NODE:
                            child.data = target
                            break

    for table in doc.getElementsByType(Table):
        for ri, row in enumerate(table.getElementsByType(TableRow)):
            for ci, cell in enumerate(row.getElementsByType(TableCell)):
                t = "".join(n.data for n in cell.childNodes if n.nodeType == OdfText.TEXT_NODE).strip()
                if t:
                    seg = next(seg_iter, None)
                    if seg and seg.get("source_location", {}).get("type") == "table_cell":
                        target = seg.get("target", "").strip()
                        if target:
                            for child in list(cell.childNodes):
                                if child.nodeType == OdfText.TEXT_NODE:
                                    child.data = target
                                    break

    doc.save(output_path)

```

## cat\_tool\formats\mqxliff.py — 67 lines

- Class **MqxliffHandler** (line 6)
- Function **load** (line 7)
- Function **save** (line 42)

```

import os
import xml.etree.ElementTree as ET
from cat_tool.formats.tag_utils import extract_all_tags, apply_tags

```

```

class MqxliffHandler:
    def load(self, file_path):
        ns = "urn:oasis:names:tc:xliff:document:1.2"
        segments = []
        tree = ET.parse(file_path)
        root = tree.getroot()
        body = root.find(f".//{{{ns}}}body")
        if body is None:

```

```

        raise ValueError("Invalid MQXLIFF: no <body> element")
    for idx, unit_el in enumerate(body.findall(f"{{{ns}}}trans-unit")):
        src_el = unit_el.find(f"{{{ns}}}source")
        tgt_el = unit_el.find(f"{{{ns}}}target")
        src_text = "".join(src_el.itertext()).strip() if src_el is not None else ""
        tgt_text = "".join(tgt_el.itertext()).strip() if tgt_el is not None else ""

        if not src_text:
            continue

        src_tags = extract_all_tags(src_text)
        tgt_tags = extract_all_tags(tgt_text)
        all_tags = list(src_tags)
        for t in tgt_tags:
            if t not in all_tags:
                all_tags.append(t)
        source_clean = apply_tags(src_text, all_tags)
        target_clean = apply_tags(tgt_text, all_tags)

        segments.append({
            "index": idx, "source": src_text, "target": tgt_text,
            "source_clean": source_clean, "target_clean": target_clean,
            "all_tags": all_tags, "notes": "",
            "fuzzy": False, "translated": bool(tgt_text.strip()),
            "locations": "", "store": None, "unit": None,
        })
    return segments

def save(self, segments, output_path, src_lang="en", tgt_lang="es"):
    ns = "urn:oasis:names:tc:xliff:document:1.2"
    xsi_ns = "http://www.w3.org/2001/XMLSchema-instance"
    ET.register_namespace("", ns)
    ET.register_namespace("xsi", xsi_ns)

    root = ET.Element(f"{{{ns}}}xliff", {"version": "1.2"})
    root.set(f"{{{xsi_ns}}}schemaLocation", f"{{{ns}}} xliff-core-1.2-transitional.xsd")

    file_el = ET.SubElement(root, f"{{{ns}}}file", {
        "original": os.path.basename(output_path),
        "source-language": src_lang,
        "target-language": tgt_lang,
        "datatype": "x-memoq",
    })

    body = ET.SubElement(file_el, f"{{{ns}}}body")
    for idx, seg in enumerate(segments):
        unit_el = ET.SubElement(body, f"{{{ns}}}trans-unit", {"id": str(idx + 1)})
        src_el = ET.SubElement(unit_el, f"{{{ns}}}source")
        src_el.text = seg["source"]
        tgt_el = ET.SubElement(unit_el, f"{{{ns}}}target")
        tgt_el.text = seg["target"] if seg["target"] else ""

    tree = ET.ElementTree(root)
    tree.write(output_path, encoding="utf-8", xml_declaration=True)

```

## cat\_toolformats\sdlxliff.py — 109 lines

- Class **SdlxliffHandler** (line 10)
- Function **load** (line 11)
- Function **save** (line 67)

```

import os
import xml.etree.ElementTree as ET
from cat_tool.formats.tag_utils import extract_all_tags, apply_tags

```

```

XLF_NS = "urn:oasis:names:tc:xliff:document:1.2"
SDL_NS = "http://sdl.com/FileTypes/SdlXliff/1.0"

```

```

class SdlxliffHandler:
    def load(self, file_path):
        segments = []
        tree = ET.parse(file_path)
        root = tree.getroot()
        for idx, unit_el in enumerate(root.findall(f".//{{{XLF_NS}}}trans-unit")):

```

```

        if unit_el.get("translate") == "no":
            continue

        src_text = ""
        tgt_text = ""

        seg_source_el = unit_el.find(f"{{{XLF_NS}}}seg-source")
        target_el = unit_el.find(f"{{{XLF_NS}}}target")
        source_el = unit_el.find(f"{{{XLF_NS}}}source")

        if seg_source_el is not None:
            src_parts = []
            for mrk in seg_source_el.findall(f"{{{XLF_NS}}}mrk"):
                src_parts.append("".join(mrk.itertext()).strip())
            src_text = " ".join(p for p in src_parts if p)
        elif source_el is not None:
            src_text = "".join(source_el.itertext()).strip()

        if target_el is not None:
            mrks = target_el.findall(f"{{{XLF_NS}}}mrk")
            if mrks:
                tgt_parts = []
                for mrk in mrks:
                    tgt_parts.append("".join(mrk.itertext()).strip())
                tgt_text = " ".join(p for p in tgt_parts if p)
            else:
                tgt_text = "".join(target_el.itertext()).strip()

        if not src_text:
            continue

        src_tags = extract_all_tags(src_text)
        tgt_tags = extract_all_tags(tgt_text)
        all_tags = list(src_tags)
        for t in tgt_tags:
            if t not in all_tags:
                all_tags.append(t)
        source_clean = apply_tags(src_text, all_tags)
        target_clean = apply_tags(tgt_text, all_tags)

        segments.append({
            "index": idx, "source": src_text, "target": tgt_text,
            "source_clean": source_clean, "target_clean": target_clean,
            "all_tags": all_tags, "notes": "",
            "fuzzy": False, "translated": bool(tgt_text.strip()),
            "locations": "", "store": None, "unit": None,
        })

    return segments

def save(self, segments, output_path, src_lang="en", tgt_lang="es"):
    ET.register_namespace("", XLF_NS)
    ET.register_namespace("sdl", SDL_NS)
    ET.register_namespace("xsi", "http://www.w3.org/2001/XMLSchema-instance")

    root = ET.Element(f"{{{XLF_NS}}}xliff", {
        "version": "1.2",
        f"{{{SDL_NS}}}version": "1.0",
    })

    file_el = ET.SubElement(root, f"{{{XLF_NS}}}file", {
        "original": os.path.basename(output_path),
        "datatype": "x-sdlfilterframework2",
        "source-language": src_lang,
        "target-language": tgt_lang,
    })

    header = ET.SubElement(file_el, f"{{{XLF_NS}}}header")
    body = ET.SubElement(file_el, f"{{{XLF_NS}}}body")

    for idx, seg in enumerate(segments):
        unit_el = ET.SubElement(body, f"{{{XLF_NS}}}trans-unit", {"id": str(idx + 1)})

        src_el = ET.SubElement(unit_el, f"{{{XLF_NS}}}source")
        src_el.text = seg["source"]

        seg_src = ET.SubElement(unit_el, f"{{{XLF_NS}}}seg-source")
        mrk_src = ET.SubElement(seg_src, f"{{{XLF_NS}}}mrk", {"mtype": "seg", "mid": "1"})
        mrk_src.text = seg["source"]

```

```

tgt_el = ET.SubElement(unit_el, f"{{{XLF_NS}}}target")
if seg["target"]:
    mrk_tgt = ET.SubElement(tgt_el, f"{{{XLF_NS}}}mrk", {"mtype": "seg", "mid": "1"})
    mrk_tgt.text = seg["target"]

seg_defs = ET.SubElement(unit_el, f"{{{SDL_NS}}}seg-defs")
seg_def = ET.SubElement(seg_defs, f"{{{SDL_NS}}}seg", {
    "id": "1",
    "conf": "Translated" if seg["target"] else "Draft",
})

tree = ET.ElementTree(root)
tree.write(output_path, encoding="utf-8", xml_declaration=True)

```

## cat\_toolformats\mcatdb.py — 125 lines

- Class **McatDbHandler** (line 25)
- Function **\_app\_status\_from\_mcat** (line 9)
- Function **\_mcat\_status\_from\_app** (line 17)
- Function **load** (line 26)
- Function **save** (line 63)
- Function **render** (line 124)

```

import os
import shutil
import sqlite3
from datetime import datetime, timezone
from cat_tool.mcat_format_manager import MCatProjectManager, SCHEMA_SQL
from cat_tool.formats.tag_utils import extract_all_tags

```

```

def _app_status_from_mcat(status_text, has_target):
    if status_text == "Confirmed":
        return {"translated": True, "fuzzy": False}
    if status_text == "Draft":
        return {"translated": bool(has_target), "fuzzy": True}
    return {"translated": False, "fuzzy": False}

```

```

def _mcat_status_from_app(fuzzy, translated):
    if fuzzy:
        return "Draft"
    if translated:
        return "Confirmed"
    return "Untranslated"

```

```

class McatDbHandler:
    def load(self, file_path):
        rows = MCatProjectManager.get_editor_stream(file_path)
        segments = []
        for r in rows:
            source = r["source_text"]
            target = r["target_text"]
            src_tags = extract_all_tags(source)
            tgt_tags = extract_all_tags(target)
            all_tags = list(src_tags)
            for t in tgt_tags:
                if t not in all_tags:
                    all_tags.append(t)
            source_clean = source
            target_clean = target
            if all_tags:
                from cat_tool.formats.tag_utils import apply_tags
                source_clean = apply_tags(source, all_tags)
                target_clean = apply_tags(target, all_tags)

            status = _app_status_from_mcat(r["status"], target)
            segments.append({
                "index": r["id"],
                "source": source,
                "target": target,
                "source_clean": source_clean,
                "target_clean": target_clean,
                "all_tags": all_tags,
            })

```

```

        "notes": "",
        "fuzzy": status["fuzzy"],
        "translated": status["translated"],
        "locations": r["file_id"],
        "source_location": {"unit_id": r["id"]},
        "store": None,
        "unit": None,
    })
    return segments

def save(self, segments, target_path):
    if not segments:
        raise ValueError("No segments to save.")

    if not os.path.exists(target_path):
        tmp_conn = sqlite3.connect(target_path)
        try:
            tmp_conn.executescript(SCHEMA_SQL)
            now = datetime.now(timezone.utc).isoformat()
            tmp_conn.execute(
                "INSERT OR REPLACE INTO project_meta (key, value) VALUES (?, ?)",
                ("created_at", now),
            )
            tmp_conn.commit()
        except BaseException:
            tmp_conn.rollback()
            raise
        finally:
            tmp_conn.close()
        conn = None
    else:
        conn = None

    conn = MCatProjectManager._connect(target_path)
    try:
        conn.execute("DELETE FROM format_tags")
        conn.execute("DELETE FROM translation_units")
        for seg in segments:
            status = _mcat_status_from_app(seg.get("fuzzy", False),
                                           bool(seg.get("target", "")))

            cursor = conn.execute(
                """INSERT INTO translation_units
                (file_id, source_text, target_text, status, match_score)
                VALUES (?, ?, ?, ?, ?)""",
                (
                    seg.get("locations", ""),
                    seg.get("source", ""),
                    seg.get("target", ""),
                    status,
                    seg.get("match_score", 0),
                ),
            )
            unit_id = cursor.lastrowid
            tags = seg.get("all_tags", [])
            if tags:
                tag_rows = [
                    (unit_id, f"{{{i}}}", tag if isinstance(tag, str) else tag.get("raw", str(tag)))
                    for i, tag in enumerate(tags)
                ]
                conn.executemany(
                    """INSERT INTO format_tags (unit_id, tag_index, raw_xml_tag)
                    VALUES (?, ?, ?)""",
                    tag_rows,
                )
            conn.commit()
        except BaseException:
            conn.rollback()
            raise
        finally:
            conn.close()

def render(self, original_path, segments, output_path):
    shutil.copy2(original_path, output_path)

```

## 5. Tests

### tests/test\_cat\_logic.py — 101 lines

- Class **TestCATTool** (line 8)
- Function **setUp** (line 9)
- Function **tearDown** (line 15)
- Function **test\_translation\_memory** (line 18)
- Function **test\_glossary** (line 35)
- Function **test\_file\_handler** (line 58)

```
import unittest
import os
import tempfile
from cat_tool.tm import TranslationMemory
from cat_tool.glossary import Glossary
from cat_tool.file_handler import CATFileHandler

class TestCATTool(unittest.TestCase):
    def setUp(self):
        # Create temp files for databases and PO files
        self.temp_dir = tempfile.TemporaryDirectory()
        self.tm_db = os.path.join(self.temp_dir.name, "test_tm.db")
        self.glossary_db = os.path.join(self.temp_dir.name, "test_glossary.db")

    def tearDown(self):
        self.temp_dir.cleanup()

    def test_translation_memory(self):
        tm = TranslationMemory(self.tm_db)
        tm.add_translation("The quick brown fox jumps over the lazy dog.", "El zorro marrón rápido salta sobre el perro perezoso.")
        tm.add_translation("Hello world", "Hola mundo")

        # Exact match
        matches = tm.get_fuzzy_matches("Hello world")
        self.assertTrue(len(matches) > 0)
        self.assertEqual(matches[0]["target"], "Hola mundo")
        self.assertEqual(matches[0]["score"], 100.0)

        # Fuzzy match
        matches_fuzzy = tm.get_fuzzy_matches("Hello worlds")
        self.assertTrue(len(matches_fuzzy) > 0)
        self.assertGreater(matches_fuzzy[0]["score"], 80)
        self.assertEqual(matches_fuzzy[0]["target"], "Hola mundo")

    def test_glossary(self):
        gl = Glossary(self.glossary_db)
        gl.add_term("cat", "gato", "feline companion")
        gl.add_term("quick brown fox", "zorro marrón rápido", "a fast fox")

        terms = gl.get_all_terms()
        self.assertEqual(len(terms), 2)

        # Token match single word
        matches = gl.check_segment("My cat is sleeping.")
        self.assertEqual(len(matches), 1)
        self.assertEqual(matches[0]["source"], "cat")
        self.assertEqual(matches[0]["target"], "gato")

        # Multi-word match
        matches_multi = gl.check_segment("The quick brown fox is fast.")
        self.assertEqual(len(matches_multi), 1)
        self.assertEqual(matches_multi[0]["source"], "quick brown fox")

        # Non-match due to boundaries
        matches_none = gl.check_segment("Concatenation is a concept.")
        self.assertEqual(len(matches_none), 0)

    def test_file_handler(self):
        # Create a basic mock PO file content
        po_content = """
msgid ""
msgstr ""
```

```

"Project-Id-Version: Test\\n"
"MIME-Version: 1.0\\n"
"Content-Type: text/plain; charset=UTF-8\\n"
"Content-Transfer-Encoding: 8bit\\n"

#. Location of segment 1
#: src/main.c:10
msgid "Hello World!"
msgstr ""

#. Location of segment 2
#: src/main.c:20
msgid "Goodbye World!"
msgstr "Adiós Mundo!"
"""

    po_file_path = os.path.join(self.temp_dir.name, "test.po")
    with open(po_file_path, "w", encoding="utf-8") as f:
        f.write(po_content.strip())

    handler = CATFileHandler()
    segments = handler.load_file(po_file_path)

    self.assertEqual(len(segments), 2)
    self.assertEqual(segments[0]["source"], "Hello World!")
    self.assertEqual(segments[0]["target"], "")
    self.assertEqual(segments[1]["source"], "Goodbye World!")
    self.assertEqual(segments[1]["target"], "Adiós Mundo!")

    # Modify first segment target
    segments[0]["target"] = "Hola Mundo!"
    handler.save_file()

    # Reload and check
    handler2 = CATFileHandler()
    segments2 = handler2.load_file(po_file_path)
    self.assertEqual(segments2[0]["target"], "Hola Mundo!")

if __name__ == "__main__":
    unittest.main()

```

## tests/test\_comprehensive.py — 644 lines

- Class **TestQAModule** (line 46)
- Class **TestTranslationMemory** (line 172)
- Class **TestGlossary** (line 217)
- Class **TestSegmentation** (line 280)
- Class **TestTagUtils** (line 318)
- Class **TestMakeSegmentsFromTexts** (line 403)
- Class **TestFileHandler** (line 427)
- Class **TestMcatProjectManager** (line 503)
- Class **TestTextHandler** (line 556)
- Class **TestMcatDbHandler** (line 581)
- Function **make\_seg** (line 23)
- Function **setUp** (line 47)
- Function **test\_missing\_translations** (line 56)
- Function **test\_missing\_translations\_none\_target** (line 63)
- Function **test\_unedited\_fuzzy** (line 68)
- Function **test\_inconsistent\_translations** (line 73)
- Function **test\_inconsistent\_translations\_no\_dupes** (line 79)
- Function **test\_tag\_mismatch** (line 87)
- Function **test\_number\_mismatch** (line 97)
- Function **test\_whitespace** (line 108)
- Function **test\_terminology** (line 120)
- Function **test\_terminology\_missing\_target** (line 136)
- Function **test\_run\_all\_checks** (line 145)
- Function **test\_run\_all\_checks\_no\_glossary** (line 153)
- Function **test\_extract\_numbers** (line 157)

- Function **test\_empty\_segments** (line 162)
- Function **test\_qa\_issue\_slots** (line 166)
- Function **setUp** (line 173)
- Function **tearDown** (line 178)
- Function **test\_add\_and\_fuzzy\_match** (line 181)
- Function **test\_fuzzy\_match\_partial** (line 188)
- Function **test\_no\_match** (line 194)
- Function **test\_clear** (line 199)
- Function **test\_add\_empty\_source** (line 205)
- Function **test\_add\_duplicate** (line 210)
- Function **setUp** (line 218)
- Function **tearDown** (line 223)
- Function **test\_add\_and\_check** (line 226)
- Function **test\_multi\_word\_term** (line 233)
- Function **test\_word\_boundary\_no\_match** (line 239)
- Function **test\_case\_insensitive\_match** (line 244)
- Function **test\_remove\_term** (line 249)
- Function **test\_get\_all\_terms** (line 255)
- Function **test\_check\_empty\_text** (line 261)
- Function **test\_add\_empty\_term** (line 267)
- Function **test\_update\_existing** (line 273)
- Function **test\_basic\_split** (line 281)
- Function **test\_exception\_no\_split** (line 285)
- Function **test\_multiple\_exceptions** (line 289)
- Function **test\_empty\_text** (line 293)
- Function **test\_no\_splitters** (line 297)
- Function **test\_custom\_rules** (line 301)
- Function **test\_semicolon\_split** (line 309)
- Function **test\_colon\_split** (line 313)
- Function **test\_extract\_all\_tags** (line 319)
- Function **test\_extract\_no\_tags** (line 324)
- Function **test\_extract\_none\_text** (line 328)
- Function **test\_apply\_and\_restore** (line 332)
- Function **test\_restore\_no\_tags** (line 341)
- Function **test\_restore\_none\_text** (line 345)
- Function **test\_apply\_no\_text** (line 349)
- Function **test\_apply\_no\_taglist** (line 353)
- Function **test\_unknown\_tag\_index** (line 357)
- Function **test\_unique\_tags** (line 364)
- Function **test\_extract\_incomplete\_open\_tag** (line 368)
- Function **test\_extract\_incomplete\_close\_tag** (line 372)
- Function **test\_extract\_bare\_less\_than** (line 376)
- Function **test\_extract\_empty\_angle\_brackets** (line 380)
- Function **test\_apply\_with\_incomplete\_tag\_preserved** (line 384)
- Function **test\_apply\_mixed\_complete\_and\_incomplete\_tags** (line 392)
- Function **test\_basic** (line 404)
- Function **test\_empty\_text\_skipped** (line 411)
- Function **test\_with\_locations** (line 415)
- Function **test\_tag\_extraction** (line 421)
- Function **setUp** (line 428)
- Function **tearDown** (line 432)
- Function **\_make\_po** (line 435)
- Function **test\_load\_po** (line 441)
- Function **test\_load\_nonexistent** (line 450)
- Function **test\_load\_unsupported** (line 454)

- Function **test\_save\_po** (line 461)
- Function **test\_save\_no\_segments** (line 474)
- Function **test\_atomic\_save** (line 478)
- Function **test\_convert\_to\_po** (line 487)
- Function **test\_render\_no\_segments** (line 498)
- Function **setUp** (line 504)
- Function **tearDown** (line 508)
- Function **test\_create\_project** (line 511)
- Function **test\_create\_project\_auto\_extension** (line 515)
- Function **test\_import\_and\_get\_stream** (line 520)
- Function **test\_get\_stream\_filtered** (line 531)
- Function **test\_update\_segment** (line 542)
- Function **setUp** (line 557)
- Function **tearDown** (line 560)
- Function **test\_load\_and\_render** (line 563)
- Function **setUp** (line 582)
- Function **tearDown** (line 587)
- Function **test\_load** (line 590)
- Function **test\_load\_with\_tags** (line 603)
- Function **test\_save** (line 615)
- Function **test\_save\_empty\_raises** (line 631)
- Function **test\_render\_copies** (line 636)

```
import unittest
import os
import tempfile
import json
import sqlite3

from cat_tool.qa import (
    QAIssue, run_all_checks,
    check_missing_translations, check_unedited_fuzzy,
    check_inconsistent_translations, check_tag_mismatch,
    check_number_mismatch, check_whitespace, check_terminology,
    _extract_numbers, _count_tags,
)

from cat_tool.tm import TranslationMemory
from cat_tool.glossary import Glossary
from cat_tool.segmentation import advanced_segmenter, DEFAULT_RULES
from cat_tool.formats.tag_utils import PH_L, PH_R, extract_all_tags, apply_tags, restore_tags
from cat_tool.file_handler import CATFileHandler, convert_to_po
from cat_tool.mcat_format_manager import MCatProjectManager, SCHEMA_SQL
from cat_tool.formats._text_base import make_segments_from_texts

def make_seg(index=0, source="Hello world", target="", fuzzy=False, translated=False, tags=None, source_location=None):
    seg = {
        "index": index,
        "source": source,
        "target": target,
        "source_clean": source,
        "target_clean": target,
        "all_tags": tags or [],
        "notes": "",
        "fuzzy": fuzzy,
        "translated": translated,
        "locations": "",
        "source_location": source_location or {},
        "store": None,
        "unit": None,
    }
    if tags:
        from cat_tool.formats.tag_utils import apply_tags
        seg["source_clean"] = apply_tags(source, tags)
        seg["target_clean"] = apply_tags(target, tags)
    return seg

class TestQAModule(unittest.TestCase):
    def setUp(self):
```

```

self.segments = [
    make_seg(0, "Hello world", "Hola mundo", translated=True),
    make_seg(1, "Good morning", "", fuzzy=True),
    make_seg(2, "How are you?", "¿Cómo estás?", translated=True),
    make_seg(3, "Goodbye", ""),
    make_seg(4, "Hello world", "Hola mundo!", translated=True),
]

def test_missing_translations(self):
    issues = check_missing_translations(self.segments)
    self.assertEqual(len(issues), 2)
    indices = {i.segment_index for i in issues}
    self.assertIn(1, indices)
    self.assertIn(3, indices)

def test_missing_translations_none_target(self):
    segs = [make_seg(0, "test", None)]
    issues = check_missing_translations(segs)
    self.assertEqual(len(issues), 1)

def test_unedited_fuzzy(self):
    issues = check_unedited_fuzzy(self.segments)
    self.assertEqual(len(issues), 1)
    self.assertEqual(issues[0].segment_index, 1)

def test_inconsistent_translations(self):
    issues = check_inconsistent_translations(self.segments)
    self.assertEqual(len(issues), 2)
    self.assertEqual(issues[0].segment_index, 0)
    self.assertEqual(issues[1].segment_index, 4)

def test_inconsistent_translations_no_dupes(self):
    segs = [
        make_seg(0, "Hello", "Hola", translated=True),
        make_seg(1, "World", "Mundo", translated=True),
    ]
    issues = check_inconsistent_translations(segs)
    self.assertEqual(len(issues), 0)

def test_tag_mismatch(self):
    segs = [
        make_seg(0, f"Start {PH_L}{PH_R} end", f"Inicio {PH_L}{PH_R} fin", tags=["<b>"]),
        make_seg(1, f"Hello {PH_L}{PH_R} world", "Hola mundo", tags=["<b>"]),
        make_seg(2, "No tags", "Sin etiquetas"),
    ]
    issues = check_tag_mismatch(segs)
    self.assertEqual(len(issues), 1)
    self.assertEqual(issues[0].segment_index, 1)

def test_number_mismatch(self):
    segs = [
        make_seg(0, "Version 2.0 released", "Versión 2.0 lanzada", translated=True),
        make_seg(1, "File 1 of 3", "Archivo 1 de 3", translated=True),
        make_seg(2, "Cost: $50.00", "Costo: $99.00", translated=True),
        make_seg(3, "No numbers here", "Sin números"),
    ]
    issues = check_number_mismatch(segs)
    self.assertEqual(len(issues), 1)
    self.assertEqual(issues[0].segment_index, 2)

def test_whitespace(self):
    segs = [
        make_seg(0, " Leading spaces", "Leading spaces", translated=True),
        make_seg(1, "No leading space", "Has leading space", translated=True),
        make_seg(2, "Trailing space ", "Trailing space", translated=True),
        make_seg(3, "No trailing", "Has trailing ", translated=True),
        make_seg(4, " Both sides ", " Both sides ", translated=True),
        make_seg(5, "Good", "", translated=False),
    ]
    issues = check_whitespace(segs)
    self.assertEqual(len(issues), 4)

def test_terminology(self):
    segs = [
        make_seg(0, "The cat is sleeping.", "El gato duerme.", translated=True),
        make_seg(1, "Quick brown fox.", "Zorro rápido.", translated=True),
        make_seg(2, "No terms here.", "Sin términos.", translated=True),
    ]
    tmp_dir = tempfile.TemporaryDirectory()

```

```

gl = Glossary(os.path.join(tmp_dir.name, "test_gl.db"))
gl.add_term("cat", "gato", "feline")
gl.add_term("quick brown fox", "zorro marrón rápido", "fast fox")

issues = check_terminology(segs, gl)
self.assertEqual(len(issues), 1)
self.assertEqual(issues[0].segment_index, 1)
tmp_dir.cleanup()

def test_terminology_missing_target(self):
    segs = [make_seg(0, "The cat is here.", "", fuzzy=True)]
    tmp_dir = tempfile.TemporaryDirectory()
    gl = Glossary(os.path.join(tmp_dir.name, "test_gl.db"))
    gl.add_term("cat", "gato")
    issues = check_terminology(segs, gl)
    self.assertEqual(len(issues), 0)
    tmp_dir.cleanup()

def test_run_all_checks(self):
    tmp_dir = tempfile.TemporaryDirectory()
    gl = Glossary(os.path.join(tmp_dir.name, "test_gl.db"))
    gl.add_term("cat", "gato")
    issues = run_all_checks(self.segments, glossary=gl)
    self.assertGreater(len(issues), 0)
    tmp_dir.cleanup()

def test_run_all_checks_no_glossary(self):
    issues = run_all_checks(self.segments)
    self.assertGreater(len(issues), 2)

def test_extract_numbers(self):
    self.assertEqual(_extract_numbers("Version 2.0"), {"2.0"})
    self.assertEqual(_extract_numbers("File 1 of 3"), {"1", "3"})
    self.assertEqual(_extract_numbers("No numbers"), set())

def test_empty_segments(self):
    issues = run_all_checks([])
    self.assertEqual(len(issues), 0)

def test_qa_issue_slots(self):
    issue = QAIssue("error", "Test", 0, "src", "tgt", "desc")
    self.assertEqual(issue.severity, "error")
    self.assertEqual(issue.segment_index, 0)

class TestTranslationMemory(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp_dir.name, "tm.db")
        self.tm = TranslationMemory(self.db_path)

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_add_and_fuzzy_match(self):
        self.tm.add_translation("Hello world", "Hola mundo")
        matches = self.tm.get_fuzzy_matches("Hello world")
        self.assertGreater(len(matches), 0)
        self.assertEqual(matches[0]["target"], "Hola mundo")
        self.assertEqual(matches[0]["score"], 100.0)

    def test_fuzzy_match_partial(self):
        self.tm.add_translation("The quick brown fox", "El zorro marrón rápido")
        matches = self.tm.get_fuzzy_matches("The quick brown fox jumps", min_score=50.0)
        self.assertGreater(len(matches), 0)
        self.assertGreater(matches[0]["score"], 50.0)

    def test_no_match(self):
        self.tm.add_translation("Hello", "Hola")
        matches = self.tm.get_fuzzy_matches("Completely different text", min_score=90.0)
        self.assertEqual(len(matches), 0)

    def test_clear(self):
        self.tm.add_translation("Hello", "Hola")
        self.tm.clear()
        matches = self.tm.get_fuzzy_matches("Hello")
        self.assertEqual(len(matches), 0)

    def test_add_empty_source(self):

```

```

        self.tm.add_translation("", "vacio")
        matches = self.tm.get_fuzzy_matches("")
        self.assertEqual(len(matches), 0)

    def test_add_duplicate(self):
        self.tm.add_translation("Hello", "Hola")
        self.tm.add_translation("Hello", ";Hola!")
        matches = self.tm.get_fuzzy_matches("Hello")
        self.assertEqual(matches[0]["target"], ";Hola!")

class TestGlossary(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp_dir.name, "glossary.db")
        self.gl = Glossary(self.db_path)

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_add_and_check(self):
        self.gl.add_term("cat", "gato", "feline companion")
        matches = self.gl.check_segment("My cat is sleeping.")
        self.assertEqual(len(matches), 1)
        self.assertEqual(matches[0]["source"], "cat")
        self.assertEqual(matches[0]["target"], "gato")

    def test_multi_word_term(self):
        self.gl.add_term("quick brown fox", "zorro marrón rápido")
        matches = self.gl.check_segment("The quick brown fox jumps.")
        self.assertEqual(len(matches), 1)
        self.assertEqual(matches[0]["source"], "quick brown fox")

    def test_word_boundary_no_match(self):
        self.gl.add_term("cat", "gato")
        matches = self.gl.check_segment("Concatenation is a concept.")
        self.assertEqual(len(matches), 0)

    def test_case_insensitive_match(self):
        self.gl.add_term("Cat", "Gato")
        matches = self.gl.check_segment("My CAT is sleeping.")
        self.assertEqual(len(matches), 1)

    def test_remove_term(self):
        self.gl.add_term("dog", "perro")
        self.gl.remove_term("dog")
        matches = self.gl.check_segment("My dog runs.")
        self.assertEqual(len(matches), 0)

    def test_get_all_terms(self):
        self.gl.add_term("a", "1")
        self.gl.add_term("b", "2")
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 2)

    def test_check_empty_text(self):
        matches = self.gl.check_segment("")
        self.assertEqual(len(matches), 0)
        matches = self.gl.check_segment(None)
        self.assertEqual(len(matches), 0)

    def test_add_empty_term(self):
        self.gl.add_term("", "test")
        self.gl.add_term("test", "")
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 0)

    def test_update_existing(self):
        self.gl.add_term("cat", "gato")
        self.gl.add_term("cat", "michi")
        matches = self.gl.check_segment("My cat is here.")
        self.assertEqual(matches[0]["target"], "michi")

class TestSegmentation(unittest.TestCase):
    def test_basic_split(self):
        result = advanced_segmenter("Hello. How are you? I'm fine.")
        self.assertGreaterEqual(len(result), 2)
    def test_exception_no_split(self):

```

```

        result = advanced_segmenter("Dr. Smith went to the store.")
        self.assertEqual(len(result), 1)

    def test_multiple_exceptions(self):
        result = advanced_segmenter("Mr. and Mrs. Jones arrived. Dr. Brown came too.")
        self.assertEqual(len(result), 2)

    def test_empty_text(self):
        result = advanced_segmenter("")
        self.assertEqual(result, [])

    def test_no Splitters(self):
        result = advanced_segmenter("Hello world no punctuation here")
        self.assertEqual(len(result), 1)

    def test_custom_rules(self):
        rules = [
            {"pattern": ".", "break_after": True, "case_sensitive": False, "whole_word": False},
            {"pattern": "Inc", "break_after": False, "case_sensitive": True, "whole_word": True},
        ]
        result = advanced_segmenter("Hello Inc. World. Next sentence.", rules)
        self.assertEqual(len(result), 2)

    def test_semicolon_split(self):
        result = advanced_segmenter("First part; second part.")
        self.assertEqual(len(result), 2)

    def test_colon_split(self):
        result = advanced_segmenter("Note: important text here.")
        self.assertEqual(len(result), 2)

class TestTagUtils(unittest.TestCase):
    def test_extract_all_tags(self):
        tags = extract_all_tags("Hello <b>world</b>!")
        self.assertIn("<b>", tags)
        self.assertIn("</b>", tags)

    def test_extract_no_tags(self):
        tags = extract_all_tags("Hello world")
        self.assertEqual(tags, [])

    def test_extract_none_text(self):
        tags = extract_all_tags(None)
        self.assertEqual(tags, [])

    def test_apply_and_restore(self):
        tags = ["<b>", "</b>"]
        text = "<b>Hello</b>"
        applied = apply_tags(text, tags)
        self.assertIn(PH_L, applied)
        self.assertIn(PH_R, applied)
        restored = restore_tags(applied, tags)
        self.assertEqual(restored, text)

    def test_restore_no_tags(self):
        result = restore_tags("Hello world", [])
        self.assertEqual(result, "Hello world")

    def test_restore_none_text(self):
        result = restore_tags(None, ["<b>"])
        self.assertIsNone(result)

    def test_apply_no_text(self):
        result = apply_tags("", ["<b>"])
        self.assertEqual(result, "")

    def test_apply_no_taglist(self):
        result = apply_tags("Hello", None)
        self.assertEqual(result, "Hello")

    def test_unknown_tag_index(self):
        tags = ["<b>"]
        applied = apply_tags("<i>text</i>", tags)
        restored = restore_tags(applied, tags)
        self.assertIn("<i>", restored)
        self.assertIn("</i>", restored)

    def test_unique_tags(self):

```

```

tags = extract_all_tags("<b><b><i></b></i>")
self.assertEqual(len(tags), 4)

def test_extract_incomplete_open_tag(self):
    tags = extract_all_tags("<b no closing angle")
    self.assertEqual(tags, [])

def test_extract_incomplete_close_tag(self):
    tags = extract_all_tags("> without opening")
    self.assertEqual(tags, [])

def test_extract_bare_less_than(self):
    tags = extract_all_tags("Hello < world")
    self.assertEqual(tags, [])

def test_extract_empty_angle_brackets(self):
    tags = extract_all_tags("<>")
    self.assertEqual(tags, [])

def test_apply_with_incomplete_tag_preserved(self):
    text = "<b>hello</b>"
    tags = ["<b>"]
    result = apply_tags(text, tags)
    self.assertIn("</b>", result)
    restored = restore_tags(result, tags)
    self.assertEqual(restored, text)

def test_apply_mixed_complete_and_incomplete_tags(self):
    text = "<b>Hello</b> <i>world"
    tags = extract_all_tags(text)
    self.assertIn("<b>", tags)
    self.assertNotIn("</b>", tags)
    self.assertNotIn("<i>", tags)
    applied = apply_tags(text, tags)
    restored = restore_tags(applied, tags)
    self.assertEqual(restored, text)

class TestMakeSegmentsFromTexts(unittest.TestCase):
    def test_basic(self):
        segs = make_segments_from_texts(["Hello", "World"], notes_prefix="Line ")
        self.assertEqual(len(segs), 2)
        self.assertEqual(segs[0]["source"], "Hello")
        self.assertEqual(segs[0]["notes"], "Line 1")
        self.assertEqual(segs[1]["source"], "World")

    def test_empty_text_skipped(self):
        segs = make_segments_from_texts(["Hello", "", "World"])
        self.assertEqual(len(segs), 2)

    def test_with_locations(self):
        locs = [{"line": 0}, {"line": 2}]
        segs = make_segments_from_texts(["Hello", "World"], source_locations=locs)
        self.assertEqual(segs[0]["source_location"], {"line": 0})
        self.assertEqual(segs[1]["source_location"], {"line": 2})

    def test_tag_extraction(self):
        segs = make_segments_from_texts("<b>Hello</b>")
        self.assertIn("<b>", segs[0]["all_tags"])
        self.assertIn("</b>", segs[0]["all_tags"])

class TestFileHandler(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.handler = CATFileHandler()

    def tearDown(self):
        self.tmp_dir.cleanup()

    def _make_po(self, content):
        path = os.path.join(self.tmp_dir.name, "test.po")
        with open(path, "w", encoding="utf-8") as f:
            f.write(content.strip())
        return path

    def test_load_po(self):
        path = self._make_po("""msgid "Hello"
msgstr "Hola"

```

```

msgid "World"
msgstr "Mundo"
"""
    segments = self.handler.load_file(path)
    self.assertGreater(len(segments), 0)

def test_load_nonexistent(self):
    with self.assertRaises(FileNotFoundError):
        self.handler.load_file("nonexistent.po")

def test_load_unsupported(self):
    path = os.path.join(self.tmp_dir.name, "test.xyz")
    with open(path, "w") as f:
        f.write("data")
    with self.assertRaises(ValueError):
        self.handler.load_file(path)

def test_save_po(self):
    path = self._make_po("""msgid "Hello"
msgstr ""
""")
    segments = self.handler.load_file(path)
    segments[0]["target"] = "Hola"
    save_path = os.path.join(self.tmp_dir.name, "output.po")
    self.handler.save_file(save_path)
    self.assertTrue(os.path.exists(save_path))
    with open(save_path, encoding="utf-8") as f:
        content = f.read()
    self.assertIn("Hola", content)

def test_save_no_segments(self):
    with self.assertRaises(ValueError):
        self.handler.save_file("out.po")

def test_atomic_save(self):
    path = self._make_po("""msgid "Hello"
msgstr ""
""")
    self.handler.load_file(path)
    save_path = os.path.join(self.tmp_dir.name, "atomic.po")
    self.handler.save_file(save_path)
    self.assertTrue(os.path.exists(save_path))

def test_convert_to_po(self):
    txt_path = os.path.join(self.tmp_dir.name, "test.txt")
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write("Hello\nWorld")
    po_out = os.path.join(self.tmp_dir.name, "out.po")
    convert_to_po(txt_path, po_out)
    self.assertTrue(os.path.exists(po_out))
    with open(po_out, encoding="utf-8") as f:
        content = f.read()
    self.assertIn("Hello", content)

def test_render_no_segments(self):
    with self.assertRaises(ValueError):
        self.handler.render_translated("out.txt")

class TestMcatProjectManager(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp_dir.name, "test.mcat.db")

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_create_project(self):
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        self.assertTrue(os.path.exists(self.db_path))

    def test_create_project_auto_extension(self):
        path = os.path.join(self.tmp_dir.name, "test")
        MCatProjectManager.create_project_file(path, "en", "es")
        self.assertTrue(os.path.exists(path + ".mcat.db"))

    def test_import_and_get_stream(self):
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        segments = [

```

```

        {"file_id": "test.txt", "source": "Hello", "target": "Hola", "status": "Confirmed", "match_score": 100},
    ]
    MCatProjectManager.import_extracted_segments(self.db_path, segments)
    rows = MCatProjectManager.get_editor_stream(self.db_path)
    self.assertEqual(len(rows), 1)
    self.assertEqual(rows[0]["source_text"], "Hello")
    self.assertEqual(rows[0]["target_text"], "Hola")

def test_get_stream_filtered(self):
    MCatProjectManager.create_project_file(self.db_path, "en", "es")
    segments = [
        {"file_id": "a.txt", "source": "Hello", "target": "", "status": "Untranslated", "match_score": 0},
        {"file_id": "a.txt", "source": "World", "target": "Mundo", "status": "Confirmed", "match_score": 100},
    ]
    MCatProjectManager.import_extracted_segments(self.db_path, segments)
    rows = MCatProjectManager.get_editor_stream(self.db_path, filter_status="Confirmed")
    self.assertEqual(len(rows), 1)
    self.assertEqual(rows[0]["source_text"], "World")

def test_update_segment(self):
    MCatProjectManager.create_project_file(self.db_path, "en", "es")
    segments = [
        {"file_id": "a.txt", "source": "Hello", "target": "", "status": "Untranslated", "match_score": 0},
    ]
    MCatProjectManager.import_extracted_segments(self.db_path, segments)
    rows = MCatProjectManager.get_editor_stream(self.db_path)
    unit_id = rows[0]["id"]
    MCatProjectManager.update_segment_translation(self.db_path, unit_id, "Hola", "Confirmed")
    rows = MCatProjectManager.get_editor_stream(self.db_path)
    self.assertEqual(rows[0]["target_text"], "Hola")
    self.assertEqual(rows[0]["status"], "Confirmed")

class TestTextHandler(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_load_and_render(self):
        from cat_tool.formats.text import TextHandler
        txt_path = os.path.join(self.tmp_dir.name, "test.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Hello\nWorld\nFoo")
        handler = TextHandler()
        segments = handler.load(txt_path)
        self.assertEqual(len(segments), 3)
        segments[0]["target"] = "Hola"
        segments[1]["target"] = "Mundo"
        out_path = os.path.join(self.tmp_dir.name, "out.txt")
        handler.render(txt_path, segments, out_path)
        with open(out_path, encoding="utf-8") as f:
            content = f.read()
        self.assertIn("Hola", content)
        self.assertIn("Mundo", content)

class TestMcatDbHandler(unittest.TestCase):
    def setUp(self):
        self.tmp_dir = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp_dir.name, "test.mcat.db")
        MCatProjectManager.create_project_file(self.db_path, "en", "es")

    def tearDown(self):
        self.tmp_dir.cleanup()

    def test_load(self):
        from cat_tool.formats.mcatdb import McatDbHandler
        MCatProjectManager.import_extracted_segments(self.db_path, [
            {"file_id": "f.txt", "source": "Hello", "target": "Hola", "status": "Confirmed", "match_score": 100},
        ])
        handler = McatDbHandler()
        segments = handler.load(self.db_path)
        self.assertEqual(len(segments), 1)
        self.assertEqual(segments[0]["source"], "Hello")
        self.assertEqual(segments[0]["target"], "Hola")
        self.assertFalse(segments[0]["fuzzy"])
        self.assertTrue(segments[0]["translated"])

```

```

def test_load_with_tags(self):
    from cat_tool.formats.mcatdb import McatDbHandler
    conn = sqlite3.connect(self.db_path)
    conn.executescript(SCHEMA_SQL)
    conn.execute("INSERT INTO translation_units (file_id, source_text, target_text) VALUES (?, ?, ?)",
                  ("f.txt", "<b>Hello</b>", "<b>Hola</b>"))
    conn.commit()
    conn.close()
    segments = McatDbHandler().load(self.db_path)
    self.assertEqual(len(segments), 1)
    self.assertIn("<b>", segments[0]["all_tags"])

def test_save(self):
    from cat_tool.formats.mcatdb import McatDbHandler
    segs = [{
        "index": 0, "source": "Hello", "target": "Hola",
        "all_tags": [], "source_clean": "Hello", "target_clean": "Hola",
        "fuzzy": False, "translated": True, "locations": "f.txt",
        "notes": "", "source_location": {}, "store": None, "unit": None,
        "match_score": 100,
    }]
    out_path = os.path.join(self.tmp_dir.name, "out.mcat.db")
    McatDbHandler().save(segs, out_path)
    self.assertTrue(os.path.exists(out_path))
    loaded = McatDbHandler().load(out_path)
    self.assertEqual(len(loaded), 1)
    self.assertEqual(loaded[0]["target"], "Hola")

def test_save_empty_raises(self):
    from cat_tool.formats.mcatdb import McatDbHandler
    with self.assertRaises(ValueError):
        McatDbHandler().save([], "out.mcat.db")

def test_render_copies(self):
    from cat_tool.formats.mcatdb import McatDbHandler
    out_path = os.path.join(self.tmp_dir.name, "copy.mcat.db")
    McatDbHandler().render(self.db_path, [], out_path)
    self.assertTrue(os.path.exists(out_path))

if __name__ == "__main__":
    unittest.main()

```

## teststest\_remaining.py — 890 lines

- Class **TestProject** (line 18)
- Class **TestMqxliiffHandler** (line 155)
- Class **TestSdliiffHandler** (line 334)
- Class **TestGuiSmoke** (line 547)
- Class **TestFileHandlerIntegration** (line 673)
- Class **TestRealDocumentLoad** (line 780)
- Function **setUp** (line 19)
- Function **tearDown** (line 22)
- Function **test\_new\_project** (line 25)
- Function **test\_save\_and\_load** (line 35)
- Function **test\_save\_no\_path\_raises** (line 46)
- Function **test\_to\_dict\_roundtrip** (line 52)
- Function **test\_default\_dir** (line 62)
- Function **test\_recent\_projects** (line 66)
- Function **test\_str** (line 73)
- Function **test\_omegat\_save\_and\_load** (line 80)
- Function **test\_omegat\_load\_raises\_on\_missing** (line 96)
- Function **test\_memoq\_save\_and\_load** (line 100)
- Function **test\_memoq\_load\_raises\_on\_missing** (line 123)
- Function **test\_load\_omegat\_from\_minimal\_xml** (line 127)
- Function **test\_load\_omegat\_missing\_project\_element** (line 142)
- Function **setUp** (line 156)
- Function **tearDown** (line 159)

- Function **\_make\_mqxliiff** (line 162)
- Function **test\_load\_simple** (line 168)
- Function **test\_load\_empty\_target** (line 187)
- Function **test\_load\_missing\_body\_raises** (line 205)
- Function **test\_save\_and\_reload** (line 215)
- Function **test\_save\_no\_target** (line 231)
- Function **test\_load\_missing\_source\_element** (line 245)
- Function **test\_load\_missing\_target\_element** (line 260)
- Function **test\_load\_source\_with\_escaped\_tags** (line 277)
- Function **test\_load\_target\_with\_escaped\_incomplete\_tag** (line 295)
- Function **test\_load\_mixed\_escaped\_and\_plain\_tags** (line 313)
- Function **setUp** (line 335)
- Function **tearDown** (line 338)
- Function **\_make\_sdlxliiff** (line 341)
- Function **test\_load\_simple** (line 347)
- Function **test\_load\_skips\_translate\_no** (line 370)
- Function **test\_load\_skip\_no\_source** (line 394)
- Function **test\_save\_and\_reload** (line 411)
- Function **test\_save\_without\_target** (line 427)
- Function **test\_target\_without\_mrk** (line 441)
- Function **test\_load\_source\_with\_escaped\_tags\_in\_mrk** (line 462)
- Function **test\_load\_target\_with\_escaped\_incomplete\_tag\_in\_mrk** (line 484)
- Function **test\_load\_missing\_seg\_source\_and\_source** (line 505)
- Function **test\_load\_plain\_target\_with\_escaped\_tag** (line 522)
- Function **setUp** (line 548)
- Function **tearDown** (line 551)
- Function **test\_create\_mainwindow** (line 554)
- Function **test\_load\_po\_file\_through\_handler** (line 570)
- Function **test\_load\_po\_and\_translate** (line 596)
- Function **test\_qa\_dialog\_creation** (line 621)
- Function **test\_shortcuts\_dialog\_creation** (line 640)
- Function **test\_glossary\_dialog\_creation** (line 655)
- Function **setUp** (line 674)
- Function **tearDown** (line 677)
- Function **\_make\_po** (line 680)
- Function **test\_po\_load\_save\_roundtrip** (line 686)
- Function **test\_txt\_load\_save\_roundtrip** (line 704)
- Function **test\_po\_convert\_to\_po\_from\_txt** (line 720)
- Function **test\_factory\_xliiff\_load** (line 732)
- Function **test\_render\_no\_current\_path\_raises** (line 752)
- Function **test\_load\_mcatdb\_and\_save\_mcatdb\_roundtrip** (line 758)
- Function **setUp** (line 781)
- Function **tearDown** (line 784)
- Function **test\_docx\_load** (line 787)
- Function **test\_docx\_render** (line 801)
- Function **test\_pptx\_load** (line 818)
- Function **test\_pptx\_render** (line 833)
- Function **test\_xlsx\_load** (line 856)
- Function **test\_xlsx\_render** (line 870)

```
import unittest
import os
import tempfile
import json
import shutil
import sys
from cat_tool.project import Project
```

```

from cat_tool.file_handler import CATFileHandler
from cat_tool.formats.mqxliiff import MqxliiffHandler
from cat_tool.formats.sdxliiff import SdxliiffHandler
from cat_tool.formats.text import TextHandler
from cat_tool.formats._text_base import make_segments_from_texts

# ===== PROJECT TESTS =====

class TestProject(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_new_project(self):
        p = Project()
        p.new("Test Proj", "en", "es", ["a.txt", "b.txt"])
        self.assertEqual(p.name, "Test Proj")
        self.assertEqual(p.source_lang, "en")
        self.assertEqual(p.target_lang, "es")
        self.assertEqual(len(p.source_files), 2)
        self.assertTrue(p.created_at)
        self.assertTrue(p.modified_at)

    def test_save_and_load(self):
        p = Project()
        p.new("SaveTest", "fr", "de", [])
        proj_path = os.path.join(self.tmp.name, "test.mcatproj")
        p.save(proj_path)
        self.assertTrue(os.path.exists(proj_path))
        loaded = Project.load(proj_path)
        self.assertEqual(loaded.name, "SaveTest")
        self.assertEqual(loaded.source_lang, "fr")
        self.assertEqual(loaded.target_lang, "de")

    def test_save_no_path_raises(self):
        p = Project()
        p.new("NoPath", "en", "es", [])
        with self.assertRaises(ValueError):
            p.save()

    def test_to_dict_roundtrip(self):
        p = Project()
        p.new("DictTest", "en", "es", ["f1.txt"])
        d = p.to_dict()
        self.assertEqual(d["version"], 1)
        self.assertEqual(d["name"], "DictTest")
        p2 = Project.from_dict(d)
        self.assertEqual(p2.name, "DictTest")
        self.assertEqual(p2.source_lang, "en")

    def test_default_dir(self):
        d = Project.default_dir()
        self.assertTrue(os.path.isabs(d))

    def test_recent_projects(self):
        Project.add_recent("/fake/path", "FakeProj")
        recents = Project.get_recent()
        found = any(r.get("path") == "/fake/path" for r in recents)
        self.assertTrue(found)
        self.assertEqual(len(recents), 10)

    def test_str(self):
        p = Project()
        p.new("StrTest", "en", "es", [])
        s = str(p)
        self.assertIn("StrTest", s)
        self.assertIn("en", s)

    def test_omegat_save_and_load(self):
        src_file = os.path.join(self.tmp.name, "doc.txt")
        with open(src_file, "w", encoding="utf-8") as f:
            f.write("Hello")
        p = Project()
        p.new("OmegaTTest", "en", "es", [src_file])
        omegat_dir = os.path.join(self.tmp.name, "OmegaTTest")
        p.save_as_omegat(omegat_dir)

```

```

        self.assertTrue(os.path.exists(os.path.join(omegat_dir, "omegat.project")))
        self.assertTrue(os.path.exists(os.path.join(omegat_dir, "source", "doc.txt")))
        loaded = Project.load_omegat(omegat_dir)
        self.assertEqual(loaded.name, "OmegaTTest")
        self.assertEqual(loaded.source_lang, "en")
        self.assertEqual(loaded.target_lang, "es")
        self.assertIn("doc.txt", loaded.source_files[0])

def test_omegat_load_raises_on_missing(self):
    with self.assertRaises(FileNotFoundError):
        Project.load_omegat("nonexistent_dir")

def test_memoq_save_and_load(self):
    src_file = os.path.join(self.tmp.name, "doc.txt")
    with open(src_file, "w", encoding="utf-8") as f:
        f.write("Hello")
    p = Project()
    p.new("MemoQTest", "en", "es", [src_file])

    handler = CATFileHandler()
    handler.segments = make_segments_from_texts(["Hello"], notes_prefix="")
    handler.segments[0]["target"] = "Hola"

    memoq_dir = os.path.join(self.tmp.name, "memoq_out")
    p.save_as_memoq(memoq_dir, handler)
    self.assertTrue(os.path.exists(os.path.join(memoq_dir, "manifest.xml")))
    self.assertTrue(os.path.exists(os.path.join(memoq_dir, "source", "doc.txt")))
    self.assertTrue(os.path.exists(os.path.join(memoq_dir, "target", "MemoQTest.mqxliiff")))

    loaded = Project.load_memoq(memoq_dir)
    self.assertEqual(loaded.name, "MemoQTest")
    self.assertEqual(loaded.source_lang, "en")
    self.assertEqual(loaded.target_lang, "es")
    self.assertGreater(len(loaded.mqxliiff_files), 0)

def test_memoq_load_raises_on_missing(self):
    with self.assertRaises(FileNotFoundError):
        Project.load_memoq("nonexistent_dir")

def test_load_omegat_from_minimal_xml(self):
    omegat_dir = os.path.join(self.tmp.name, "minimal_omegat")
    os.makedirs(omegat_dir)
    os.makedirs(os.path.join(omegat_dir, "source"))
    et = ET = __import__("xml.etree.ElementTree", fromlist=["ElementTree"])
    root = et.Element("omegat")
    pe = et.SubElement(root, "project")
    et.SubElement(pe, "source_lang").text = "EN-US"
    et.SubElement(pe, "target_lang").text = "FR"
    tree = et.ElementTree(root)
    tree.write(os.path.join(omegat_dir, "omegat.project"), encoding="utf-8", xml_declaration=True)
    loaded = Project.load_omegat(omegat_dir)
    self.assertEqual(loaded.source_lang, "en-us")
    self.assertEqual(loaded.target_lang, "fr")

def test_load_omegat_missing_project_element(self):
    omegat_dir = os.path.join(self.tmp.name, "bad_omegat")
    os.makedirs(omegat_dir)
    et = ET = __import__("xml.etree.ElementTree", fromlist=["ElementTree"])
    root = et.Element("omegat")
    tree = et.ElementTree(root)
    tree.write(os.path.join(omegat_dir, "omegat.project"), encoding="utf-8", xml_declaration=True)
    with self.assertRaises(ValueError):
        Project.load_omegat(omegat_dir)

# ===== MQXLIFF TESTS =====

class TestMqxliiffHandler(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def _make_mqxliiff(self, content):
        path = os.path.join(self.tmp.name, "test.mqxliiff")
        with open(path, "w", encoding="utf-8") as f:
            f.write(content)
        return path

```

```

def test_load_simple(self):
    xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <target>Hola</target>
      </trans-unit>
    </body>
  </file>
</xliff>"""
    path = self._make_mqxliiff(xml)
    segs = MqxliiffHandler().load(path)
    self.assertEqual(len(segs), 1)
    self.assertEqual(segs[0]["source"], "Hello")
    self.assertEqual(segs[0]["target"], "Hola")
    self.assertTrue(segs[0]["translated"])

def test_load_empty_target(self):
    xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <target></target>
      </trans-unit>
    </body>
  </file>
</xliff>"""
    path = self._make_mqxliiff(xml)
    segs = MqxliiffHandler().load(path)
    self.assertEqual(len(segs), 1)
    self.assertEqual(segs[0]["target"], "")
    self.assertFalse(segs[0]["translated"])

def test_load_missing_body_raises(self):
    xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    </file>
  </xliff>"""
    path = self._make_mqxliiff(xml)
    with self.assertRaises(ValueError):
        MqxliiffHandler().load(path)

def test_save_and_reload(self):
    segs = [{
        "index": 0, "source": "Hello", "target": "Hola",
        "source_clean": "Hello", "target_clean": "Hola",
        "all_tags": [], "notes": "",
        "fuzzy": False, "translated": True,
        "locations": "", "store": None, "unit": None,
    }]
    out = os.path.join(self.tmp.name, "out.mqxliiff")
    MqxliiffHandler().save(segs, out, src_lang="en", tgt_lang="es")
    self.assertTrue(os.path.exists(out))
    reloaded = MqxliiffHandler().load(out)
    self.assertEqual(len(reloaded), 1)
    self.assertEqual(reloaded[0]["source"], "Hello")
    self.assertEqual(reloaded[0]["target"], "Hola")

def test_save_no_target(self):
    segs = [{
        "index": 0, "source": "Hello", "target": "",
        "source_clean": "Hello", "target_clean": "",
        "all_tags": [], "notes": "",
        "fuzzy": False, "translated": False,
        "locations": "", "store": None, "unit": None,
    }]
    out = os.path.join(self.tmp.name, "empty.mqxliiff")
    MqxliiffHandler().save(segs, out)
    self.assertTrue(os.path.exists(out))
    reloaded = MqxliiffHandler().load(out)
    self.assertEqual(reloaded[0]["target"], "")

def test_load_missing_source_element(self):
    xml = """<?xml version="1.0"?>

```

```

<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <target>Hola</target>
      </trans-unit>
    </body>
  </file>
</xliff>
"""
    path = self._make_mqxiff(xml)
    segs = MqxiffHandler().load(path)
    self.assertEqual(len(segs), 0)

    def test_load_missing_target_element(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <source>Hello</source>
      </trans-unit>
    </body>
  </file>
</xliff>
"""
        path = self._make_mqxiff(xml)
        segs = MqxiffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["target"], "")
        self.assertFalse(segs[0]["translated"])

    def test_load_source_with_escaped_tags(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <source>&lt;b&gt;Hello</source>
        <target>&lt;b&gt;Hola</target>
      </trans-unit>
    </body>
  </file>
</xliff>
"""
        path = self._make_mqxiff(xml)
        segs = MqxiffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["source"], "<b>Hello")
        self.assertIn("<b>", segs[0]["all_tags"])

    def test_load_target_with_escaped_incomplete_tag(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <target>Hola&lt;b</target>
      </trans-unit>
    </body>
  </file>
</xliff>
"""
        path = self._make_mqxiff(xml)
        segs = MqxiffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["target"], "Hola<b")
        self.assertTrue(segs[0]["translated"])

    def test_load_mixed_escaped_and_plain_tags(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file original="test" source-language="en" target-language="es">
    <body>
      <trans-unit id="1">
        <source>&lt;b&gt;Hello world</source>
        <target>&lt;b&gt;Hola mundo</target>
      </trans-unit>
    </body>
  </file>
</xliff>
"""
        path = self._make_mqxiff(xml)

```

```

        segs = MqxliffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertIn("<b>", segs[0]["all_tags"])
        self.assertEqual(len(segs[0]["all_tags"]), 1)

# ===== SDLXLIFF TESTS =====

class TestSdlxliFFHandler(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def _make_sdlxliFF(self, content):
        path = os.path.join(self.tmp.name, "test.sdlxliFF")
        with open(path, "w", encoding="utf-8") as f:
            f.write(content)
        return path

    def test_load_simple(self):
        xml = "<?xml version='1.0'?>
<xliFF xmlns='urn:oasis:names:tc:xliFF:document:1.2'
  xmlns:sdl='http://sdl.com/FileTypes/SdlXliFF/1.0' version='1.2'>
  <file original='test' datatype='x-sdlfilterframework2' source-language='en' target-language='es'>
    <header>
    <body>
      <trans-unit id='1'>
        <source>Hello</source>
        <seg-source><mrk mtype='seg' mid='1'>Hello</mrk></seg-source>
        <target><mrk mtype='seg' mid='1'>Hola</mrk></target>
        <sdl:seg-defs><sdl:seg id='1' conf='Translated'></sdl:seg-defs>
      </trans-unit>
    </body>
  </file>
</xliFF>""
        path = self._make_sdlxliFF(xml)
        segs = SdlxliFFHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["source"], "Hello")
        self.assertEqual(segs[0]["target"], "Hola")
        self.assertTrue(segs[0]["translated"])

    def test_load_skips_translate_no(self):
        xml = "<?xml version='1.0'?>
<xliFF xmlns='urn:oasis:names:tc:xliFF:document:1.2'
  xmlns:sdl='http://sdl.com/FileTypes/SdlXliFF/1.0' version='1.2'>
  <file original='test' datatype='x-sdlfilterframework2' source-language='en' target-language='es'>
    <header>
    <body>
      <trans-unit id='1' translate='no'>
        <source>Skip me</source>
      </trans-unit>
      <trans-unit id='2'>
        <source>Hello</source>
        <seg-source><mrk mtype='seg' mid='1'>Hello</mrk></seg-source>
        <target><mrk mtype='seg' mid='1'>Hola</mrk></target>
        <sdl:seg-defs><sdl:seg id='1' conf='Translated'></sdl:seg-defs>
      </trans-unit>
    </body>
  </file>
</xliFF>""
        path = self._make_sdlxliFF(xml)
        segs = SdlxliFFHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["source"], "Hello")

    def test_load_skip_no_source(self):
        xml = "<?xml version='1.0'?>
<xliFF xmlns='urn:oasis:names:tc:xliFF:document:1.2'
  xmlns:sdl='http://sdl.com/FileTypes/SdlXliFF/1.0' version='1.2'>
  <file original='test' datatype='x-sdlfilterframework2' source-language='en' target-language='es'>
    <header>
    <body>
      <trans-unit id='1'>
        <seg-source><mrk mtype='seg' mid='1'></mrk></seg-source>
      </trans-unit>
    </body>
  </file>
</xliFF>""

```

```

    </file>
    </xliff>"""
    path = self._make_sdlxliff(xml)
    segs = SdlxliffHandler().load(path)
    self.assertEqual(len(segs), 0)

def test_save_and_reload(self):
    segs = [{
        "index": 0, "source": "Hello", "target": "Hola",
        "source_clean": "Hello", "target_clean": "Hola",
        "all_tags": [], "notes": "",
        "fuzzy": False, "translated": True,
        "locations": "", "store": None, "unit": None,
    }]
    out = os.path.join(self.tmp.name, "out.sdlxliff")
    SdlxliffHandler().save(segs, out, src_lang="en", tgt_lang="es")
    self.assertTrue(os.path.exists(out))
    reloaded = SdlxliffHandler().load(out)
    self.assertEqual(len(reloaded), 1)
    self.assertEqual(reloaded[0]["source"], "Hello")
    self.assertEqual(reloaded[0]["target"], "Hola")

def test_save_without_target(self):
    segs = [{
        "index": 0, "source": "Hello", "target": "",
        "source_clean": "Hello", "target_clean": "",
        "all_tags": [], "notes": "",
        "fuzzy": False, "translated": False,
        "locations": "", "store": None, "unit": None,
    }]
    out = os.path.join(self.tmp.name, "empty.sdlxliff")
    SdlxliffHandler().save(segs, out)
    self.assertTrue(os.path.exists(out))
    reloaded = SdlxliffHandler().load(out)
    self.assertEqual(reloaded[0]["target"], "")

def test_target_without_mrk(self):
    xml = """<?xml version="1.0"?>
    <xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
        xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
    <file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
    <header>
    <body>
    <trans-unit id="1">
    <source>Hello</source>
    <seg-source><mrk mtype="seg" mid="1">Hello</mrk></seg-source>
    <target>Hola</target>
    <sdl:seg-defs><sdl:seg id="1" conf="Translated"/></sdl:seg-defs>
    </trans-unit>
    </body>
    </file>
    </xliff>"""
    path = self._make_sdlxliff(xml)
    segs = SdlxliffHandler().load(path)
    self.assertEqual(len(segs), 1)
    self.assertEqual(segs[0]["target"], "Hola")

def test_load_source_with_escaped_tags_in_mrk(self):
    xml = """<?xml version="1.0"?>
    <xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
        xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
    <file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
    <header>
    <body>
    <trans-unit id="1">
    <source>&lt;b&gt;Hello</source>
    <seg-source><mrk mtype="seg" mid="1">&lt;b&gt;Hello</mrk></seg-source>
    <target><mrk mtype="seg" mid="1">&lt;b&gt;Hola</mrk></target>
    <sdl:seg-defs><sdl:seg id="1" conf="Translated"/></sdl:seg-defs>
    </trans-unit>
    </body>
    </file>
    </xliff>"""
    path = self._make_sdlxliff(xml)
    segs = SdlxliffHandler().load(path)
    self.assertEqual(len(segs), 1)
    self.assertEqual(segs[0]["source"], "<b>Hello")
    self.assertIn("<b>", segs[0]["all_tags"])

def test_load_target_with_escaped_incomplete_tag_in_mrk(self):

```

```

        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
  xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
  <file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
    <header/>
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <seg-source><mrk mtype="seg" mid="1">Hello</mrk></seg-source>
        <target><mrk mtype="seg" mid="1">Hola</b></mrk></target>
        <sdl:seg-defs><sdl:seg id="1" conf="Translated"/></sdl:seg-defs>
      </trans-unit>
    </body>
  </file>
</xliff>"""
        path = self._make_sdlxliff(xml)
        segs = SdlxliffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["target"], "Hola<b>")

```

```

    def test_load_missing_seg_source_and_source(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
  xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
  <file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
    <header/>
    <body>
      <trans-unit id="1">
        <target><mrk mtype="seg" mid="1">Hola</mrk></target>
      </trans-unit>
    </body>
  </file>
</xliff>"""
        path = self._make_sdlxliff(xml)
        segs = SdlxliffHandler().load(path)
        self.assertEqual(len(segs), 0)

```

```

    def test_load_plain_target_with_escaped_tag(self):
        xml = """<?xml version="1.0"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2"
  xmlns:sdl="http://sdl.com/FileTypes/SdlXliff/1.0" version="1.2">
  <file original="test" datatype="x-sdlfilterframework2" source-language="en" target-language="es">
    <header/>
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <seg-source><mrk mtype="seg" mid="1">Hello</mrk></seg-source>
        <target>Hola&lt;i></i></target>
        <sdl:seg-defs><sdl:seg id="1" conf="Translated"/></sdl:seg-defs>
      </trans-unit>
    </body>
  </file>
</xliff>"""
        path = self._make_sdlxliff(xml)
        segs = SdlxliffHandler().load(path)
        self.assertEqual(len(segs), 1)
        self.assertEqual(segs[0]["target"], "Hola<i>")
        self.assertIn("<i>", segs[0]["all_tags"])

```

# ===== GUI SMOKE TEST =====

```

class TestGuiSmoke(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_create_mainwindow(self):
        from PyQt5.QtWidgets import QApplication
        app = QApplication.instance()
        existed = app is not None
        if not existed:
            app = QApplication(sys.argv)
        try:
            from cat_tool.ui import MainWindow
            win = MainWindow()
            self.assertIsNotNone(win)

```

```

        self.assertEqual(win.windowTitle()[4:], "mcat")
    finally:
        if not existed:
            pass
        app = None

def test_load_po_file_through_handler(self):
    from PyQt5.QtWidgets import QApplication
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import MainWindow
        win = MainWindow()
        po_content = '''msgid "Hello"

msgstr "Hola"
msgid "World"
msgstr "Mundo"
'''
        po_path = os.path.join(self.tmp.name, "test.po")
        with open(po_path, "w", encoding="utf-8") as f:
            f.write(po_content)
        segs = win.state.file_handler.load_file(po_path)
        win.populate_table(segs)
        self.assertEqual(win.table.rowCount(), 2)
        self.assertEqual(win.table.item(0, 0).text(), "Hello")
    finally:
        if not existed:
            pass
        app = None

def test_load_po_and_translate(self):
    from PyQt5.QtWidgets import QApplication
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import MainWindow
        win = MainWindow()
        po_content = '''msgid "Hello"

msgstr ""
'''
        po_path = os.path.join(self.tmp.name, "test.po")
        with open(po_path, "w", encoding="utf-8") as f:
            f.write(po_content)
        segs = win.state.file_handler.load_file(po_path)
        win.populate_table(segs)
        win.state.file_handler.segments[0]["target"] = "Hola"
        win.update_table_row_status(0, win.state.file_handler.segments[0])
        self.assertEqual(win.state.file_handler.segments[0]["target"], "Hola")
    finally:
        if not existed:
            pass
        app = None

def test_qa_dialog_creation(self):
    from PyQt5.QtWidgets import QApplication, QDialog
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.qa import QAIssue
        from cat_tool.ui import QADialog
        issues = [
            QAIssue("error", "Missing translation", 0, "Hello", "", "No target"),
        ]
        dlg = QADialog(issues)
        self.assertIsInstance(dlg, QDialog)
    finally:
        if not existed:
            pass
        app = None

def test_shortcuts_dialog_creation(self):
    from PyQt5.QtWidgets import QApplication, QDialog
    app = QApplication.instance()

```

```

        existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import ShortcutsDialog
        dlg = ShortcutsDialog()
        self.assertIsInstance(dlg, QDialog)
    finally:
        if not existed:
            pass
        app = None

def test_glossary_dialog_creation(self):
    from PyQt5.QtWidgets import QApplication, QDialog
    app = QApplication.instance()
    existed = app is not None
    if not existed:
        app = QApplication(sys.argv)
    try:
        from cat_tool.ui import GlossaryDialog
        dlg = GlossaryDialog()
        self.assertIsInstance(dlg, QDialog)
    finally:
        if not existed:
            pass
        app = None

# ===== FORMAT HANDLER INTEGRATION =====

class TestFileHandlerIntegration(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def _make_po(self, content):
        path = os.path.join(self.tmp.name, "test.po")
        with open(path, "w", encoding="utf-8") as f:
            f.write(content)
        return path

    def test_po_load_save_roundtrip(self):
        po = '''msgid "Hello"
msgstr "Hola"
msgid "World"
msgstr ""
'''
        path = self._make_po(po)
        fh = CATFileHandler()
        segs = fh.load_file(path)
        segs[0]["target"] = ";Hola!"
        segs[1]["target"] = "Mundo"
        out = os.path.join(self.tmp.name, "out.po")
        fh.save_file(out)
        fh2 = CATFileHandler()
        segs2 = fh2.load_file(out)
        self.assertEqual(segs2[0]["target"], ";Hola!")
        self.assertEqual(segs2[1]["target"], "Mundo")

    def test_txt_load_save_roundtrip(self):
        txt_path = os.path.join(self.tmp.name, "input.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Line one\nLine two\nLine three")
        fh = CATFileHandler()
        segs = fh.load_file(txt_path)
        self.assertEqual(len(segs), 3)
        segs[0]["target"] = "Línea uno"
        segs[1]["target"] = "Línea dos"
        out = os.path.join(self.tmp.name, "out.txt")
        fh.render_translated(out)
        with open(out, encoding="utf-8") as f:
            content = f.read()
        self.assertIn("Línea uno", content)
        self.assertIn("Línea dos", content)

    def test_po_convert_to_po_from_txt(self):
        from cat_tool.file_handler import convert_to_po

```

```

txt_path = os.path.join(self.tmp.name, "source.txt")
with open(txt_path, "w", encoding="utf-8") as f:
    f.write("Hello\nWorld")
po_out = os.path.join(self.tmp.name, "converted.po")
convert_to_po(txt_path, po_out)
self.assertTrue(os.path.exists(po_out))
fh = CATFileHandler()
segs = fh.load_file(po_out)
self.assertEqual(len(segs), 2)

def test_factory_xliff_load(self):
    xliff = """<?xml version="1.0" encoding="utf-8"?>
<xliff xmlns="urn:oasis:names:tc:xliff:document:1.2" version="1.2">
  <file source-language="en" target-language="es" datatype="plaintext" original="test.txt">
    <body>
      <trans-unit id="1">
        <source>Hello</source>
        <target>Hola</target>
      </trans-unit>
    </body>
  </file>
</xliff>"""
    path = os.path.join(self.tmp.name, "test.xlf")
    with open(path, "w", encoding="utf-8") as f:
        f.write(xliff)
    fh = CATFileHandler()
    segs = fh.load_file(path)
    self.assertGreater(len(segs), 0)
    self.assertEqual(segs[0]["source"].strip(), "Hello")

def test_render_no_current_path_raises(self):
    fh = CATFileHandler()
    fh.segments = [{"source": "Hello", "target": "Hola"}]
    with self.assertRaises(ValueError):
        fh.render_translated("out.txt")

def test_load_mcatdb_and_save_mcatdb_roundtrip(self):
    from cat_tool.mcat_format_manager import MCatProjectManager
    db_path = os.path.join(self.tmp.name, "proj.mcat.db")
    MCatProjectManager.create_project_file(db_path, "en", "es")
    MCatProjectManager.import_extracted_segments(db_path, [
        {"file_id": "f.txt", "source": "Hello", "target": "Hola", "status": "Confirmed", "match_score": 100},
    ])
    fh = CATFileHandler()
    segs = fh.load_file(db_path)
    self.assertEqual(len(segs), 1)
    self.assertEqual(segs[0]["source"], "Hello")
    self.assertEqual(segs[0]["target"], "Hola")
    out_db = os.path.join(self.tmp.name, "copy.mcat.db")
    fh.save_file(out_db)
    self.assertTrue(os.path.exists(out_db))
    fh2 = CATFileHandler()
    segs2 = fh2.load_file(out_db)
    self.assertEqual(segs2[0]["source"], "Hello")

# ===== REAL DOCUMENT LOAD TESTS =====

class TestRealDocumentLoad(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_docx_load(self):
        from docx import Document
        doc = Document()
        doc.add_paragraph("Hello World")
        doc.add_paragraph("Second paragraph")
        docx_path = os.path.join(self.tmp.name, "test.docx")
        doc.save(docx_path)
        from cat_tool.formats.docx import DocxHandler
        segs = DocxHandler().load(docx_path)
        self.assertGreaterEqual(len(segs), 2)
        texts = [s["source"] for s in segs]
        self.assertIn("Hello World", texts)
        self.assertIn("Second paragraph", texts)
    def test_docx_render(self):

```

```

from docx import Document
doc = Document()
doc.add_paragraph("Hello")
docx_path = os.path.join(self.tmp.name, "test.docx")
doc.save(docx_path)
from cat_tool.formats.docx import DocxHandler
handler = DocxHandler()
segs = handler.load(docx_path)
segs[0]["target"] = "Hola"
out_path = os.path.join(self.tmp.name, "out.docx")
handler.render(docx_path, segs, out_path)
self.assertTrue(os.path.exists(out_path))
doc2 = Document(out_path)
texts = [p.text for p in doc2.paragraphs]
self.assertIn("Hola", texts)

def test_pptx_load(self):
    from pptx import Presentation
    from pptx.util import Inches
    prs = Presentation()
    slide = prs.slides.add_slide(prs.slide_layouts[6])
    textBox = slide.shapes.add_textbox(Inches(1), Inches(1), Inches(8), Inches(2))
    tf = textBox.text_frame
    tf.text = "Hello from PowerPoint"
    pptx_path = os.path.join(self.tmp.name, "test.pptx")
    prs.save(pptx_path)
    from cat_tool.formats.pptx import PptxHandler
    segs = PptxHandler().load(pptx_path)
    texts = [s["source"] for s in segs]
    self.assertIn("Hello from PowerPoint", texts)

def test_pptx_render(self):
    from pptx import Presentation
    from pptx.util import Inches
    prs = Presentation()
    slide = prs.slides.add_slide(prs.slide_layouts[6])
    textBox = slide.shapes.add_textbox(Inches(1), Inches(1), Inches(8), Inches(2))
    tf = textBox.text_frame
    tf.text = "Hello"
    pptx_path = os.path.join(self.tmp.name, "test.pptx")
    prs.save(pptx_path)
    from cat_tool.formats.pptx import PptxHandler
    handler = PptxHandler()
    segs = handler.load(pptx_path)
    segs[0]["target"] = "Hola"
    out_path = os.path.join(self.tmp.name, "out.pptx")
    handler.render(pptx_path, segs, out_path)
    self.assertTrue(os.path.exists(out_path))
    prs2 = Presentation(out_path)
    for slide in prs2.slides:
        for shape in slide.shapes:
            if shape.has_text_frame:
                self.assertIn("Hola", shape.text_frame.text)

def test_xlsx_load(self):
    from openpyxl import Workbook
    wb = Workbook()
    ws = wb.active
    ws["A1"] = "Hello"
    ws["B1"] = "World"
    xlsx_path = os.path.join(self.tmp.name, "test.xlsx")
    wb.save(xlsx_path)
    from cat_tool.formats.xlsx import XlsxHandler
    segs = XlsxHandler().load(xlsx_path)
    texts = [s["source"] for s in segs]
    self.assertIn("Hello", texts)
    self.assertIn("World", texts)

def test_xlsx_render(self):
    from openpyxl import Workbook
    wb = Workbook()
    ws = wb.active
    ws["A1"] = "Hello"
    xlsx_path = os.path.join(self.tmp.name, "test.xlsx")
    wb.save(xlsx_path)
    from cat_tool.formats.xlsx import XlsxHandler
    handler = XlsxHandler()
    segs = handler.load(xlsx_path)
    segs[0]["target"] = "Hola"

```

```

        out_path = os.path.join(self.tmp.name, "out.xlsx")
        handler.render(xlsx_path, segs, out_path)
        self.assertTrue(os.path.exists(out_path))
        from openpyxl import load_workbook
        wb2 = load_workbook(out_path)
        self.assertEqual(wb2.active["A1"].value, "Hola")

if __name__ == "__main__":
    unittest.main()

```

## teststest\_more.py — 1026 lines

- Class **TestSpellChecker** (line 12)
- Class **TestSpellCheckerConstants** (line 250)
- Class **TestAppSettings** (line 267)
- Class **TestQAModuleAdditional** (line 362)
- Class **TestSegmentationAdditional** (line 396)
- Class **TestTagUtilsAdditional** (line 446)
- Class **TestFileHandlerAdditional** (line 479)
- Class **TestXlsHandler** (line 544)
- Class **TestOdfHandler** (line 597)
- Class **TestMakeSegmentsFromTextsAdditional** (line 664)
- Class **TestConvertToPoAdditional** (line 683)
- Class **TestMcatProjectManagerAdditional** (line 717)
- Class **TestQACheckFunctions** (line 768)
- Class **TestTranslationMemoryAdditional** (line 799)
- Class **TestGlossaryAdditional** (line 829)
- Function **setUpClass** (line 14)
- Function **tearDownClass** (line 22)
- Function **setUp** (line 29)
- Function **\_make\_real\_dic** (line 34)
- Function **test\_empty\_checker** (line 44)
- Function **test\_auto\_load\_picks\_up\_dict** (line 54)
- Function **test\_check\_known\_word** (line 67)
- Function **test\_check\_unknown\_word** (line 80)
- Function **test\_auto\_load\_no\_dicts\_dir** (line 92)
- Function **test\_auto\_load\_empty\_dicts\_dir** (line 98)
- Function **test\_auto\_load\_skips\_non\_dic\_files** (line 105)
- Function **test\_auto\_load\_skips\_missing\_aff** (line 114)
- Function **test\_load\_dictionary\_missing\_aff** (line 124)
- Function **test\_load\_dictionary\_copies\_and\_loads** (line 130)
- Function **test\_suggest\_with\_dict** (line 144)
- Function **test\_check\_text\_deduplicates** (line 157)
- Function **test\_check\_text\_catches\_errors** (line 171)
- Function **test\_lang\_id\_from\_archive\_with\_underscore** (line 185)
- Function **test\_lang\_id\_from\_archive\_uses\_filename\_fallback** (line 196)
- Function **test\_lang\_id\_from\_archive\_with\_dic\_inside** (line 206)
- Function **test\_load\_from\_archive\_nonexistent\_returns\_false** (line 217)
- Function **test\_load\_from\_archive\_bad\_zip\_returns\_false** (line 223)
- Function **test\_load\_from\_archive\_good\_zip** (line 232)
- Function **test\_dicts\_dir\_uses\_appdata** (line 251)
- Function **setUp** (line 268)
- Function **tearDown** (line 275)
- Function **\_settings\_path** (line 281)
- Function **test\_defaults** (line 284)
- Function **test\_save\_and\_load** (line 290)
- Function **test\_partial\_overrides\_keep\_defaults** (line 304)

- Function **test\_corrupted\_json\_falls\_back\_to\_defaults** (line 313)
- Function **test\_missing\_file\_falls\_back\_to\_defaults** (line 321)
- Function **test\_contains** (line 326)
- Function **test\_get** (line 332)
- Function **test\_as\_dict** (line 339)
- Function **test\_setitem\_updates\_dict** (line 347)
- Function **test\_save\_creates\_directory** (line 353)
- Function **test\_mixed\_severity** (line 363)
- Function **test\_empty\_description** (line 370)
- Function **test\_extract\_numbers\_edge\_cases** (line 375)
- Function **test\_whitespace\_no\_source** (line 381)
- Function **test\_count\_tags\_empty** (line 387)
- Function **test\_none\_text** (line 397)
- Function **test\_multiple\_periods** (line 402)
- Function **test\_exclamation\_and\_question** (line 407)
- Function **test\_break\_before\_literal** (line 412)
- Function **test\_break\_before\_combined** (line 421)
- Function **test\_break\_before\_and\_after** (line 432)
- Function **test\_extract\_single\_tag** (line 447)
- Function **test\_extract\_nested\_tags** (line 452)
- Function **test\_extract\_self\_closing\_xml** (line 460)
- Function **test\_restore\_with\_none\_tags** (line 466)
- Function **test\_apply\_with\_none\_text** (line 471)
- Function **setUp** (line 480)
- Function **tearDown** (line 484)
- Function **test\_atomic\_save\_failure\_cleans\_temp** (line 487)
- Function **test\_save\_file\_without\_loading\_raises** (line 499)
- Function **test\_current\_file\_path\_none\_on\_new** (line 504)
- Function **test\_load\_with\_mcatdb\_extension** (line 507)
- Function **test\_render\_without\_handler\_raises** (line 521)
- Function **test\_save\_file\_mcatdb\_preserves\_path** (line 527)
- Function **setUp** (line 545)
- Function **tearDown** (line 548)
- Function **test\_xls\_load\_basic** (line 551)
- Function **test\_xls\_load\_empty\_workbook** (line 570)
- Function **test\_xls\_load\_nonexistent\_raises** (line 586)
- Function **setUp** (line 598)
- Function **tearDown** (line 601)
- Function **test\_load\_odt** (line 604)
- Function **test\_odf\_load\_nonexistent\_raises** (line 620)
- Function **test\_odf\_render\_updates\_target** (line 628)
- Function **test\_odf\_render\_no\_segments\_output\_created** (line 648)
- Function **test\_none\_text\_skipped** (line 665)
- Function **test\_all\_empty\_skipped** (line 670)
- Function **test\_normal\_works** (line 675)
- Function **setUp** (line 684)
- Function **tearDown** (line 687)
- Function **test\_convert\_to\_po\_creates\_file** (line 690)
- Function **test\_convert\_to\_po\_roundtrip** (line 703)
- Function **setUp** (line 718)
- Function **tearDown** (line 722)
- Function **test\_get\_editor\_stream\_empty\_db** (line 725)
- Function **test\_get\_editor\_stream\_filter\_nonexistent\_status** (line 731)
- Function **test\_update\_segment\_invalid\_id** (line 737)
- Function **test\_import\_and\_get\_stream** (line 742)

- Function **test\_update\_segment** (line 753)
- Function **test\_check\_terminology\_no\_glossary** (line 769)
- Function **test\_check\_inconsistent\_translations\_empty\_source** (line 775)
- Function **test\_tag\_mismatch\_no\_source\_tags** (line 784)
- Function **test\_count\_tags\_with\_tags** (line 790)
- Function **setUp** (line 800)
- Function **tearDown** (line 806)
- Function **test\_add\_translation\_none\_target** (line 809)
- Function **test\_get\_fuzzy\_matches\_empty\_db** (line 814)
- Function **test\_get\_fuzzy\_matches\_with\_min\_score** (line 818)
- Function **test\_get\_fuzzy\_matches\_extreme\_min\_score** (line 823)
- Function **setUp** (line 830)
- Function **tearDown** (line 836)
- Function **test\_remove\_nonexistent\_term** (line 839)
- Function **test\_check\_segment\_none\_text** (line 844)
- Function **test\_add\_term\_special\_chars** (line 848)
- Function **test\_add\_term\_with\_langs** (line 853)
- Function **test\_import\_tmx\_basic** (line 860)
- Function **test\_import\_tmx\_nonexistent\_file** (line 887)
- Function **test\_import\_tmx\_empty\_body** (line 891)
- Function **test\_import\_tmx\_missing\_body** (line 905)
- Function **test\_import\_tmx\_no\_srclang\_header** (line 917)
- Function **test\_import\_tmx\_with\_explicit\_source\_lang** (line 935)
- Function **test\_peek\_tmx\_basic** (line 957)
- Function **test\_peek\_tmx\_no\_srclang** (line 975)
- Function **test\_export\_tmx\_empty** (line 992)
- Function **test\_export\_tmx\_roundtrip** (line 997)
- Function **test\_export\_tmx\_source\_lang\_override** (line 1014)
- Function **\_fail** (line 492)

```
import unittest
import os
import sys
import tempfile
import json
import shutil
import importlib

# ===== SPELL CHECKER TESTS =====

class TestSpellChecker(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls._tmp = tempfile.TemporaryDirectory()
        cls._orig_appdata = os.environ.get("APPDATA")
        os.environ["APPDATA"] = cls._tmp.name
        import cat_tool.spellcheck
        importlib.reload(cat_tool.spellcheck)

    @classmethod
    def tearDownClass(cls):
        if cls._orig_appdata:
            os.environ["APPDATA"] = cls._orig_appdata
        else:
            del os.environ["APPDATA"]
        cls._tmp.cleanup()

    def setUp(self):
        dicts_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts")
        if os.path.isdir(dicts_dir):
            shutil.rmtree(dicts_dir)

    def _make_real_dic(self, dic_path, aff_path, words=None):
        os.makedirs(os.path.dirname(dic_path), exist_ok=True)
        with open(aff_path, "w", encoding="utf-8") as f:
```

```

        f.write("SET UTF-8\n")
    words = words or ["hello", "world"]
    with open(dic_path, "w", encoding="utf-8") as f:
        f.write(f"{len(words)}\n")
        for w in words:
            f.write(f"{w}\n")

def test_empty_checker(self):
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    self.assertFalse(sc.has_dict)
    self.assertEqual(sc.loaded_names, [])
    self.assertTrue(sc.check("anything"))
    self.assertEqual(sc.suggest("anything"), [])
    self.assertEqual(sc.check_text(""), [])
    self.assertEqual(sc.check_text(None), [])

def test_auto_load_picks_up_dict(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["Hello", "World"],
    )
    sc = SpellChecker()
    sc.auto_load()
    self.assertTrue(sc.has_dict)
    self.assertIn("en_US", sc.loaded_names)

def test_check_known_word(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["Hello", "World"],
    )
    sc = SpellChecker()
    sc.auto_load()
    self.assertTrue(sc.check("Hello"))
    self.assertTrue(sc.check("World"))

def test_check_unknown_word(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["Hello", "World"],
    )
    sc = SpellChecker()
    sc.auto_load()
    self.assertFalse(sc.check("xyzzzzz"))

def test_auto_load_no_dicts_dir(self):
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    sc.auto_load()
    self.assertFalse(sc.has_dict)

def test_auto_load_empty_dicts_dir(self):
    from cat_tool.spellcheck import SpellChecker
    os.makedirs(os.path.join(os.environ["APPDATA"], "mcat", "dicts"), exist_ok=True)
    sc = SpellChecker()
    sc.auto_load()
    self.assertFalse(sc.has_dict)

def test_auto_load_skips_non_dic_files(self):
    from cat_tool.spellcheck import SpellChecker
    dicts_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "es")
    os.makedirs(dicts_dir, exist_ok=True)
    open(os.path.join(dicts_dir, "readme.txt"), "w").close()
    sc = SpellChecker()
    sc.auto_load()
    self.assertFalse(sc.has_dict)

def test_auto_load_skips_missing_aff(self):
    from cat_tool.spellcheck import SpellChecker

```

```

dicts_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "fr")
os.makedirs(dict_dir, exist_ok=True)
with open(os.path.join(dict_dir, "fr.dic"), "w", encoding="utf-8") as f:
    f.write("l\nmot\n")
sc = SpellChecker()
sc.auto_load()
self.assertFalse(sc.has_dict)

def test_load_dictionary_missing_aff(self):
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    ok = sc.load_dictionary(r"C:\nonexistent.dic")
    self.assertFalse(ok)

def test_load_dictionary_copies_and_loads(self):
    from cat_tool.spellcheck import SpellChecker
    src_dir = os.path.join(self._tmp.name, "src")
    os.makedirs(src_dir, exist_ok=True)
    dic_path = os.path.join(src_dir, "custom.dic")
    aff_path = os.path.join(src_dir, "custom.aff")
    self._make_real_dic(dic_path, aff_path, ["foo", "bar"])
    sc = SpellChecker()
    ok = sc.load_dictionary(dic_path)
    self.assertTrue(ok)
    self.assertIn("custom", sc.loaded_names)
    self.assertTrue(sc.check("foo"))
    self.assertFalse(sc.check("baz"))

def test_suggest_with_dict(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["hello", "world"],
    )
    sc = SpellChecker()
    sc.auto_load()
    suggs = sc.suggest("worlt")
    self.assertIsInstance(suggs, list)

def test_check_text_deduplicates(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["hello"],
    )
    sc = SpellChecker()
    sc.auto_load()
    errors = sc.check_text("hello world world hello")
    err_words = [e[0] for e in errors]
    self.assertEqual(err_words.count("world"), 1)

def test_check_text_catches_errors(self):
    from cat_tool.spellcheck import SpellChecker
    dic_dir = os.path.join(os.environ["APPDATA"], "mcat", "dicts", "en_US")
    self._make_real_dic(
        os.path.join(dic_dir, "en_US.dic"),
        os.path.join(dic_dir, "en_US.aff"),
        ["hello", "world", "test"],
    )
    sc = SpellChecker()
    sc.auto_load()
    errors = sc.check_text("hello worlt test")
    err_words = [e[0] for e in errors]
    self.assertIn("worlt", err_words)

def test_lang_id_from_archive_with_underscore(self):
    import zipfile
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    archive_path = os.path.join(self._tmp.name, "dict_en_US.oxt")
    with zipfile.ZipFile(archive_path, "w") as z:
        z.writestr("en_US.dic", "l\nword\n")
        z.writestr("en_US.aff", "")
    lang_id = sc._lang_id_from_archive(archive_path)
    self.assertEqual(lang_id, "en_US")

```

```

def test_lang_id_from_archive_uses_filename_fallback(self):
    import zipfile
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    archive_path = os.path.join(self._tmp.name, "spanish.oxt")
    with zipfile.ZipFile(archive_path, "w") as z:
        pass
    lang_id = sc._lang_id_from_archive(archive_path)
    self.assertEqual(lang_id, "spanish")

def test_lang_id_from_archive_with_dic_inside(self):
    import zipfile
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    archive_path = os.path.join(self._tmp.name, "dicts.zip")
    with zipfile.ZipFile(archive_path, "w") as z:
        z.writestr("es_MX.dic", "1\nhola\n")
        z.writestr("es_MX.aff", "")
    lang_id = sc._lang_id_from_archive(archive_path)
    self.assertEqual(lang_id, "es_MX")

def test_load_from_archive_nonexistent_returns_false(self):
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    ok = sc.load_from_archive(r"C:\nonexistent.zip")
    self.assertFalse(ok)

def test_load_from_archive_bad_zip_returns_false(self):
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    bad_path = os.path.join(self._tmp.name, "bad.oxt")
    with open(bad_path, "w") as f:
        f.write("not a zip")
    ok = sc.load_from_archive(bad_path)
    self.assertFalse(ok)

def test_load_from_archive_good_zip(self):
    import zipfile
    from cat_tool.spellcheck import SpellChecker
    sc = SpellChecker()
    dic_dir = os.path.join(self._tmp.name, "dictsrc")
    os.makedirs(dic_dir, exist_ok=True)
    with open(os.path.join(dic_dir, "es.dic"), "w", encoding="utf-8") as f:
        f.write("2\nhola\nmundo\n")
    with open(os.path.join(dic_dir, "es.aff"), "w", encoding="utf-8") as f:
        f.write("SET UTF-8\n")
    archive_path = os.path.join(self._tmp.name, "spanish.zip")
    with zipfile.ZipFile(archive_path, "w") as z:
        z.write(os.path.join(dic_dir, "es.dic"), "es.dic")
        z.write(os.path.join(dic_dir, "es.aff"), "es.aff")
    ok = sc.load_from_archive(archive_path)
    self.assertTrue(ok)

class TestSpellCheckerConstants(unittest.TestCase):
    def test_dicts_dir_uses_appdata(self):
        with tempfile.TemporaryDirectory() as tmp:
            old = os.environ.get("APPDATA")
            os.environ["APPDATA"] = tmp
            import cat_tool.spellcheck
            importlib.reload(cat_tool.spellcheck)
            expected = os.path.join(tmp, "mcat", "dicts")
            self.assertEqual(cat_tool.spellcheck.DICTS_DIR, expected)
            if old:
                os.environ["APPDATA"] = old
            else:
                del os.environ["APPDATA"]

# ===== SETTINGS TESTS =====

class TestAppSettings(unittest.TestCase):
    def setUp(self):
        self._tmp = tempfile.TemporaryDirectory()
        self._orig_appdata = os.environ.get("APPDATA")
        os.environ["APPDATA"] = self._tmp.name
        import cat_tool.settings
        importlib.reload(cat_tool.settings)
    def tearDown(self):

```

```

    if self._orig_appdata:
        os.environ["APPDATA"] = self._orig_appdata
    else:
        del os.environ["APPDATA"]

def _settings_path(self):
    return os.path.join(self._tmp.name, "mcat", "settings.json")

def test_defaults(self):
    from cat_tool.settings import AppSettings, DEFAULTS
    s = AppSettings().load()
    for k, v in DEFAULTS.items():
        self.assertEqual(s[k], v, f"Default {k} mismatch")

def test_save_and_load(self):
    from cat_tool.settings import AppSettings
    s = AppSettings()
    s.load()
    s["source_font_size"] = 20
    s["target_font_size"] = 16
    s["auto_save_on_segment_change"] = True
    s.save()
    self.assertTrue(os.path.exists(self._settings_path()))
    s2 = AppSettings().load()
    self.assertEqual(s2["source_font_size"], 20)
    self.assertEqual(s2["target_font_size"], 16)
    self.assertTrue(s2["auto_save_on_segment_change"])

def test_partial_overrides_keep_defaults(self):
    from cat_tool.settings import AppSettings
    os.makedirs(os.path.dirname(self._settings_path()), exist_ok=True)
    with open(self._settings_path(), "w") as f:
        json.dump({"source_font_size": 18}, f)
    s = AppSettings().load()
    self.assertEqual(s["source_font_size"], 18)
    self.assertEqual(s["target_font_size"], 13)

def test_corrupted_json_falls_back_to_defaults(self):
    from cat_tool.settings import AppSettings, DEFAULTS
    os.makedirs(os.path.dirname(self._settings_path()), exist_ok=True)
    with open(self._settings_path(), "w") as f:
        f.write("not valid json")
    s = AppSettings().load()
    self.assertEqual(s["source_font_size"], DEFAULTS["source_font_size"])

def test_missing_file_falls_back_to_defaults(self):
    from cat_tool.settings import AppSettings, DEFAULTS
    s = AppSettings().load()
    self.assertEqual(s["source_font_size"], DEFAULTS["source_font_size"])

def test_contains(self):
    from cat_tool.settings import AppSettings
    s = AppSettings().load()
    self.assertIn("source_font_size", s)
    self.assertNotIn("nonexistent_key", s)

def test_get(self):
    from cat_tool.settings import AppSettings
    s = AppSettings().load()
    self.assertEqual(s.get("source_font_size"), 13)
    self.assertIsNone(s.get("nonexistent"))
    self.assertEqual(s.get("nonexistent", 42), 42)

def test_as_dict(self):
    from cat_tool.settings import AppSettings
    s = AppSettings().load()
    d = s.as_dict()
    self.assertIsInstance(d, dict)
    self.assertIn("source_font_size", d)
    self.assertEqual(d["source_font_size"], 13)

def test_setitem_updates_dict(self):
    from cat_tool.settings import AppSettings
    s = AppSettings().load()
    s["source_font_size"] = 99
    self.assertEqual(s["source_font_size"], 99)

def test_save_creates_directory(self):
    from cat_tool.settings import AppSettings

```

```

s = AppSettings().load()
s.save()
self.assertTrue(os.path.isdir(os.path.join(self._tmp.name, "mcat")))

# ===== ADDITIONAL QA EDGE CASES =====

class TestQAModuleAdditional(unittest.TestCase):
    def test_mixed_severity(self):
        from cat_tool.qa import QAIssue
        e = QAIssue("error", "E", 0, "s", "t", "d")
        w = QAIssue("warning", "W", 1, "s", "t", "d")
        self.assertEqual(e.severity, "error")
        self.assertEqual(w.severity, "warning")

    def test_empty_description(self):
        from cat_tool.qa import QAIssue
        issue = QAIssue("error", "test", 0, "src", "tgt", "")
        self.assertEqual(issue.description, "")

    def test_extract_numbers_edge_cases(self):
        from cat_tool.qa import _extract_numbers
        self.assertEqual(_extract_numbers(""), set())
        self.assertEqual(_extract_numbers("No digits here!"), set())
        self.assertEqual(_extract_numbers("Version 1.2.3"), {"1.2", "3"})

    def test_whitespace_no_source(self):
        from cat_tool.qa import check_whitespace
        segs = [{"source": None, "target": "Hello", "translated": True}]
        issues = check_whitespace(segs)
        self.assertEqual(len(issues), 0)

    def test_count_tags_empty(self):
        from cat_tool.qa import _count_tags
        self.assertEqual(_count_tags(""), 0)
        self.assertEqual(_count_tags(None), 0)
        self.assertEqual(_count_tags("plain text"), 0)

# ===== ADDITIONAL SEGMENTATION EDGE CASES =====

class TestSegmentationAdditional(unittest.TestCase):
    def test_none_text(self):
        from cat_tool.segmentation import advanced_segmenter
        result = advanced_segmenter(None)
        self.assertEqual(result, [])

    def test_multiple_periods(self):
        from cat_tool.segmentation import advanced_segmenter
        result = advanced_segmenter("Hello... World...")
        self.assertGreaterEqual(len(result), 2)

    def test_exclamation_and_question(self):
        from cat_tool.segmentation import advanced_segmenter
        result = advanced_segmenter("Wow! Really? Yes.")
        self.assertGreaterEqual(len(result), 3)

    def test_break_before_literal(self):
        from cat_tool.segmentation import advanced_segmenter
        rules = [
            {"pattern": "@", "break_after": False, "break_before": True,
             "case_sensitive": False, "whole_word": False, "language": "All"},
        ]
        result = advanced_segmenter("hello @world @test", rules)
        self.assertGreaterEqual(len(result), 3)

    def test_break_before_combined(self):
        from cat_tool.segmentation import advanced_segmenter
        rules = [
            {"pattern": ".", "break_after": True,
             "case_sensitive": False, "whole_word": False, "language": "All"},
            {"pattern": "@", "break_after": False, "break_before": True,
             "case_sensitive": False, "whole_word": False, "language": "All"},
        ]
        result = advanced_segmenter("hello. @world. @test", rules)
        self.assertGreaterEqual(len(result), 3)

    def test_break_before_and_after(self):
        from cat_tool.segmentation import advanced_segmenter

```

```

rules = [
    {"pattern": ".", "break_after": True,
     "case_sensitive": False, "whole_word": False, "language": "All"},
    {"pattern": "@", "break_after": False, "break_before": True,
     "case_sensitive": False, "whole_word": False, "language": "All"},
]
result = advanced_segmenter("Hello. Next @mention. Done.", rules)
self.assertGreaterEqual(len(result), 3)

# ===== ADDITIONAL TAG UTILS EDGE CASES =====

class TestTagUtilsAdditional(unittest.TestCase):
    def test_extract_single_tag(self):
        from cat_tool.formats.tag_utils import extract_all_tags
        tags = extract_all_tags("<br/>")
        self.assertIn("<br/>", tags)

    def test_extract_nested_tags(self):
        from cat_tool.formats.tag_utils import extract_all_tags
        tags = extract_all_tags("<outer><inner>text</inner></outer>")
        self.assertIn("<outer>", tags)
        self.assertIn("<inner>", tags)
        self.assertIn("</inner>", tags)
        self.assertIn("</outer>", tags)

    def test_extract_self_closing_xml(self):
        from cat_tool.formats.tag_utils import extract_all_tags
        tags = extract_all_tags("<img src='x'/> and <hr/>")
        self.assertIn("<img src='x'/>", tags)
        self.assertIn("<hr/>", tags)

    def test_restore_with_none_tags(self):
        from cat_tool.formats.tag_utils import restore_tags
        result = restore_tags("Hello", None)
        self.assertEqual(result, "Hello")

    def test_apply_with_none_text(self):
        from cat_tool.formats.tag_utils import apply_tags
        result = apply_tags(None, ["<b>"])
        self.assertIsNone(result)

# ===== ADDITIONAL FILE HANDLER EDGE CASES =====

class TestFileHandlerAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.fh = __import__("cat_tool.file_handler", fromlist=["CATFileHandler"]).CATFileHandler()

    def tearDown(self):
        self.tmp.cleanup()

    def test_atomic_save_failure_cleans_temp(self):
        tmp_file = os.path.join(self.tmp.name, "test.po")
        with open(tmp_file, "w", encoding="utf-8") as f:
            f.write('msgid "Hello"\nmsgstr ""\n')
        self.fh.load_file(tmp_file)
        def _fail():
            raise RuntimeError("simulated failure")
        with self.assertRaises(RuntimeError):
            self.fh._atomic_save(_fail, tmp_file)
        temps = [f for f in os.listdir(self.tmp.name) if f.startswith(".mcat_tmp_")]
        self.assertEqual(len(temps), 0)

    def test_save_file_without_loading_raises(self):
        fh = __import__("cat_tool.file_handler", fromlist=["CATFileHandler"]).CATFileHandler()
        with self.assertRaises(ValueError):
            fh.save_file()

    def test_current_file_path_none_on_new(self):
        self.assertIsNone(self.fh.current_file_path)

    def test_load_with_mcatdb_extension(self):
        from cat_tool.formats.mcatdb import McatDbHandler
        path = os.path.join(self.tmp.name, "project.mcat.db")
        McatDbHandler().save([
            "index": 0, "source": "Hello", "target": "Hola",
            "source_clean": "Hello", "target_clean": "Hola",

```

```

        "all_tags": [], "fuzzy": False, "translated": True,
        "locations": "", "notes": "", "source_location": {},
        "store": None, "unit": None,
    }], path)
    segs = self.fh.load_file(path)
    self.assertEqual(len(segs), 1)
    self.assertEqual(segs[0]["source"], "Hello")

def test_render_without_handler_raises(self):
    self.fh.segments = [{"source": "Hello", "target": "Hola"}]
    self.fh.current_file_path = os.path.join(self.tmp.name, "dummy.txt")
    with self.assertRaises(ValueError):
        self.fh.render_translated(os.path.join(self.tmp.name, "out.txt"))

def test_save_file_mcatdb_preserves_path(self):
    from cat_tool.formats.mcatdb import McatDbHandler
    path = os.path.join(self.tmp.name, "project.mcat.db")
    McatDbHandler().save([
        {
            "index": 0, "source": "Hello", "target": "",
            "source_clean": "Hello", "target_clean": "",
            "all_tags": [], "fuzzy": False, "translated": False,
            "locations": "", "notes": "", "source_location": {},
            "store": None, "unit": None,
        }
    ], path)
    self.fh.load_file(path)
    self.fh.save_file()
    self.assertEqual(self.fh.current_file_path, path)

# ===== XLS HANDLER TESTS =====

class TestXlsHandler(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_xls_load_basic(self):
        try:
            from cat_tool.formats.xls import XlsHandler
        except ImportError:
            self.skipTest("XlsHandler not available")
        try:
            import xlwt
        except ImportError:
            self.skipTest("xlwt not available")
        wb = xlwt.Workbook()
        ws = wb.add_sheet("Sheet1")
        ws.write(0, 0, "Hello")
        ws.write(0, 1, "World")
        path = os.path.join(self.tmp.name, "test.xls")
        wb.save(path)
        segs = XlsHandler().load(path)
        texts = [s["source"] for s in segs]
        self.assertIn("Hello", texts)

    def test_xls_load_empty_workbook(self):
        try:
            from cat_tool.formats.xls import XlsHandler
        except ImportError:
            self.skipTest("XlsHandler not available")
        try:
            import xlwt
        except ImportError:
            self.skipTest("xlwt not available")
        wb = xlwt.Workbook()
        wb.add_sheet("Empty")
        path = os.path.join(self.tmp.name, "empty.xls")
        wb.save(path)
        segs = XlsHandler().load(path)
        self.assertEqual(len(segs), 0)

    def test_xls_load_nonexistent_raises(self):
        try:
            from cat_tool.formats.xls import XlsHandler
        except ImportError:
            self.skipTest("XlsHandler not available")
        with self.assertRaises(FileNotFoundError):

```

```

        XlsHandler().load(r"C:\nonexistent.xls")

# ===== ODF HANDLER TESTS =====

class TestOdfHandler(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_load_odt(self):
        try:
            from odf.opendocument import OpenDocumentText
            from odf.text import P
        except ImportError:
            self.skipTest("odf not available")
        doc = OpenDocumentText()
        p = P(text="Hello ODF World")
        doc.text.addElement(p)
        path = os.path.join(self.tmp.name, "test.odt")
        doc.save(path)
        from cat_tool.formats.odf import OdfHandler
        segs = OdfHandler().load(path)
        texts = [s["source"] for s in segs]
        self.assertIn("Hello ODF World", texts)

    def test_odf_load_nonexistent_raises(self):
        try:
            from cat_tool.formats.odf import OdfHandler
        except ImportError:
            self.skipTest("odf not available")
        with self.assertRaises(FileNotFoundError):
            OdfHandler().load(r"C:\nonexistent.odt")

    def test_odf_render_updates_target(self):
        try:
            from odf.opendocument import OpenDocumentText
            from odf.text import P
        except ImportError:
            self.skipTest("odf not available")
        doc = OpenDocumentText()
        p = P(text="Hello")
        doc.text.addElement(p)
        path = os.path.join(self.tmp.name, "test.odt")
        doc.save(path)
        from cat_tool.formats.odf import OdfHandler
        handler = OdfHandler()
        segs = handler.load(path)
        segs[0]["target"] = "Hola"
        out_path = os.path.join(self.tmp.name, "out.odt")
        handler.render(path, segs, out_path)
        segs2 = handler.load(out_path)
        self.assertEqual(segs2[0]["source"], "Hola")

    def test_odf_render_no_segments_output_created(self):
        try:
            from odf.opendocument import OpenDocumentText
        except ImportError:
            self.skipTest("odf not available")
        doc = OpenDocumentText()
        path = os.path.join(self.tmp.name, "empty.odt")
        doc.save(path)
        from cat_tool.formats.odf import OdfHandler
        out_path = os.path.join(self.tmp.name, "out.odt")
        OdfHandler().render(path, [], out_path)
        self.assertTrue(os.path.exists(out_path))

# ===== ADDITIONAL FORMAT EDGE CASES =====

class TestMakeSegmentsFromTextsAdditional(unittest.TestCase):
    def test_none_text_skipped(self):
        from cat_tool.formats.text_base import make_segments_from_texts
        segs = make_segments_from_texts(["Hello", None, "World"])
        self.assertEqual(len(segs), 2)

    def test_all_empty_skipped(self):

```

```

        from cat_tool.formats._text_base import make_segments_from_texts
        segs = make_segments_from_texts(["", "", None])
        self.assertEqual(len(segs), 0)

    def test_normal_works(self):
        from cat_tool.formats._text_base import make_segments_from_texts
        segs = make_segments_from_texts(["Hello", "World"])
        self.assertEqual(len(segs), 2)

# ===== CONVERT TO PO EDGE CASES =====

class TestConvertToPoAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()

    def tearDown(self):
        self.tmp.cleanup()

    def test_convert_to_po_creates_file(self):
        from cat_tool.file_handler import convert_to_po
        txt_path = os.path.join(self.tmp.name, "source.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Hello\nWorld\nTest")
        out = os.path.join(self.tmp.name, "output.po")
        convert_to_po(txt_path, out)
        self.assertTrue(os.path.exists(out))
        with open(out, encoding="utf-8") as f:
            content = f.read()
        self.assertIn("Hello", content)
        self.assertIn("World", content)

    def test_convert_to_po_roundtrip(self):
        from cat_tool.file_handler import CATFileHandler, convert_to_po
        txt_path = os.path.join(self.tmp.name, "source.txt")
        with open(txt_path, "w", encoding="utf-8") as f:
            f.write("Line one\nLine two")
        po_path = os.path.join(self.tmp.name, "conv.po")
        convert_to_po(txt_path, po_path)
        fh = CATFileHandler()
        segs = fh.load_file(po_path)
        self.assertEqual(len(segs), 2)

# ===== MCAT FORMAT MANAGER ADDITIONAL TESTS =====

class TestMcatProjectManagerAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp.name, "test.mcat.db")

    def tearDown(self):
        self.tmp.cleanup()

    def test_get_editor_stream_empty_db(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        rows = MCatProjectManager.get_editor_stream(self.db_path)
        self.assertEqual(len(rows), 0)

    def test_get_editor_stream_filter_nonexistent_status(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        rows = MCatProjectManager.get_editor_stream(self.db_path, filter_status="NonExistent")
        self.assertEqual(len(rows), 0)

    def test_update_segment_invalid_id(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        MCatProjectManager.update_segment_translation(self.db_path, 999, "test", "Confirmed")

    def test_import_and_get_stream(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        segments = [
            {"file_id": "test.txt", "source": "Hello", "target": "Hola", "status": "Confirmed", "match_score": 100},
        ]
        MCatProjectManager.import_extracted_segments(self.db_path, segments)
        rows = MCatProjectManager.get_editor_stream(self.db_path)

```

```

        self.assertEqual(len(rows), 1)
        self.assertEqual(rows[0]["source_text"], "Hello")

    def test_update_segment(self):
        from cat_tool.mcat_format_manager import MCatProjectManager
        MCatProjectManager.create_project_file(self.db_path, "en", "es")
        MCatProjectManager.import_extracted_segments(self.db_path, [
            {"file_id": "a.txt", "source": "Hello", "target": "", "status": "Untranslated", "match_score": 0},
        ])
        rows = MCatProjectManager.get_editor_stream(self.db_path)
        unit_id = rows[0]["id"]
        MCatProjectManager.update_segment_translation(self.db_path, unit_id, "Hola", "Confirmed")
        rows = MCatProjectManager.get_editor_stream(self.db_path)
        self.assertEqual(rows[0]["target_text"], "Hola")

# ===== DEDICATED QA CHECK TESTS =====

class TestQACheckFunctions(unittest.TestCase):
    def test_check_terminology_no_glossary(self):
        from cat_tool.qa import check_terminology
        segs = [{"source": "cat", "target": "gato", "translated": True}]
        with self.assertRaises(AttributeError):
            check_terminology(segs, None)

    def test_check_inconsistent_translations_empty_source(self):
        from cat_tool.qa import check_inconsistent_translations
        segs = [
            {"source": "", "target": "Hola", "translated": True},
            {"source": "", "target": "Hello", "translated": True},
        ]
        issues = check_inconsistent_translations(segs)
        self.assertEqual(len(issues), 0)

    def test_tag_mismatch_no_source_tags(self):
        from cat_tool.qa import check_tag_mismatch
        segs = [{"source": "Hello", "target": "Hola {0}", "all_tags": []}]
        issues = check_tag_mismatch(segs)
        self.assertEqual(len(issues), 0)

    def test_count_tags_with_tags(self):
        from cat_tool.qa import _count_tags
        from cat_tool.formats.tag_utils import PH_L, PH_R
        text = f"{PH_L}0{PH_R} and {PH_L}1{PH_R}"
        self.assertEqual(_count_tags(text), 2)

# ===== MEMORY / EDGE CASE TESTS =====

class TestTranslationMemoryAdditional(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp.name, "tm.db")
        from cat_tool.tm import TranslationMemory
        self.tm = TranslationMemory(self.db_path)

    def tearDown(self):
        self.tmp.cleanup()

    def test_add_translation_none_target(self):
        self.tm.add_translation("Hello", None)
        matches = self.tm.get_fuzzy_matches("Hello")
        self.assertEqual(len(matches), 0)

    def test_get_fuzzy_matches_empty_db(self):
        matches = self.tm.get_fuzzy_matches("Anything")
        self.assertEqual(len(matches), 0)

    def test_get_fuzzy_matches_with_min_score(self):
        self.tm.add_translation("Hello world", "Hola mundo")
        matches = self.tm.get_fuzzy_matches("Hello world!", min_score=90.0)
        self.assertGreater(len(matches), 0)

    def test_get_fuzzy_matches_extreme_min_score(self):
        self.tm.add_translation("Hello", "Hola")
        matches = self.tm.get_fuzzy_matches("Completely different", min_score=0.0)
        self.assertGreater(len(matches), 0)

class TestGlossaryAdditional(unittest.TestCase):
    def setUp(self):

```

```

        self.tmp = tempfile.TemporaryDirectory()
        self.db_path = os.path.join(self.tmp.name, "glossary.db")
        from cat_tool.glossary import Glossary
        self.gl = Glossary(self.db_path)

    def tearDown(self):
        self.tmp.cleanup()

    def test_remove_nonexistent_term(self):
        self.gl.remove_term("nonexistent")
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 0)

    def test_check_segment_none_text(self):
        matches = self.gl.check_segment(None)
        self.assertEqual(len(matches), 0)

    def test_add_term_special_chars(self):
        self.gl.add_term("cafe", "coffee")
        matches = self.gl.check_segment("I like cafe")
        self.assertEqual(len(matches), 1)

    def test_add_term_with_langs(self):
        self.gl.add_term("hello", "hola", source_lang="en", target_lang="es")
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 1)
        self.assertEqual(terms[0]["source_lang"], "en")
        self.assertEqual(terms[0]["target_lang"], "es")

    def test_import_tmx_basic(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext"/>
  <body>
    <tu>
      <tuv xml:lang="en"><seg>Hello</seg></tuv>
      <tuv xml:lang="es"><seg>Hola</seg></tuv>
    </tu>
    <tu>
      <tuv xml:lang="en"><seg>Goodbye</seg></tuv>
      <tuv xml:lang="es"><seg>Adiós</seg></tuv>
    </tu>
  </body>
</tmx>"""
        tmx_path = os.path.join(self.tmp.name, "test.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path)
        self.assertEqual(count, 2)
        terms = self.gl.get_all_terms()
        self.assertEqual(len(terms), 2)
        for t in terms:
            self.assertEqual(t["source_lang"], "en")
            self.assertEqual(t["target_lang"], "es")

    def test_import_tmx_nonexistent_file(self):
        with self.assertRaises(FileNotFoundError):
            self.gl.import_tmx("nonexistent.tmx")

    def test_import_tmx_empty_body(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext"/>
  <body>
  </body>
</tmx>"""
        tmx_path = os.path.join(self.tmp.name, "empty.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path)
        self.assertEqual(count, 0)

    def test_import_tmx_missing_body(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext"/>

```

```

</tmx>"""
    tmx_path = os.path.join(self.tmp.name, "nobody.tmx")
    with open(tmx_path, "w", encoding="utf-8") as f:
        f.write(tmx_content)
    count = self.gl.import_tmx(tmx_path)
    self.assertEqual(count, 0)

    def test_import_tmx_no_srclang_header(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header datatype="plaintext">
    <body>
      <tu>
        <tuv xml:lang="fr"><seg>Bonjour</seg></tuv>
        <tuv xml:lang="en"><seg>Hello</seg></tuv>
      </tu>
    </body>
  </tmx>"""
        tmx_path = os.path.join(self.tmp.name, "nosrclang.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path)
        self.assertEqual(count, 1)

    def test_import_tmx_with_explicit_source_lang(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext">
    <body>
      <tu>
        <tuv xml:lang="en"><seg>Cat</seg></tuv>
        <tuv xml:lang="fr"><seg>Chat</seg></tuv>
      </tu>
    </body>
  </tmx>"""
        tmx_path = os.path.join(self.tmp.name, "explicit.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        count = self.gl.import_tmx(tmx_path, source_lang="fr")
        self.assertEqual(count, 1)
        terms = self.gl.get_all_terms()
        self.assertEqual(terms[0]["source"], "Chat")
        self.assertEqual(terms[0]["target"], "Cat")
        self.assertEqual(terms[0]["source_lang"], "fr")

    def test_peek_tmx_basic(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header srclang="en" datatype="plaintext">
    <body>
      <tu><tuv xml:lang="en"><seg>Hello</seg></tuv><tuv xml:lang="es"><seg>Hola</seg></tuv>
      <tu><tuv xml:lang="en"><seg>Bye</seg></tuv><tuv xml:lang="es"><seg>Adiós</seg></tuv>
    </body>
  </tmx>"""
        tmx_path = os.path.join(self.tmp.name, "peek.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        info = self.gl.peek_tmx(tmx_path)
        self.assertEqual(info["source_lang"], "en")
        self.assertEqual(info["target_lang"], "es")
        self.assertEqual(info["count"], 2)

    def test_peek_tmx_no_srclang(self):
        import xml.etree.ElementTree as ET
        tmx_content = """<?xml version="1.0" encoding="UTF-8"?>
<tmx version="1.4">
  <header datatype="plaintext">
    <body>
      <tu><tuv xml:lang="fr"><seg>Bonjour</seg></tuv><tuv xml:lang="en"><seg>Hello</seg></tuv>
    </body>
  </tmx>"""
        tmx_path = os.path.join(self.tmp.name, "peek2.tmx")
        with open(tmx_path, "w", encoding="utf-8") as f:
            f.write(tmx_content)
        info = self.gl.peek_tmx(tmx_path)
        self.assertEqual(info["source_lang"], "")

```

```

        self.assertIn("fr", info["target_lang"])
        self.assertEqual(info["count"], 1)

    def test_export_tmx_empty(self):
        tmx_path = os.path.join(self.tmp.name, "out.tmx")
        count = self.gl.export_tmx(tmx_path)
        self.assertEqual(count, 0)

    def test_export_tmx_roundtrip(self):
        from cat_tool.glossary import Glossary
        import xml.etree.ElementTree as ET
        self.gl.add_term("hello", "hola", source_lang="en", target_lang="es")
        self.gl.add_term("goodbye", "adiós", source_lang="en", target_lang="es")
        tmx_path = os.path.join(self.tmp.name, "roundtrip.tmx")
        count = self.gl.export_tmx(tmx_path)
        self.assertEqual(count, 2)

        self.gl2 = Glossary(os.path.join(self.tmp.name, "glossary2.db"))
        count2 = self.gl2.import_tmx(tmx_path)
        self.assertEqual(count2, 2)
        terms = self.gl2.get_all_terms()
        self.assertEqual(len(terms), 2)
        self.assertEqual(terms[0]["source_lang"], "en")
        self.assertEqual(terms[0]["target_lang"], "es")

    def test_export_tmx_source_lang_override(self):
        import xml.etree.ElementTree as ET
        self.gl.add_term("bonjour", "hello", source_lang="fr", target_lang="en")
        tmx_path = os.path.join(self.tmp.name, "override.tmx")
        self.gl.export_tmx(tmx_path, source_lang="fr")
        tree = ET.parse(tmx_path)
        root = tree.getroot()
        header = root.find("header")
        self.assertEqual(header.get("srclang"), "fr")

if __name__ == "__main__":
    unittest.main()

```